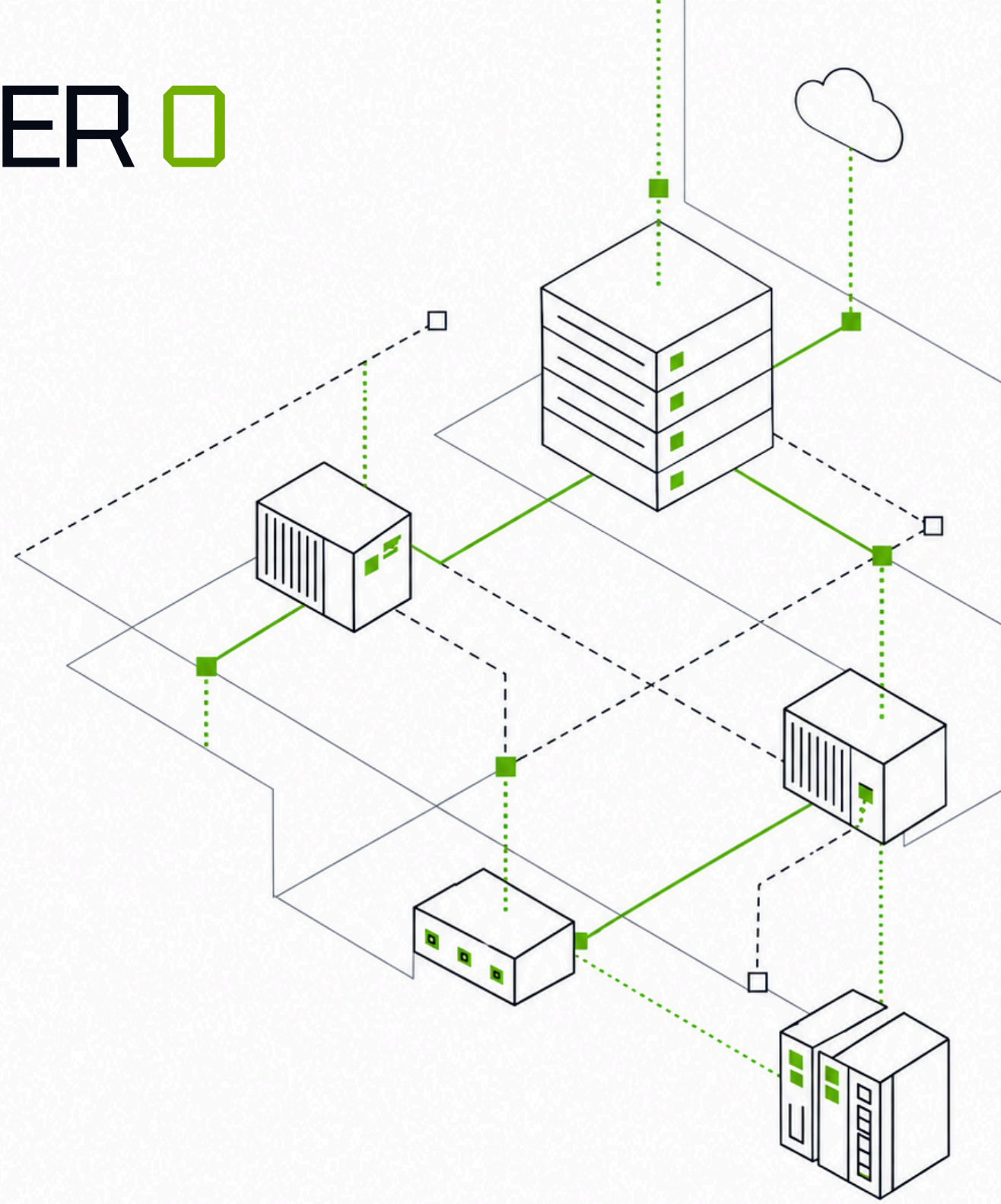
TIER 0



Tier0 Enterprise

User's Manual

Table of Contents

Overview

- What is Tier0?
- Feature List
- User Journey

Quick Start Guide

- Deploy Tier0
- High Availability Deployment Plan
- Standard Deployment Plan
- Login

How-to Guide

Connect Data

- Build Data Models
- Connect Data Sources
- Common Data Sources
- Connecting OPC UA
- Connecting OPC DA
- Connecting Modbus
- Connecting MQTT
- Connecting File
- Connecting RestAPI
- Connect Data via MQTT

Process Data

- Process Data
- Common Data Processing Methods
- Combining Multiple Sources
- Filtering Data
- Changing Data
- Spliting Data
- Visualize Data
- Work with UNS Agent

Advanced Guide

Sync Data
Sync Data between Center and Edge
Obtain Data from Tier0
Use Data
Industrial Apps
Configure 3D Models
Build 3D Scenes
Analyze Data
Analyze Data
Visual Analytics
System Settings
Glossary

Overview

What is Tier0?

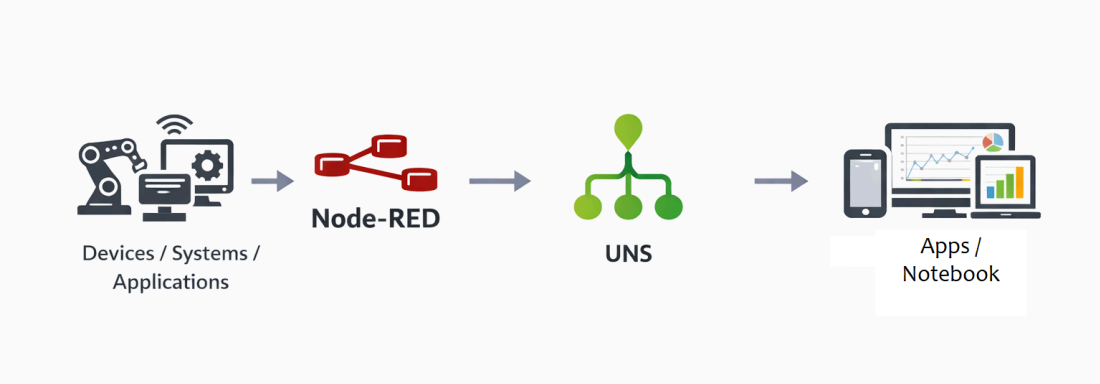
Tier0 is a Unified Namespace (UNS)-based industrial data platform.

It connects data through Node-RED, organizes it into a unified structure, and makes it available for applications and analytics.

How It Works?

Tier0 moves and structures data in a single workflow:

- According to Unified Namespace (UNS) concept, build data models with clear structures.
- Node-RED connects and processes data from devices and systems
- Data is published into the Unified Namespace (UNS) data models and stored in Tier0 database
- Data is consumed by dashboards, applications and analytics

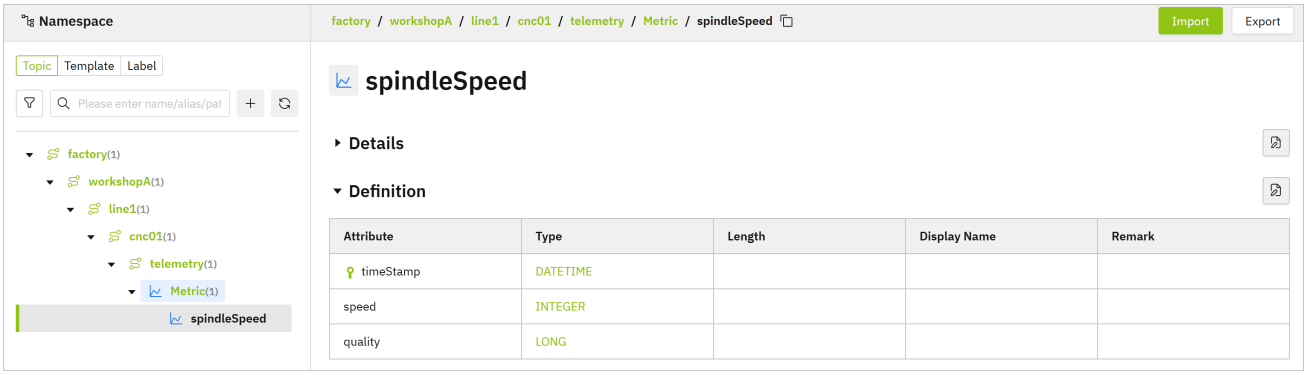


Key Components

Unified Namespace (UNS)

Defines a hierarchical structure for all industrial data.

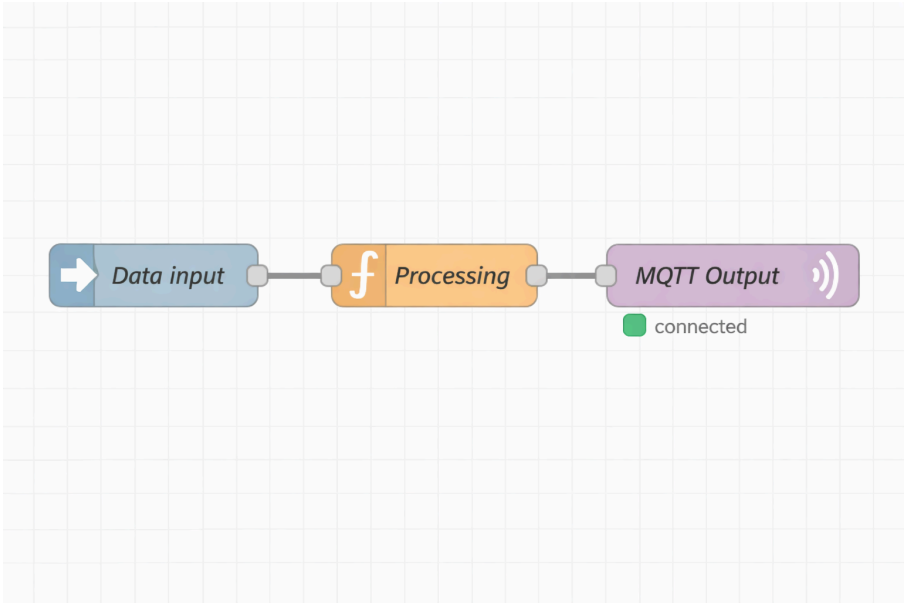
`factory/workshopA/line1/cnc01/telemetry/spindleSpeed`



Node-RED Integration

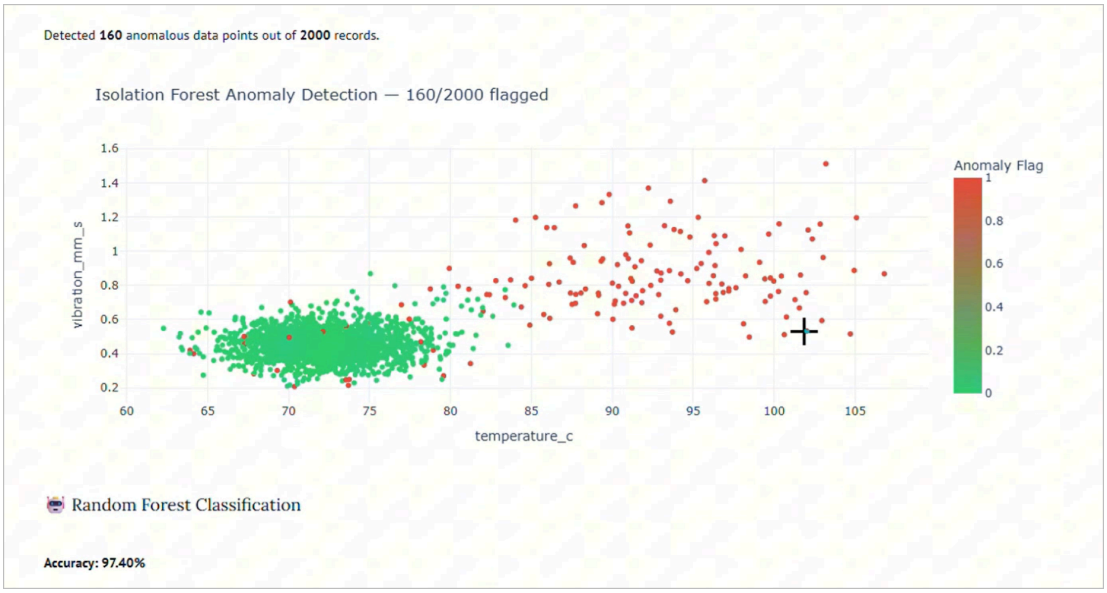
The core engine for data connection and processing.

- Connect devices, applications, and systems
- Format and publish data into the namespace



Python Notebook

Analyze data with full flexibility using Python with an interactive environment.

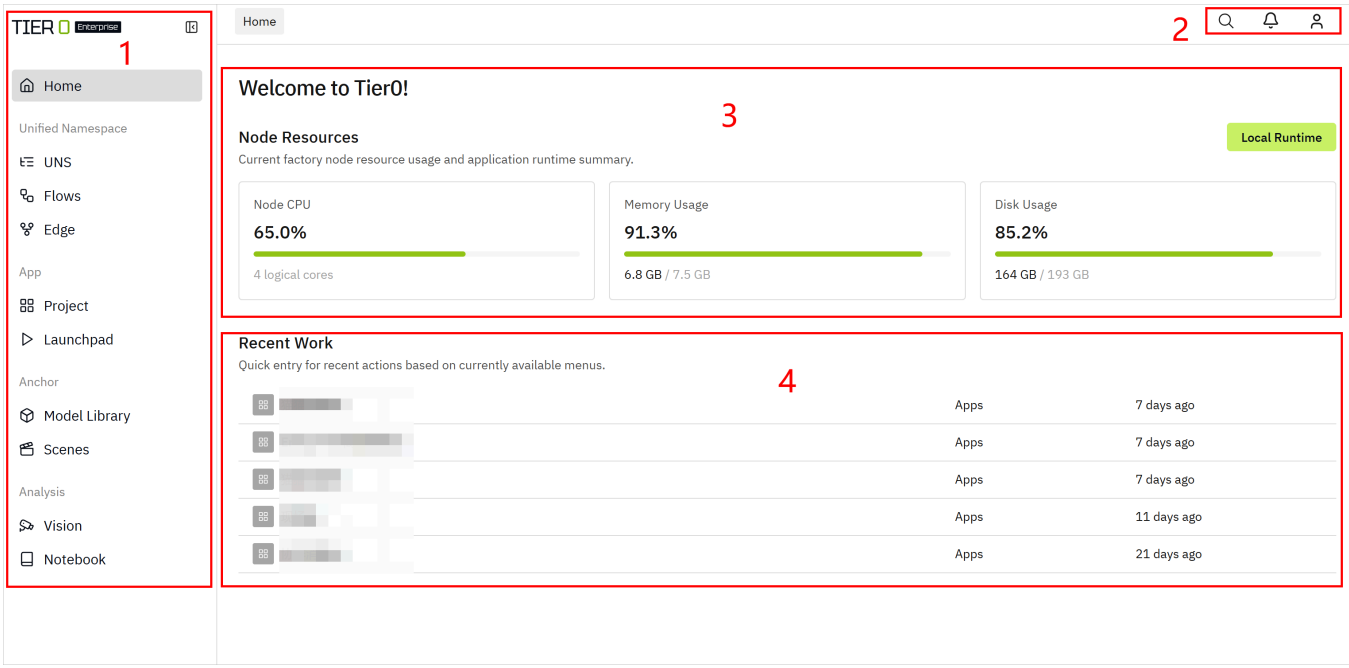


Agentic App Builder

Create industrial applications using natural language without consuming any of your local resources. The whole process is done on cloud, and provide a package for local installation.



Feature List



Homepage

No.	Name	Description
1	Menu Bar	Lists the function menus of the platform, enabling quick access to various function modules.
2	Tool Bar	Common operations on the platform, including search, notifications, and system settings.
3	Node Resources	Displays the resources of the current Tier0 node. **Local Runtime** shows the runtime of all installed applications.
4	Recent Work	List of recent actions, providing quick accesses to recent work.

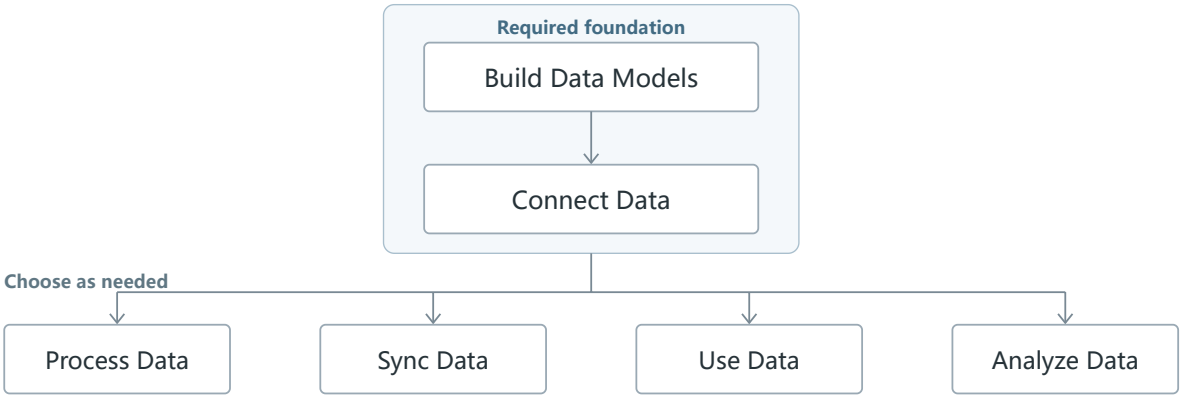
Function Module Description

Module	Name	Description
Unified Namespace	UNS	Models industrial data into a hierarchical structure (folder-file) for clarity.
	Flows	Uses Node-RED to define the data source of the created model.
	Edge	Connect MQTT clients or Tier0 edge nodes for data management.
App	Project	Categorizes and manages imported applications built on Tier0 Cloud.
	Launchpad	Displays all launched applications you have access to.

Module	Name	Description
Anchor	Model Library	Digital twin models that connect with UNS data.
	Scenes	Visualizes equipment and production scenarios in your plant.
Analysis	Vision	Apply visual analytics to your live camera images.
	Notebook	Adopts Marimo Notebook for easy python script usage. For example, train models, run SQL queries, interactive data reporting and others.

User Journey

Start with the two required steps shown in the diagram:



- **Necessary**
    i. **Build Data Models** to organize your assets and define where data belongs.
    ii. **Connect Data** to bring data from your sources into those models.
- **Optional**
    - **Process Data** to transform and prepare data with **Event Flow**.
    - **Sync Data** to share data with other Tier0 or make it available through OpenAPI and MQTT.
    - **Use Data** in applications and business scenarios.
    - **Analyze Data** to explore and visualize what the data reveals.

Quick Start Guide

Deploy Tier0

Requirements

- Single-Node Deployment
  - Minimum: 4C / 8G / 100G
  - Recommended: 8C / 16G / 500G
- Standard Deployment
  - Minimum: 4 VMs
  - Required baseline per VM types:
    - App: 8C / 32G / 100G
    - Database: 4C / 8G / 500G
    - Witness: 2C / 4G / 40G
- High Availability Deployment
  - Minimum: 6 VMs
  - Required baseline per VM types:
    - App (Swarm): 8C / 32G / 100G x 3
    - Database: 4C / 8G / 500G x 2
    - Witness: 2C / 4G / 40G

> **HARDWARE REQUIREMENTS MAY VARY**
>
> Enterprise hardware requirements vary by deployment scenarios.

> **NOTE**
>
> Detailed deployment information is available in the Standard Deployment Plan and the High Availability Deployment Plan.

Operations

1. Get the package from your vendor, and verify it.

```
sha256sum -c tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz.sha256
```

2. Decompress the package.

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. Edit `install.env` based on the deployment details, then start the deployment.

```
vi install.env
bash install.sh
```

Common Maintenance Operations

- Check the deployment environment without installing Enterprise.

```
./install.sh --check-only
```

- Clear all nodes after a failed deployment.

```
./full-cleanup.sh --env install.env --yes
```

> ⚠ **WHAT DIFFERENCE IT MAKES**
>
> This command wipes all Tier0 runtime data and database backups while keeping the VMs, operating system, and installed Docker software.

- A deployment report is generated after installation.

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> ⚠ **WHAT YOU SHOULD DO WHEN FAIL APPEARS IN THE REPORT**
>
> Do not deliver the project if the report contains any **FAIL** item. Contact support first.

High Availability Deployment Plan

Plan Objective

This plan deploys one Tier0 Enterprise environment in an offline or private deployment environment. It uses 6 virtual machines in total:

- 3 application nodes: form the `Docker Swarm` cluster base and run the single-replica Tier0 Enterprise service and the stateful **EMQX** service.
- 2 database nodes: form the `PostgreSQL + TimescaleDB` primary-standby cluster.
- 1 `Witness` node: participates in database monitoring, arbitration, and failover.

OPTIONAL

- An external `NFS/NAS` shared storage can be added and mounted by the three Swarm nodes. It is used for Tier0 Enterprise shared files across nodes.
- This shared storage is an external dependency and is not counted as one of the 6 business VMs in this plan.

INFO

This plan assumes that each database VM has `4C / 8G / 500G`. If the user requirement is `4C / 8G / 500G` for the whole database cluster, performance and capacity must be recalculated before implementation.

Planned IP Addresses

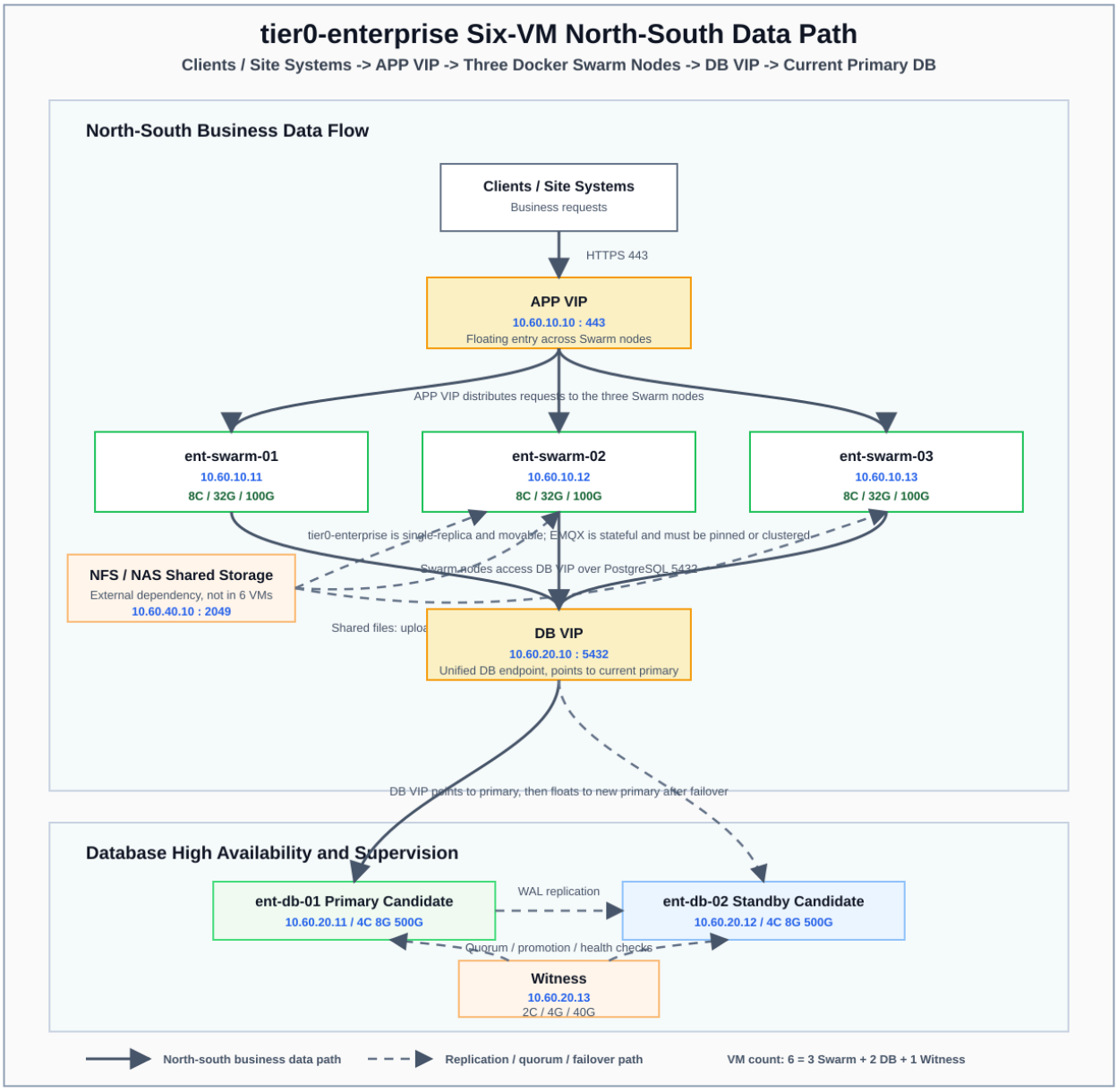
INFO

The following IP addresses are placeholders. Replace them with the actual on-site network addresses before implementation.

Type	Name	Planned IP	Description
Logical entry	APP VIP	10.60.10.10	Unified business entry. It can fail over between the 3 Swarm nodes.
External dependency	NFS/NAS	10.60.40.10	Optional shared storage. It is not counted as one of the 6 VMs.
Application node	ent-swarm-01	10.60.10.11	Docker Swarm manager/worker. Runs services that can be rescheduled. It can also be used as the fixed EMQX node or as an EMQX cluster node.
Application node	ent-swarm-02	10.60.10.12	Docker Swarm manager/worker. Runs services that can be rescheduled. It can also be used as an EMQX cluster node.
Application node	ent-swarm-03	10.60.10.13	Docker Swarm manager/worker. Runs services that can be rescheduled. It can also be used as an EMQX cluster node.
Logical entry	DB VIP	10.60.20.10	Unified database entry. It only points to the current primary database.
Database node	ent-db-01	10.60.20.11	PostgreSQL + TimescaleDB primary candidate.
Database node	ent-db-02	10.60.20.12	PostgreSQL + TimescaleDB standby candidate.

Type	Name	Planned IP	Description
Arbitration node	ent-db-witness-01	10.60.20.13	repmgr witness / arbitration and monitoring node.

Overall Topology



> **INFO**
>
> The diagram only shows the core links.

- Business access path: users / on-site systems -> `APP VIP` -> three `Docker Swarm` nodes -> `DB VIP` -> current primary database.

- The topology is arranged from top to bottom to show request entry, application processing, database access, and high availability monitoring.
- The topology diagram does not show internal cluster ports, SSH O&M links, or monitoring links. These are described in the Connection List and Detailed Port List.
- `Witness` only participates in database arbitration, primary selection, and health checks. It is not in the business access path.
- `APP VIP` distributes traffic to the three Swarm nodes. The Swarm nodes access the database through `DB VIP`.
- `APP VIP` can fail over between the three Swarm nodes. `DB VIP` follows the current PostgreSQL Primary role and can fail over between the two database nodes.
- Tier0 Enterprise runs as a single-replica service. If the current node fails, Swarm reschedules and restarts it on another healthy node.
- `EMQX` is a stateful service. It should not be randomly rescheduled like a normal stateless service. By default, it should be fixed to one node with persistent storage. If high availability is required for the messaging layer, deploy a three-node EMQX cluster.
- The optional `NFS/NAS` is only used as shared file storage for the Swarm nodes. It is not part of the main database request path.

Database Monitoring and Failover

Database failover path:

Stage	Link	Description
Normal access	Tier0 Enterprise replica on a Swarm node -> `DB VIP` -> current primary database	The application always connects only to `10.60.20.10:5432`.
Normal replication	Current primary database -> current standby database	Data is synchronized through PostgreSQL WAL streaming replication.
Normal monitoring	`DB-01` / `DB-02` -> `Witness`	`repmgrd` reports status, and `Witness` participates in arbitration.
Primary database failure	Standby database -> `Witness`	The standby database requests arbitration to avoid split-brain.
Standby promotion	Standby database `promote`	After `Witness` confirms the failover, the standby database is promoted to the new primary database.
VIP failover	`DB VIP` -> new primary database	`10.60.20.10` is bound to the new primary database.
Application recovery	Tier0 Enterprise replica on a Swarm node -> `DB VIP` -> new primary database	The application connection pool reconnects, and read/write access is restored.

Key boundaries:

- `Witness` does not store business data, does not handle business read/write traffic, and does not expose database access to applications.
- Business services only connect to `DB VIP:5432`. Direct connections to `ent-db-01/02` are not allowed.

- `DB VIP` must be bound to the current PostgreSQL Primary node. It must not fail over based only on whether the database process is alive.
- `TimescaleDB` is deployed as a PostgreSQL extension on both database nodes. It is not separated into an independent database instance.
- The recommended design is `PostgreSQL streaming replication + repmgr + repmgr witness + keepalived DB VIP`.
- Automatic failover only handles primary database unavailability. After the original primary database recovers, it should rejoin as a standby database first. It should not automatically take the primary role back.

Node Configuration List

Node	Role	vCPU	Memory	Storage	System Disk Recommendation	Data Disk Recommendation	Main Components
`ent-swarm-01`	Swarm manager/worker	8	32G	100G	Single system disk / 100G	No separate data disk	Docker Engine, Swarm, Keepalived, HAProxy/Nginx
`ent-swarm-02`	Swarm manager/worker	8	32G	100G	Single system disk / 100G	No separate data disk	Docker Engine, Swarm, Keepalived, HAProxy/Nginx
`ent-swarm-03`	Swarm manager/worker	8	32G	100G	Single system disk / 100G	No separate data disk	Docker Engine, Swarm, Keepalived, HAProxy/Nginx
`ent-db-01`	PostgreSQL/TSDB primary candidate	4	8G	500G	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-02`	PostgreSQL/TSDB standby candidate	4	8G	500G	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-witness-01`	Witness / arbitration and monitoring	2	4G	40G	`/` 40G	No separate data disk	repmgr witness, monitoring agent, audit logs

Configuration Summary

Category	Quantity	Specification per Node	Subtotal vCPU	Subtotal Memory	Subtotal Storage
Docker Swarm application nodes	3	8C / 32G / 100G	24C	96G	300G
PostgreSQL + TimescaleDB nodes	2	4C / 8G / 500G	8C	16G	1000G

Category	Quantity	Specification per Node	Subtotal vCPU	Subtotal Memory	Subtotal Storage
Witness node	1	2C / 4G / 40G	2C	4G	40G
NFS/NAS shared storage	1	Existing on-site storage or separately provided storage	Not counted	Not counted	Plan based on file volume
Total	6	-	34C	116G	1340G

Converted hardware requirements:

- Total VMs: 6
- Total vCPU: 34
- Total memory: 116G
- Total storage: 1.34T
- Optional external shared storage: not counted in the 6 VM total. Capacity should be planned separately based on attachments, reports, import/export files, and log archive retention.

Swarm Service Runtime Rules

Service	Runtime Mode	State Type	Failure Handling	Description
Tier0 Enterprise	`replicas: 1`	Stateless or weakly stateful	If the current node fails, Swarm reschedules and restarts the service on another node.	Data must be stored in PostgreSQL/TimescaleDB. Do not rely on local container files.
EMQX single-node mode	Fixed to a specified Swarm node	Stateful	If the fixed node fails, restore the node or recover from backup.	Suitable for small-scale scenarios where a single point of failure in the messaging layer is acceptable.
EMQX three-node cluster mode	Run one fixed EMQX instance on each of the three Swarm nodes	Stateful cluster	If one EMQX node fails, the remaining EMQX nodes continue to serve traffic.	Use this mode first if high availability is required for the messaging layer.
APP VIP / HAProxy / Nginx	Deployed on three nodes, with VIP failover	Runtime configuration	A healthy node takes over the VIP.	Only used as an entry point. It does not store business data.
NFS/NAS shared directory	Mounted by all three Swarm nodes	File state	If NFS fails, functions that depend on shared files are affected.	Only used for shared files such as uploads, imports, exports, and reports.

Key rules:

- Tier0 Enterprise can run as a single replica. Swarm only provides the cluster base and failover rescheduling capability.
- `EMQX` cannot use the same random single-replica rescheduling strategy as Tier0 Enterprise, because it has sessions, subscriptions, retained messages, rule engine state, and persistent directories.

- If `EMQX` is deployed as a single node, it must be treated as a single point of failure in the messaging layer and fixed to one Swarm node with a persistent directory.
- If the business cannot accept a single point of failure in `EMQX`, deploy a three-node EMQX cluster on the three Swarm nodes. Each node must have a fixed hostname, node name, and persistent directory.

NFS/NAS Shared Storage Rules

NFS is introduced so that the Tier0 Enterprise single replica can still access the same business files after it is rescheduled between Swarm nodes. It does not change the boundaries of database primary-standby replication or the stateful EMQX service.

Suitable content for NFS:

- User-uploaded attachments
- Import/export files
- Report files
- Offline packages, templates, and reproducible intermediate files
- Business static files that must be read across nodes

Content not recommended for NFS:

- PostgreSQL / TimescaleDB data directories and WAL directories
- EMQX core data directories, Mnesia data, and runtime state
- Docker Engine data directory `/var/lib/docker`
- High-frequency database-like writes, lock-sensitive files, and strongly consistent queues

Recommended mount rules:

Item	Recommendation
NFS address	`10.60.40.10:/exports/tier0-enterprise`
Swarm mount point	`/mnt/tier0-shared`
Container mount point	`/app/data/shared` or based on application configuration
Protocol version	Prefer NFSv4.1 or the standard version used on site
Mount options	`rw,hard,timeo=600,retrans=2,_netdev,noatime`. Adjust based on on-site NAS compatibility.
Permissions	Use a fixed business UID/GID to avoid permission differences after containers move between nodes.
Backup	Include the shared directory in NAS snapshots or independent backups.

NFS boundaries:

- NFS is a shared file dependency. It is not a database high availability component.

- If NFS itself is a single point of failure, file-related functions will also have a single point of failure. In production, use an existing HA NAS or dual-controller storage.
- Before the Tier0 Enterprise single replica is rescheduled to another Swarm node, the target node must have NFS mounted and verified for read/write access.
- If Tier0 Enterprise does not write local files at all, NFS is not required.

Connection List

O&M Access

Target	Method	Port	Description
All VMs	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	Use after unified passwordless SSH is configured. Before that, use the on-site account.
APP VIP	`https://10.60.10.10`	`443/TCP`	Business entry
APP VIP	`http://10.60.10.10`	`80/TCP`	Optional. Recommended only for redirecting to HTTPS.
DB VIP	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	Only application nodes and O&M allowlists can access it.
NFS/NAS	`10.60.40.10:/exports/tier0-enterprise`	`2049/TCP`	Optional shared file storage

Swarm Cluster Communication

Source	Target	Port	Purpose
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`2377/TCP`	Swarm manager control plane
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`7946/TCP/UDP`	Node discovery and gossip
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`4789/UDP`	Overlay network VXLAN
`APP VIP`	`ent-swarm-01/02/03`	`80/443/TCP`	Business entry forwarding

EMQX Access and Cluster Communication

Source	Target	Port	Purpose
Devices / on-site systems	`APP VIP` or `EMQX` entry	`1883/TCP`	MQTT plaintext access. Open based on on-site security requirements.

Source	Target	Port	Purpose
Devices / on-site systems	`APP VIP` or `EMQX` entry	`8883/TCP`	MQTT TLS access. Recommended first.
Devices / on-site systems	`APP VIP` or `EMQX` entry	`8083/TCP`	MQTT WebSocket. Optional.
Devices / on-site systems	`APP VIP` or `EMQX` entry	`8084/TCP`	MQTT WebSocket TLS. Optional.
O&M network segment	`EMQX` management entry	`18083/TCP`	EMQX Dashboard. Only allow O&M allowlists.
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`4369/TCP`	EMQX/Erlang EPMD. Required only in three-node EMQX cluster mode.
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`4370/TCP`	EMQX node-to-node communication. Required only in three-node EMQX cluster mode.
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`5370/TCP`	EMQX cluster RPC. Confirm based on the actual EMQX version.

Database and Monitoring Communication

Source	Target	Port	Purpose
Tier0 Enterprise container	`DB VIP`	`5432/TCP`	Business database access
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL streaming replication
`ent-db-02`	`ent-db-01`	`5432/TCP`	Reverse replication after failover recovery
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr metadata / witness communication
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP failover. Protocol 112 must be allowed.
`ent-swarm-01/02/03`	`ent-db-01/02/Witness`	`9100/TCP`	Optional node exporter monitoring

NFS/NAS Shared Storage Communication

Source	Target	Port	Purpose
`ent-swarm-01/02/03`	`NFS/NAS 10.60.40.10`	`2049/TCP`	NFSv4 shared file access
O&M network segment	`NFS/NAS 10.60.40.10`	Based on on-site management ports	NAS management, snapshots, and backup

Service Deployment Recommendations

Docker Swarm

- Set all three application nodes as `manager + worker`.
- Deploy Tier0 Enterprise by using `docker stack deploy`.
- Tier0 Enterprise uses `replicas: 1` by default. Swarm reschedules it to another healthy node after a node failure.
- Tier0 Enterprise containers do not store critical persistent data. Business data is stored in PostgreSQL/TimescaleDB.
- Each Swarm node has only `100G` storage and uses it as a single system disk. The Docker data directory stays on the system disk. No separate data disk is planned.
- Use `Docker config` for configuration files.
- Use `Docker secret` for passwords, database credentials, and certificate private keys.
- Use an on-site internal registry for container images to avoid public network dependencies during deployment.
- Configure health checks when publishing services to avoid forwarding traffic to containers that are not ready.
- If the application needs shared files across nodes, mount the shared directory from NFS/NAS. Do not rely on local temporary container directories for critical files.
- Configure cleanup policies for images, container logs, and build caches to avoid filling `/var/lib/docker`.

Single-replica deployment example:

```
deploy:
  replicas: 1
  restart_policy:
    condition: any
  update_config:
    order: stop-first
```

EMQX Stateful Service

`EMQX` must be handled as a stateful service. It must not be randomly rescheduled like a normal stateless Swarm replica.

Mode	Recommended Scenario	Deployment Rule	Risk
Single-node fixed mode	Small-scale scenarios where short messaging-layer downtime is acceptable	Fix it to `ent-swarm-01`. Use a local persistent directory and backup.	If `ent-swarm-01` fails, EMQX cannot fail over automatically without data loss.
Three-node cluster mode	Production scenarios where a single point of failure in EMQX is not acceptable	Run one EMQX node on each of the three Swarm nodes. Use fixed hostnames, node names, and data directories.	Requires additional configuration for cluster ports, certificates, node discovery, and monitoring.

Minimum requirements for single-node fixed mode:

- Use a Swarm placement constraint to fix the node, for example `node.labels.emqx == true`.

- Fix the data directory to the host, for example `/data/emqx`.
- Include configurations, certificates, plugins, and rule engine configurations in backup.
- If the node fails, restore the original node first. If it cannot be restored, recover to a new node from backup. Do not handle it as a stateless random failover.

Minimum requirements for three-node cluster mode:

- Fix one EMQX instance on each Swarm node.
- Use a fixed node name for each EMQX node. The node name must not change when the container is recreated.
- Open cluster communication ports only between the three Swarm nodes.
- Expose the Dashboard only to the O&M network segment.
- MQTT access can be exposed through `APP VIP`, a separate MQTT VIP, or a layer 4 load balancer. Confirm the entry method before implementation.
- Do not place the EMQX core data directory on NFS. In single-node fixed mode, use the fixed node local disk and backup it properly. In three-node mode, rely on the EMQX cluster mechanism.

NFS/NAS Shared Storage

- NFS/NAS is an optional external dependency and does not count as one of the six VMs.
- Mount the same shared directory on all three Swarm nodes.
- Use it only for shared files that Tier0 Enterprise needs to access across nodes.
- Do not use it for PostgreSQL/TimescaleDB data directories.
- Do not use it for EMQX core runtime data.
- Configure a pre-start mount check to avoid writing to a local directory after a service moves to a node where NFS is not mounted.
- If NFS fails, database read/write access is not directly affected, but functions such as uploads, imports, exports, and reports may fail.

APP VIP

- Deploy `APP VIP` on the three Swarm nodes. Use `keepalived + HAProxy` or `keepalived + Nginx`.
- The VIP is only the unified entry. It does not require a separate VM.
- Health checks should use the application `/health` or `/ready` endpoint.
- If any Swarm node fails, the VIP moves to another healthy node.

PostgreSQL + TimescaleDB

Recommended baseline:

- PostgreSQL: `16` or a version certified on site.
- TimescaleDB: choose a stable version that matches the PostgreSQL major version.
- Primary-standby replication: physical streaming replication.
- Failover: `repmgrd + witness`.
- Database entry: `DB VIP`.
- Connection pool: preferably use `PgBouncer` on the application side or in a separate container to avoid too many database connections.

Recommended starting parameters. Adjust them based on business pressure tests during implementation.

Parameter	Recommended Value
`shared_buffers`	`2GB`
`effective_cache_size`	`6GB`
`work_mem`	Start from `16MB`
`maintenance_work_mem`	`512MB`
`max_connections`	Keep within `200`. Prefer using a connection pool.
`wal_level`	`replica`
`max_wal_senders`	`10`
`max_replication_slots`	`10`
`hot_standby`	`on`
TimescaleDB compression	Enable it based on the retention period of time-series tables.

Witness

- The `Witness` node only stores `repmgr witness` metadata and helps database failure detection and arbitration.
- `Witness` does not store business data, does not handle business read/write traffic, and does not expose database access to applications.
- `Witness` needs stable network access. It does not require high storage performance.
- A `Witness` failure should not stop current business access, but it lowers the reliability of automatic failover.
- After `Witness` recovers, verify that it has rejoined the database cluster state.

Port and Firewall Baseline

Port/Protocol	Allowed Range	Purpose
`22/TCP`	O&M network segment to the 6 VMs	SSH
`80/TCP`	User network segment to `APP VIP`	HTTP redirect or internal plaintext entry
`443/TCP`	User network segment to `APP VIP`	HTTPS business entry
`2377/TCP`	Communication between the 3 Swarm nodes	Swarm management plane
`7946/TCP/UDP`	Communication between the 3 Swarm nodes	Swarm gossip

Port/Protocol	Allowed Range	Purpose
`4789/UDP`	Communication between the 3 Swarm nodes	Swarm overlay
`1883/TCP`	Devices / on-site systems to MQTT entry	EMQX MQTT plaintext. Optional.
`8883/TCP`	Devices / on-site systems to MQTT entry	EMQX MQTT TLS. Recommended.
`8083/TCP`	Devices / on-site systems to MQTT WebSocket entry	EMQX WebSocket. Optional.
`8084/TCP`	Devices / on-site systems to MQTT WebSocket TLS entry	EMQX WebSocket TLS. Optional.
`18083/TCP`	O&M network segment to EMQX management entry	EMQX Dashboard. Allowlist only.
`5432/TCP`	Swarm nodes to `DB VIP`; internal communication between DB/Witness nodes	PostgreSQL / repmgr
`2049/TCP`	Swarm nodes to NFS/NAS	NFSv4 shared file access. Optional.
`112/VRRP`	Between Swarm nodes and between DB nodes	VIP failover
`9100/TCP`	Monitoring system to the 6 VMs	node exporter. Optional.

Detailed Port List

External Entry Ports

Port/Protocol	Listening Location	Access Source	Purpose	Required
`80/TCP`	`APP VIP 10.60.10.10`	User network segment / O&M network segment	HTTP entry or redirect to HTTPS	Optional
`443/TCP`	`APP VIP 10.60.10.10`	User network segment / O&M network segment	HTTPS business entry	Yes
`1883/TCP`	MQTT entry	Devices / on-site systems	EMQX MQTT plaintext	Optional
`8883/TCP`	MQTT entry	Devices / on-site systems	EMQX MQTT TLS	Recommended
`8083/TCP`	MQTT WebSocket entry	Devices / on-site systems	EMQX WebSocket	Optional
`8084/TCP`	MQTT WebSocket TLS entry	Devices / on-site systems	EMQX WebSocket TLS	Optional
`18083/TCP`	EMQX Dashboard	O&M allowlist	EMQX management console	Optional. Source must be restricted.

Port/Protocol	Listening Location	Access Source	Purpose	Required
`22/TCP`	Management IPs of the 6 VMs	O&M network segment	SSH O&M login	Yes
`5432/TCP`	`DB VIP 10.60.20.10`	3 Swarm nodes and O&M allowlist	PostgreSQL business access	Yes
`2049/TCP`	`NFS/NAS 10.60.40.10`	3 Swarm nodes	NFSv4 shared file access	Depends on whether NFS is used

Docker Swarm Node Ports

Port/Protocol	Listening Node	Access Source	Purpose	Required
`2377/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm cluster management plane	Yes
`7946/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm node discovery / gossip	Yes
`7946/UDP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm node discovery / gossip	Yes
`4789/UDP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm overlay VXLAN	Yes
`80/TCP`	`ent-swarm-01/02/03`	`APP VIP` / internal LB	Application HTTP backend	Depends on deployment method
`443/TCP`	`ent-swarm-01/02/03`	`APP VIP` / internal LB	Application HTTPS backend	Depends on deployment method
`4369/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	EMQX/Erlang EPMD, three-node cluster mode	Depends on EMQX mode
`4370/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	EMQX node-to-node communication, three-node cluster mode	Depends on EMQX mode
`5370/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	EMQX cluster RPC. Confirm based on version.	Depends on EMQX mode

Database Cluster Ports

Port/Protocol	Listening Node	Access Source	Purpose	Required
`5432/TCP`	`ent-db-01`	`ent-swarm-01/02/03`, `ent-db-02`, `ent-db-witness-01`	Primary database business access, replication, and repmgr metadata	Yes
`5432/TCP`	`ent-db-02`	`ent-swarm-01/02/03`, `ent-db-01`, `ent-db-witness-01`	Business access after standby promotion, replication, and repmgr metadata	Yes
`5432/TCP`	`ent-db-witness-01`	`ent-db-01/02`	repmgr witness communication	Yes
`873/TCP`	`ent-db-01/02`	O&M backup node	rsync backup channel	Optional
`6432/TCP`	PgBouncer container or database node	Tier0 Enterprise container	PostgreSQL connection pool entry	Optional

VIP and Monitoring Ports

Port/Protocol	Listening Node	Access Source	Purpose	Required
`112/VRRP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	APP VIP failover	Yes, if Keepalived is used
`112/VRRP`	`ent-db-01/02`	`ent-db-01/02`	DB VIP failover	Yes, if Keepalived is used
`9100/TCP`	6 VMs	Monitoring system	node exporter	Recommended
`9187/TCP`	`ent-db-01/02`	Monitoring system	postgres exporter	Recommended
`9323/TCP`	`ent-swarm-01/02/03`	Monitoring system	Docker metrics. Must be explicitly enabled.	Optional
EMQX metrics port	EMQX node	Monitoring system	EMQX metrics. Confirm based on version and configuration.	Recommended

Ports Not Open by Default

Port/Protocol	Description
`2375/TCP`	Docker plaintext remote API. Do not open it.
`2376/TCP`	Docker TLS remote API. Do not open it by default. If it must be used, allow only the O&M network segment.

Port/Protocol	Description
Database node `80/443`	Database nodes do not host Web entries. Keep these ports closed by default.
Witness `5432` to application network segment	Witness does not handle business SQL. Application access is not allowed.
EMQX Dashboard to user network segment	`18083/TCP` can only be opened to the O&M allowlist.
NFS to non-Swarm network segments	`2049/TCP` can only be accessed by the three Swarm nodes and backup/O&M allowlists.

High Availability Capabilities and Failure Boundaries

Failure Scenario	Expected Behavior	Business Impact
A single Swarm node fails and affects the Tier0 Enterprise single replica	Swarm reschedules the single replica to another healthy node. APP VIP continues to point to a healthy entry.	Seconds to minutes of interruption, depending on image pulling and health checks.
The EMQX node fails in single-node fixed mode	Restore the original node first. If it cannot be restored, recover to a new node from backup.	The messaging layer is interrupted. It cannot fail over without data loss like a stateless service.
One node fails in the three-node EMQX cluster	The remaining EMQX nodes continue to serve traffic. The failed node rejoins after recovery.	Connections reconnect, and capacity is reduced.
NFS/NAS fails	Functions that depend on shared files fail. The database and business processes already in memory may not be interrupted.	File-related functions such as uploads, imports, exports, and reports are affected.
Two Swarm nodes fail	The Swarm managers lose quorum. Running containers may continue, but reliable scheduling is not available.	High risk. Manual recovery is required.
The current primary database fails	The standby database is promoted after Witness arbitration, and DB VIP fails over.	Seconds to minutes of write interruption. The application must reconnect.
The standby database fails	The primary database continues to serve traffic, and a replication interruption alert is generated.	No direct business interruption, but the database hot standby is lost.
Witness fails	The current primary database continues to serve traffic, but automatic failover capability is reduced.	No direct business interruption. Restore Witness first.
DB VIP failover fails	The database has switched to a new primary, but the entry is unreachable.	Business is interrupted. Bind the VIP manually.

Backup, Monitoring, and Inspection

Minimum delivery requirements:

- Perform a full database backup every day and keep backups for at least `7` days.
- Enable WAL archiving or incremental backups to meet the on-site RPO.
- Perform a restore drill every month to verify that backups are usable.
- Monitor Swarm service health checks, container restart counts, node disk usage, memory, and CPU.
- Monitor EMQX connections, subscriptions, message throughput, queue backlog, rule engine failures, and persistent directory usage.
- Monitor NFS availability, mount status, capacity, inode usage, and read/write latency.
- Monitor PostgreSQL primary-standby delay, replication slots, WAL backlog, connections, slow SQL, and database bloat.
- Include the current VIP owner, repmgr cluster status, and Witness status in routine inspections.

Recommended inspection commands:

```
docker node ls
docker service ls
docker stack ps tier0-enterprise
ssh ent-db-01 'repmgr cluster show'
ssh ent-db-01 'psql -h 10.60.20.10 -p 5432 -c "select now();"'
ssh ent-db-01 'ip addr show | grep 10.60.20.10'
```

Pre-Go-Live Checklist

Check Item	Pass Criteria
Hostnames of the 6 VMs are fixed	`hostnamectl` matches the plan.
Unified SSH is available	The O&M terminal can access all 6 VMs without a password.
Time is synchronized	The time difference between the 6 VMs is less than `1s`.
DNS/hosts are fixed	Node names can be resolved stably.
Firewall rules are applied	Ports in Section 9 are opened according to the required ranges.
Docker Swarm is normal	`docker node ls` shows all three nodes as `Ready`.
Swarm disk usage is normal	The `/` usage of all three Swarm nodes is lower than `70%`. Docker log and image cache cleanup policies are configured.
Tier0 Enterprise single replica is normal	`docker service ls` shows `1/1`, and one node failure rescheduling test has been completed.
EMQX status is confirmed	In single-node fixed mode, the persistent directory and backup are confirmed. In three-node cluster mode, the cluster health is confirmed.
NFS mount is confirmed	All three Swarm nodes can read and write `/mnt/tier0-shared`, and automatic mount after restart is verified.

Check Item	Pass Criteria
DB primary-standby status is normal	`repmgr cluster show` shows that primary, standby, and witness are normal.
DB VIP is normal	The application nodes can connect to `10.60.20.10:5432`.
APP VIP is normal	The user network segment can access `https://10.60.10.10`.
Primary database failure drill is completed	The standby database can be promoted, and DB VIP can fail over.
Application node failure drill is completed	APP VIP and the Swarm replica can recover.
Backup restore drill is completed	Data can be restored from backup to a test database.

Implementation Sequence

1. Create 6 VMs and fix hostnames, IP addresses, time synchronization, and SSH access.
2. Initialize system baselines, firewall rules, disk mounts, and kernel parameters.
3. If NFS/NAS is used, first complete shared directory setup, permissions, mounting, and restart auto-mount verification.
4. Deploy the three-node Docker Swarm.
5. Deploy `APP VIP + HAProxy/Nginx`.
6. Deploy EMQX in either single-node fixed mode or three-node cluster mode.
7. Deploy PostgreSQL + TimescaleDB primary-standby.
8. Deploy repmgr witness, repmgrd, and the DB VIP failover script.
9. Publish the Tier0 Enterprise single-replica stack.
10. Verify connectivity, NFS read/write access, EMQX recovery, database failover, backup restore, and routine inspection.
11. Finalize operation documents, archive account credentials, and set up routine inspection tasks.

Risks and Items to Confirm

- The actual on-site network segment, gateway, DNS, NTP, certificates, and image registry address must be confirmed.
- It is not yet confirmed whether Tier0 Enterprise needs local file persistence. If it does, add a shared storage or object storage solution.
- It must be confirmed whether NFS/NAS is provided by existing on-site storage. If NFS is a single point of failure, shared file functions will also have a single point of failure.
- Each Swarm node has only a `100G` single system disk. Control the number of images, container logs, and temporary files. It is not suitable for long-term business data storage.
- `EMQX` is a stateful service. Before implementation, confirm whether to use single-node fixed mode or three-node cluster mode. In single-node fixed mode, the messaging layer has a single point of failure.
- Database RPO/RTO has not been confirmed. The current plan uses primary-standby automatic failover and cannot promise zero data loss.
- The `DB VIP` failover script must be verified on site, especially whether the virtualization platform allows VRRP.

- If the on-site network does not allow VRRP, use a load balancer, DNS switch, or virtualization-platform floating IP instead.

Standard Deployment Plan

Objective

The simplified edition does not use Docker Swarm. It deploys one `enterprise` application node with a total of `4` virtual machines:

- `1` enterprise node hosts `tier0-enterprise + EMQX`.
- `2` database nodes form a PostgreSQL + TimescaleDB primary/standby cluster.
- `1` Witness node participates in database supervision, quorum, and failover.

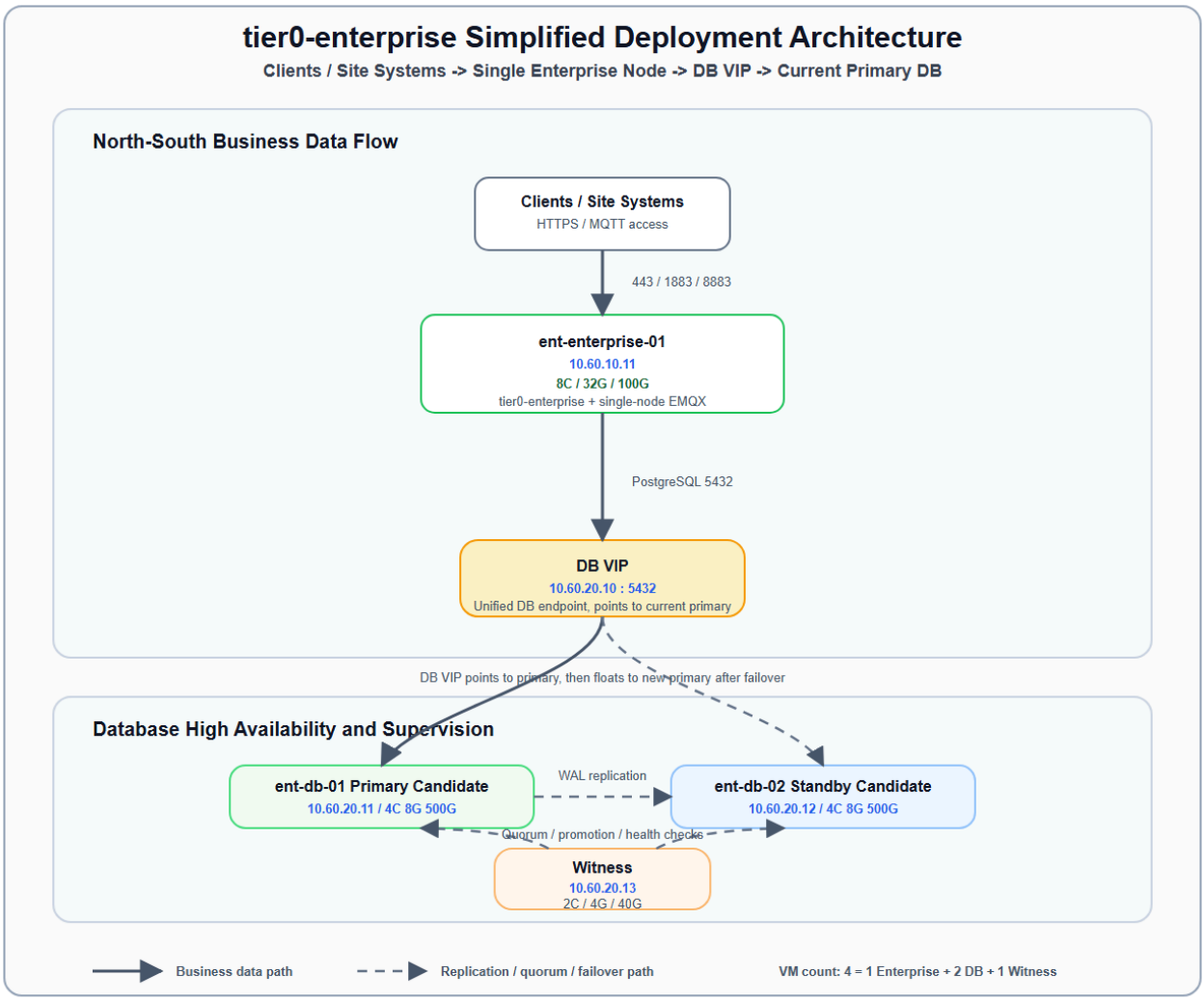
The simplified edition reduces deployment complexity. The tradeoff is that the application layer and EMQX are single points of failure. Only the database layer keeps `DB VIP + Witness` high availability.

Planned IP Addresses

The following IP addresses are placeholders. Replace them with the real site network before implementation.

Type	Name	Planned IP	Description
Application node	`ent-enterprise-01`	`10.60.10.11`	Single enterprise node hosting `tier0-enterprise + EMQX`
Logical entry	`DB VIP`	`10.60.20.10`	Unified database endpoint, points only to the current primary
Database node	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB primary candidate
Database node	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB standby candidate
Quorum node	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` / database quorum supervisor

Overall Topology



Core paths:

- Business path: clients / site systems -> `ent-enterprise-01` -> `DB VIP` -> current primary DB.
- `tier0-enterprise` and `EMQX` both run on `ent-enterprise-01`.
- Database access still goes through `DB VIP`; direct application access to `ent-db-01/02` is prohibited.
- `Witness` participates only in database quorum, promotion, and health checks. It is not on the business path.

Database Supervision and Failover

Phase	Path	Description
Normal access	`ent-enterprise-01 -> DB VIP -> current primary`	Application always connects to `10.60.20.10:5432`
Normal replication	`current primary -> current standby`	PostgreSQL WAL streaming replication
Normal supervision	`DB-01/DB-02 -> Witness`	`repmgrd` reports status; Witness participates in quorum

Phase	Path	Description
Primary failure	`standby -> Witness`	Standby requests quorum confirmation to avoid split-brain
Standby promotion	`standby promote`	Standby is promoted after Witness confirmation
VIP floating	`DB VIP -> new primary`	`10.60.20.10` binds to the new primary
Application recovery	`ent-enterprise-01 -> DB VIP -> new primary`	Application recovers after connection-pool reconnect

Node Configuration

Node	Role	vCPU	Memory	Storage	System disk recommendation	Data disk recommendation	Main components
`ent-enterprise-01`	Single enterprise node	`8`	`32G`	`100G`	Single `/` system disk, 100G	No separate data disk	tier0-enterprise, EMQX, optional Nginx/HAProxy
`ent-db-01`	PostgreSQL/TSDB primary candidate	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-02`	PostgreSQL/TSDB standby candidate	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-witness-01`	Witness / quorum supervisor	`2`	`4G`	`40G`	`/` 40G	Not planned separately	repmgr witness, monitoring agent, audit logs

Configuration Summary

Category	Count	Per-node spec	vCPU subtotal	Memory subtotal	Storage subtotal
enterprise application node	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB nodes	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`
Witness node	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
Total	`4`	-	`18C`	`52G`	`1140G`

Converted total:

- Total VMs: `4`

- Total vCPU: `18`
- Total memory: `52G`
- Total storage: `1.14T`

Connectivity Matrix

Operations Access

Target	Method	Port	Description
All VMs	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	Use after unified key-based SSH is implemented; use site account before that
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	Business entry
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	Allow only enterprise node and operations whitelist

Application and EMQX Ports

Source	Target	Port	Purpose
Users / site systems	`ent-enterprise-01`	`80/TCP`	HTTP entry or HTTPS redirect, optional
Users / site systems	`ent-enterprise-01`	`443/TCP`	HTTPS business entry
Devices / site systems	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT plaintext, optional
Devices / site systems	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS, recommended
Devices / site systems	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket, optional
Devices / site systems	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS, optional
Operations network	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard, operations whitelist only
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL business access

Database and Supervision Communication

Source	Target	Port	Purpose
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL streaming replication
`ent-db-02`	`ent-db-01`	`5432/TCP`	Reverse replication after recovery

Source	Target	Port	Purpose
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr metadata / witness communication
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP floating; allow protocol `112`

Deployment Recommendations

enterprise Application Node

- Do not deploy Docker Swarm.
- Deploy `tier0-enterprise` and `EMQX` on the same `ent-enterprise-01` node.
- `ent-enterprise-01` is an application-layer single point of failure. If it fails, recover manually or rebuild from backup.
- The node has only one `100G` system disk and is not suitable for long-term business data.
- `tier0-enterprise` business data must be stored in PostgreSQL/TimescaleDB.
- Uploads, imports/exports, and reports may be stored locally and backed up. If later upgraded to the standard edition, migrate these files to NFS/NAS or object storage.

Single-node EMQX

- EMQX is a single-node stateful service in the simplified edition.
- Pin its data directory on `ent-enterprise-01`, for example `/data/emqx`.
- Back up configuration, certificates, plugins, and rule-engine configuration.
- No message-layer HA is provided. Use the standard edition with a three-node EMQX cluster if this is unacceptable.

PostgreSQL + TimescaleDB

- PostgreSQL: `16` or the site-certified version.
- TimescaleDB: stable version compatible with the selected PostgreSQL major version.
- Replication: physical streaming replication.
- Failover: `repmgrd + witness`.
- Database entry: `DB VIP`.
- Connection pool: PgBouncer can run on `ent-enterprise-01` to reduce database connection pressure.

Witness

- Witness stores only `repmgr witness` metadata and supervision logic.
- Witness does not host business SQL.
- Witness failure does not stop the current primary, but reduces trusted automatic failover capability.

Port and Firewall Baseline

Port / protocol	Allowed scope	Purpose
`22/TCP`	Operations network to 4 VMs	SSH

Port / protocol	Allowed scope	Purpose
`80/TCP`	User network to `ent-enterprise-01`	HTTP redirect or internal plaintext entry
`443/TCP`	User network to `ent-enterprise-01`	HTTPS business entry
`1883/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX MQTT plaintext, optional
`8883/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX MQTT TLS, recommended
`8083/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX WebSocket, optional
`8084/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX WebSocket TLS, optional
`18083/TCP`	Operations network to `ent-enterprise-01`	EMQX Dashboard, whitelist only
`5432/TCP`	enterprise to `DB VIP`; DB/Witness internal	PostgreSQL / repmgr
`112/VRRP`	Between DB nodes	DB VIP floating
`9100/TCP`	Monitoring system to 4 VMs	node exporter, optional

HA Capabilities and Failure Boundaries

Failure scenario	Expected behavior	Business impact
`ent-enterprise-01` fails	Application and EMQX both stop; manual recovery or rebuild from backup is required	Full application-layer outage
Current DB primary fails	Standby is promoted after Witness quorum; DB VIP floats	Write interruption for seconds to minutes; application must reconnect
DB standby fails	Primary continues serving, replication alert is raised	No direct business outage, but DB hot standby is lost
Witness fails	Current primary continues serving, but automatic failover confidence is reduced	No direct business outage, restore Witness first
DB VIP floating fails	Database may have a new primary, but unified entry is unreachable	Business outage; manual VIP binding required

Pre-Go-Live Checklist

Check item	Pass criteria
4 VM hostnames are fixed	`hostnamectl` matches the plan

Check item	Pass criteria
Unified SSH works	Operations terminal can access all 4 VMs with key-based SSH
Time synchronization	Time drift across 4 VMs is below `1s`
Firewall is open	Ports from section 9 are allowed by scope
enterprise service is healthy	`tier0-enterprise` health check passes
EMQX state is verified	EMQX node, listening ports, and persistent directory are healthy
enterprise disk usage is safe	`/` usage is below `70%`; log and cache cleanup is configured
DB primary/standby is healthy	`repmgr cluster show` shows primary, standby, and witness healthy
DB VIP works	enterprise node can connect to `10.60.20.10:5432`
Primary DB failover drill	Standby can be promoted and DB VIP can float
Backup restore drill	Backup can be restored to a test database

Implementation Order

1. Create 4 VMs and fix hostnames, IPs, time synchronization, and SSH.
2. Initialize OS baseline, firewall, mounts, and kernel parameters.
3. Deploy `tier0-enterprise` on `ent-enterprise-01`.
4. Deploy single-node EMQX on `ent-enterprise-01`.
5. Deploy PostgreSQL + TimescaleDB primary/standby.
6. Deploy repmgr witness, repmgrd, and DB VIP floating scripts.
7. Verify connectivity, EMQX recovery, database failover, backup restore, and routine checks.
8. Finalize runbooks, credential archives, and routine check tasks.

Risks and Open Items

- The simplified edition has an application-layer single point. If `ent-enterprise-01` fails, the whole business service is unavailable.
- EMQX is single-node and does not provide message-layer HA.
- `ent-enterprise-01` has only a single `100G` system disk. Logs, cache, attachments, and temporary files must be controlled.
- If application-layer or EMQX HA is required later, switch to the standard edition.
- Database RPO/RTO is not confirmed. This primary/standby design cannot guarantee zero data loss.
- DB VIP floating scripts must be verified on site, especially whether the virtualization/network platform allows VRRP.

Login

1. Access Tier0 with the set domain and port on a browser.
2. On the login page, use your license to activate Tier0.

- **Online Activation**: Enter the license key, and click **Activate Now**.



- **Offline Activation**: Send the token to your vendor to get the JSON file, and import the file.



3. Enter the default username and password (tier0/tier0) after activation.

4. Click **Sign In**.

How-to Guide

Connect Data

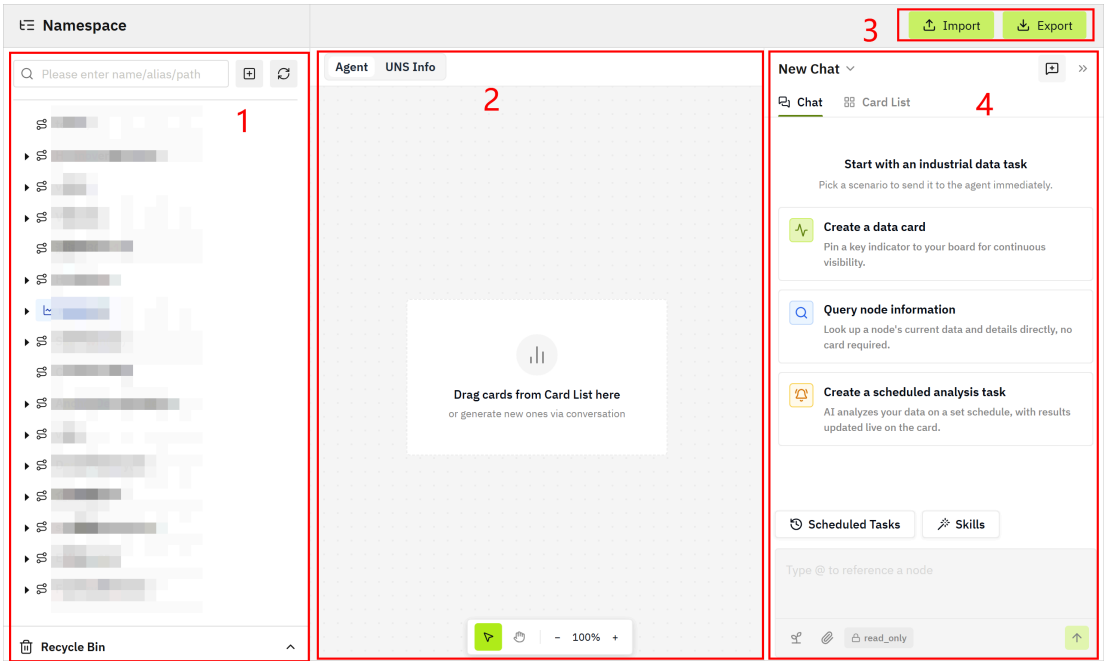
Build Data Models

Tier0 uses UNS (Unified Namespace) to organize data into a clear tree structure. Each path and topic in the model automatically generates an MQTT topic, making it easy to get real-time data.

Things to Notice before Start

Page Overview

Select **Unified Namespace** > **UNS** after logging in to Tier0, you will see the main page of this module.



No.	Item	Description
1	Namespace	Displays data model list. You can add models, refresh the list and recover models from recycle bin.
2	Canvas	Displays selected UNS information or cards generated through chats on the right.
3	Import/Export	Import/export namespace models in JSON text or file.
4	Chat	Use natural language to interact with UNS topics, and cards generated from the chat can be dragged to and displayed on the canvas.

Additional Parameter Definition

- **Path-Additional Settings**

Item	Description
Extended Attribute	Add extended attributes for the path. Eg. unit.

- **Topic-Topic Type**

Type	Description
State	Works as an API to receive system state data.
Action	Works as an API to execute CRUD commands.
Metric	Real-time data. For example, equipment temperature.

- **Topic-Attribute Generation Method**

Attribute Generation Method	Description
Custom	Customize attributes.
Auto Parsing	Use JSON text to generate attributes with the same structure.

- **Topic-Data Quality**

Data Quality	Description
Value range check	Check if the values of attributes fall within the specified range.
Collection interruption check	Check if the data collection for the topic has been interrupted based on the set interval and delay.

- **Topic-Optional Behaviors**

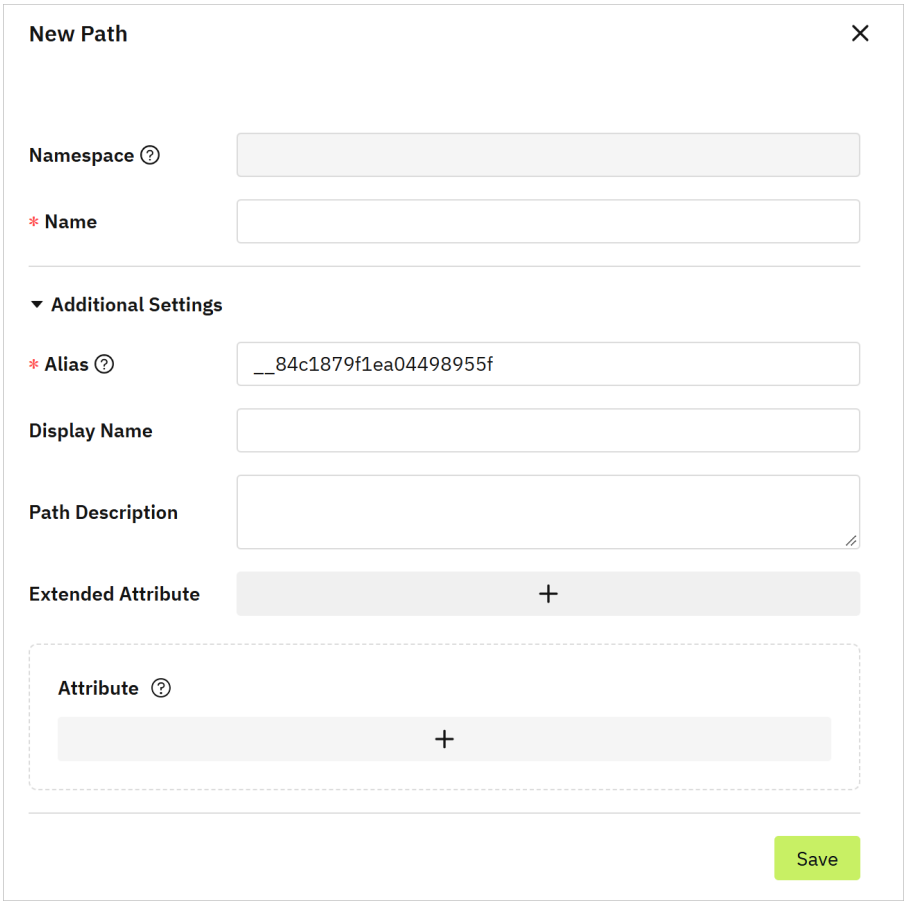
Item	Description
Mock Data	Automatically create a data flow for the topic in **Source Flow** to send generated mock data.
Enable History	Enable historical data storage for the topic.

How to Build a Data Model

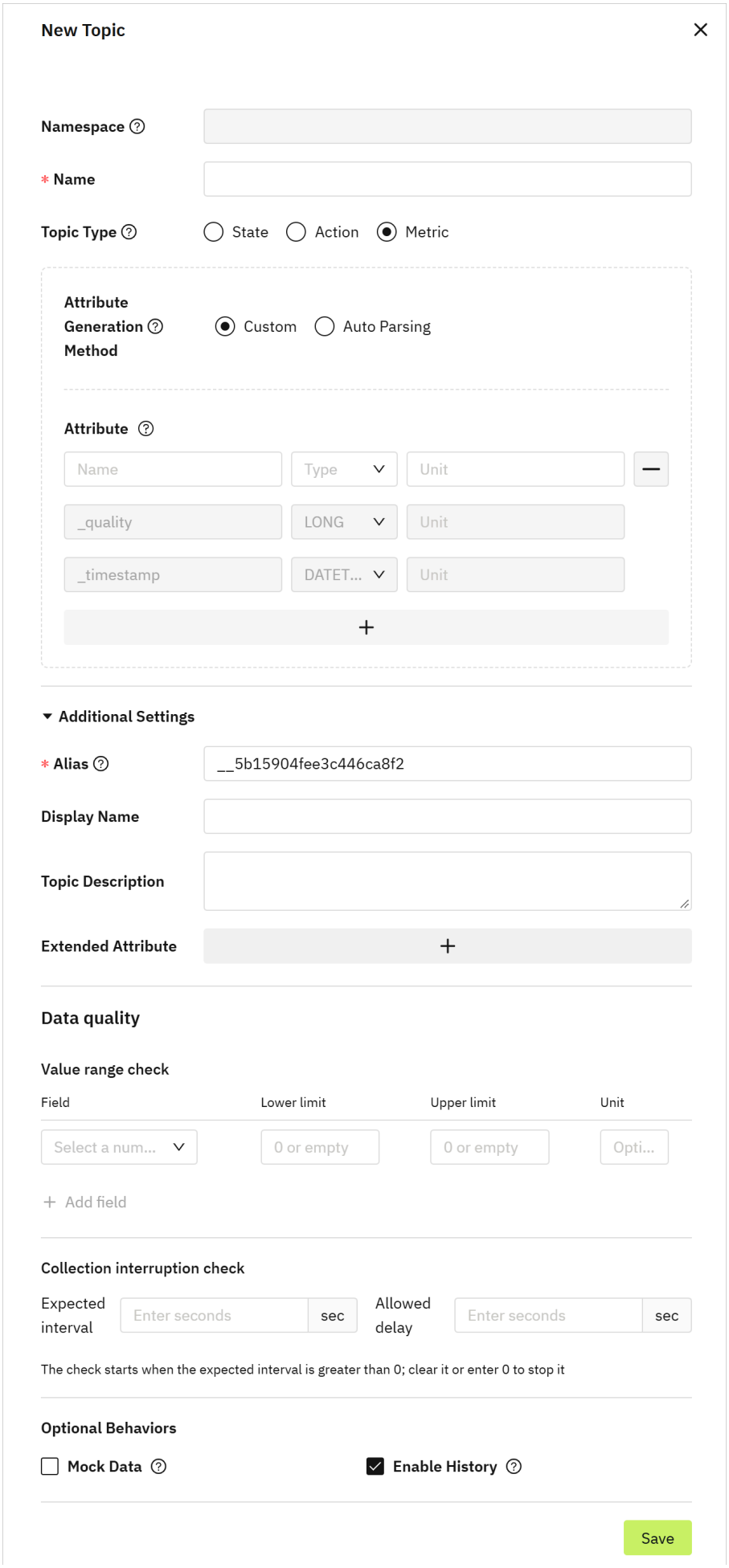
Based on simple folder-file structure, you can define the data hierarchy to a tree map.

Building Models Manually

1. Log in to Tier0, and select **Unified Namespace** > **UNS**.
2. Click  > **New Path** to add a path (e.g. `factory`).



3. Enter the path information, set the alias as needed, and then click **Save**.
4. Select **factory**, and then click  > **New Topic** to add a topic (e.g. `CNC`) under it.


New Topic
Namespace
* Name
Topic Type
State
Action
Metric
Attribute Generation Method
Custom
Auto Parsing
Attribute
Name
Type
Unit
_quality
LONG
Unit
_timestamp
DATET...
Unit
Additional Settings
* Alias
__5b15904fee3c446ca8f2
Display Name
Topic Description
Extended Attribute
Data quality
Value range check
Field
Lower limit
Upper limit
Unit
Select a num...
0 or empty
0 or empty
Opti...
Add field
Collection interruption check
Expected interval
Enter seconds
sec
Allowed delay
Enter seconds
sec
The check starts when the expected interval is greater than 0; clear it or enter 0 to stop it
Optional Behaviors
Mock Data
Enable History
Save

5. Enter the information of the topic, and then click **Save**.

Importing Models

1. Log in to Tier0, landed on the **Namespace** page.
2. Click **Import** at the upper-right corner.
3. Import JSON to create models.

- Click **UNS**, and directly enter JSON.

> **HOW TO GET MODEL JSON FAST**
>
> Click **Copy for Agent**, send it to an agent, get the result and paste it back.

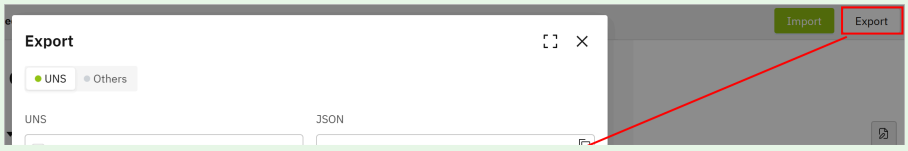


- Click **Download Template** at the **UNS** tab, enter the model content and upload it at the **All** tab.

```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION_
  "Label": [{"name": "debug"},{"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
```


TIP

You can manually add a path and topic, export it and use it as an example for import.

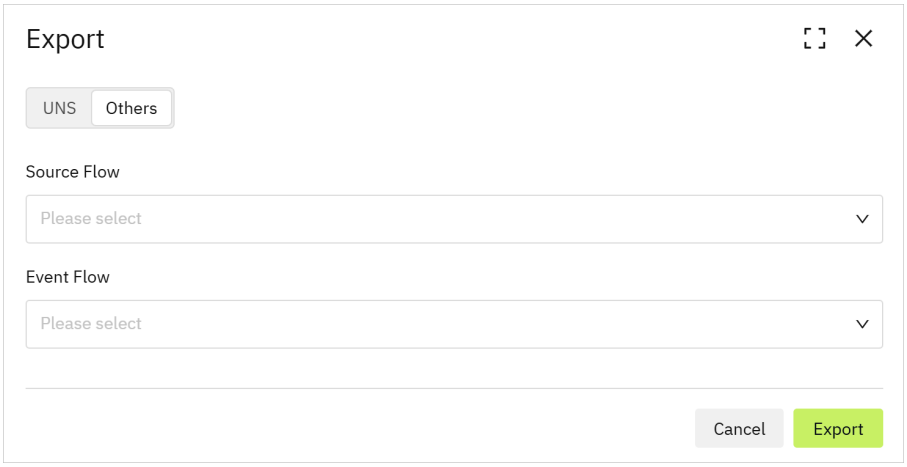


4. Click **Save**.

Exporting Data

Tier0 can export UNS models and other data, such as source flows, event flows, and data dashboards.

1. On the **UNS** page, click **Export** at the upper-right corner.
2. Click the **Others** tab and select the corresponding data type.



3. Click **Export**.

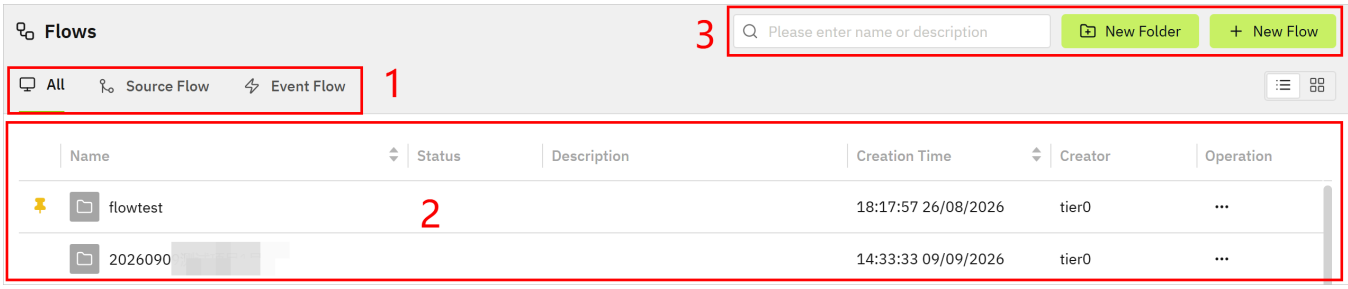
Connect Data Sources

How to Add Data Sources through Source Flow

Once the data model is built under **UNS**, you need to connect real data to the model for subsequent processing and use.

Adding Data Flow Manually

1. Log in to Tier0, select **Flows**.

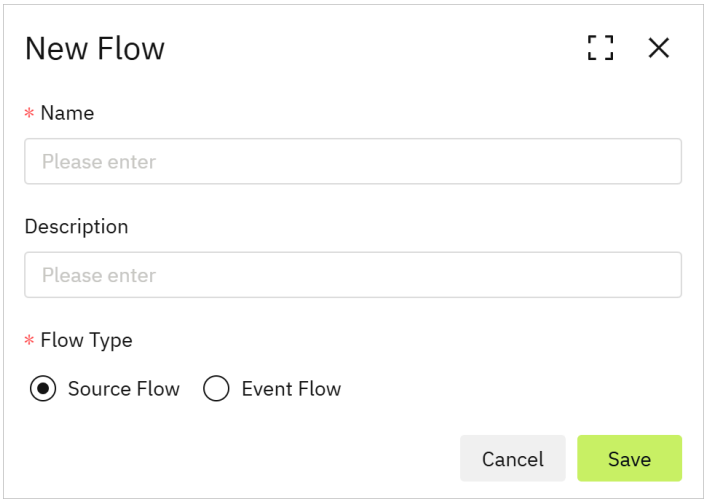


Index	Item	Description
1	Flow Display	Select to view all flows or only one category.
2	Flow List	Displays flow information. Click the flow name to start editing the data flow.
3	Search/Filter	Search for/filter existing data flows.
	New Folder	Add folders to categorize data flows.
	New Flow	Add new data flows.

2. Click **New Flow**, enter a name and description in the pop-up window, select **Source Flow** as **Flow Type**, and click **Save**.

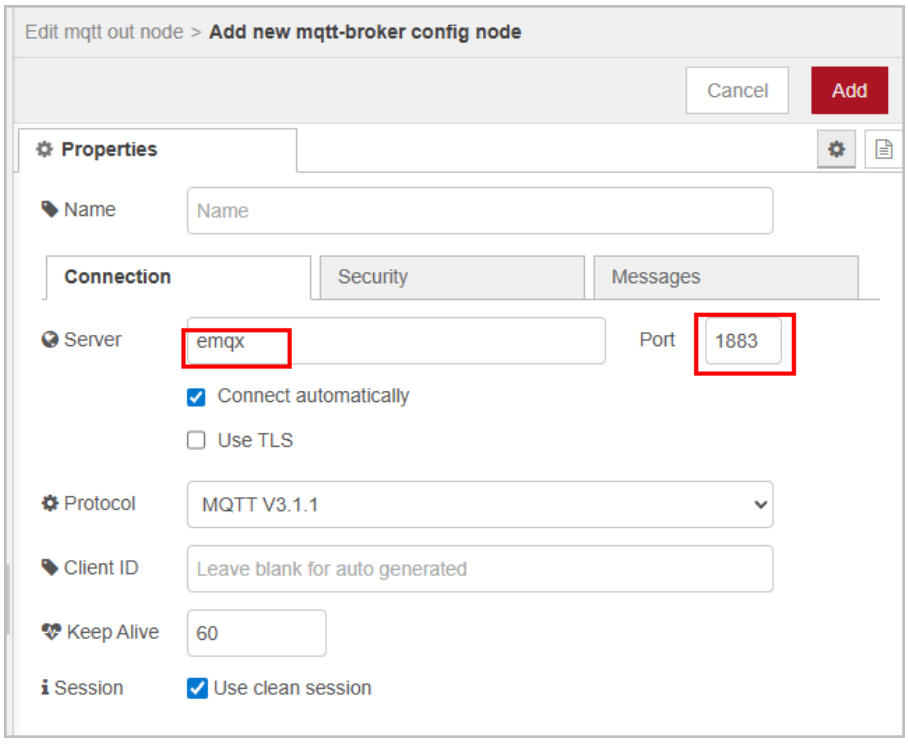
- **Source Flow**: Used to collect data from data sources and send it to UNS.

- **Event Flow**: Used to process data from UNS.



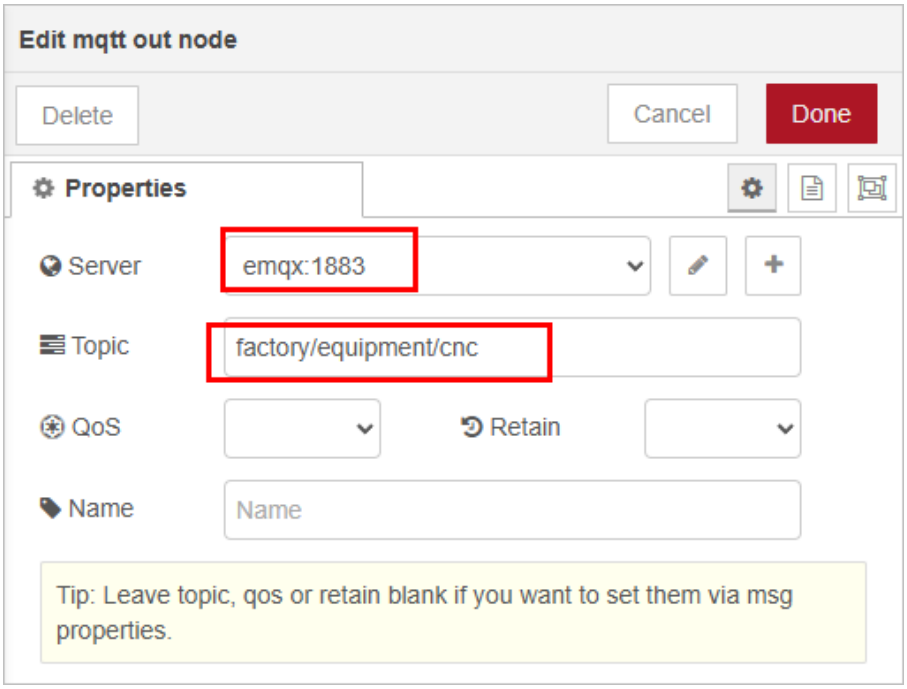
3. Click the flow you just added to enter the configuration page, and drag in nodes according to your data source type (e.g. `modbus`).
4. Configure the data source node, and then drag in an `mqtt out` node, finish its configurations.

    i. Add the internal broker: `emqx:1883`.



    ii. Set its subscription topic to the data model in **Namespace**.

    For example, the data model structure is `factory/equipment/cnc`, the topic will be `factory/equipment/cnc`.



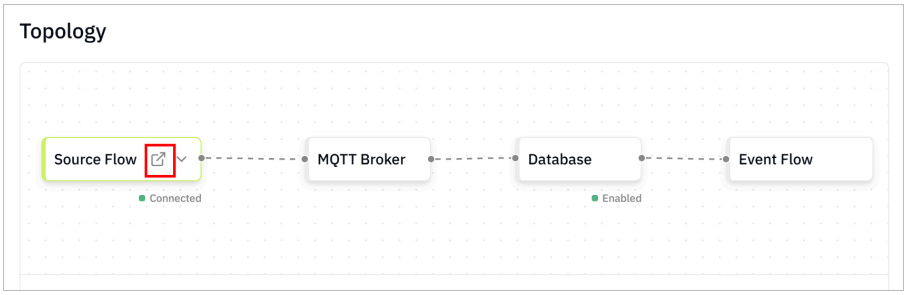
5. Click **Deploy** at the upper-right corner, trigger the flow and the data will be sent to the corresponding model under **Namespace**.

> **TIP**
>
> You can save the the flow as different versions by clicking **Save Version**, and switch to previous versions when needed.

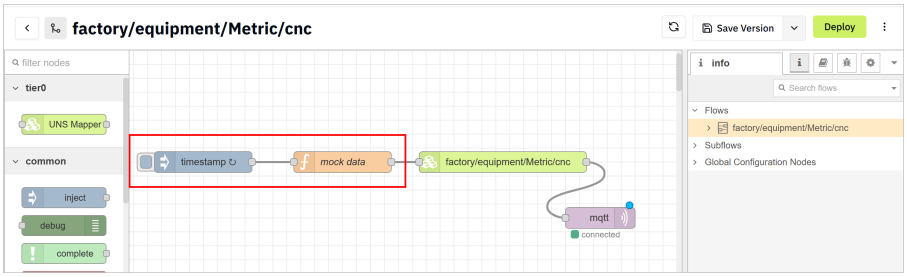
Configuring Data Source through Generated Flow

> **NOTE**
>
> Data flow will only be automatically generated if you enabled **Mock Data** when creating topics.

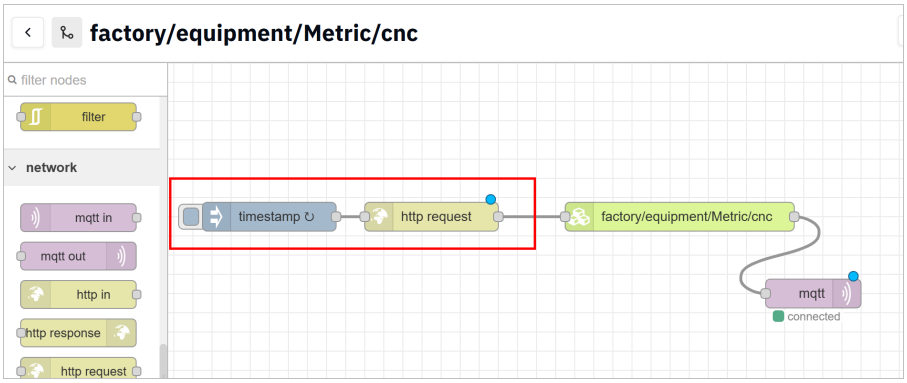
1. Log in to Tier0, go to **UNS**.
2. Check the details of the created topic, scroll down to **Topology**, and click the **Source Flow** icon to access the generated flow.



3. Delete the simulated data source of the flow.



4. Connect the real source node to the rest of the flow and configure it.



5. Click **Deploy** at the upper-right corner, and then trigger the flow.
6. Go back to **UNS**, check whether the data is received.

Payload

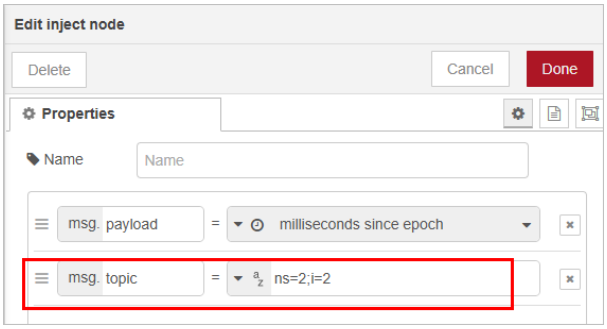
```
{
  "temp": 85.23,
  "humidity": 69.3
}
```

Common Data Sources

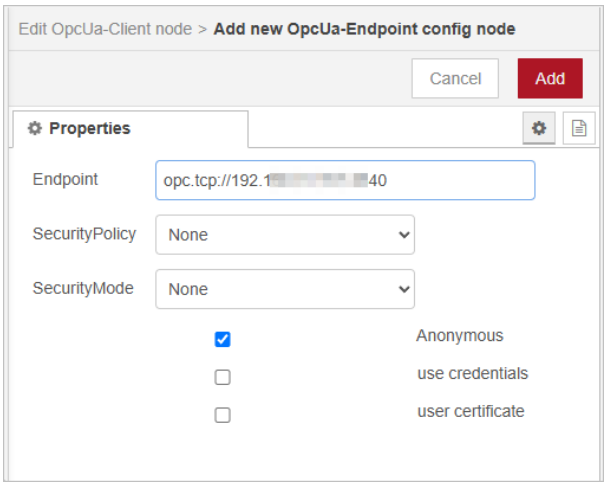
Connecting OPC UA

Read Data from OPC UA Server

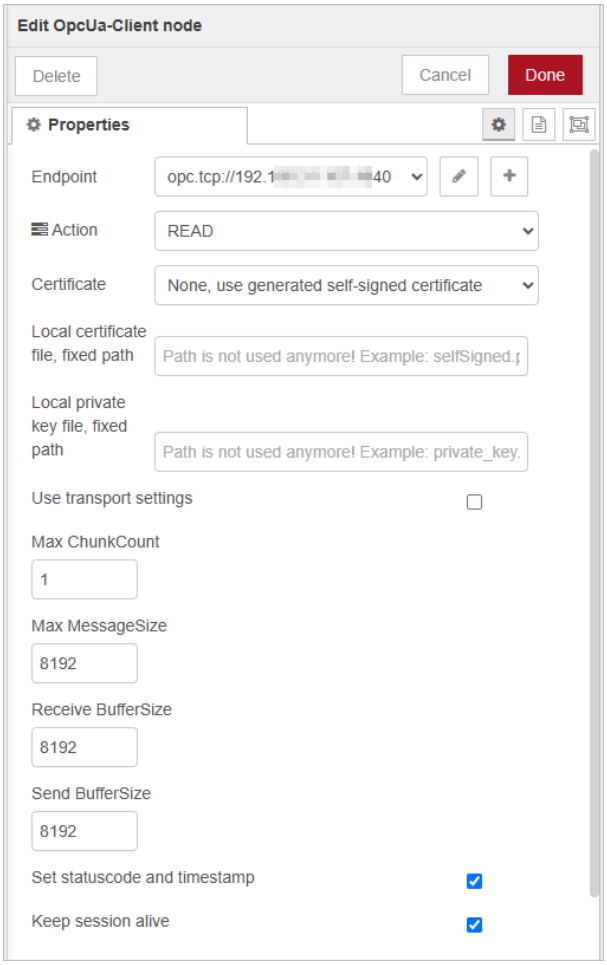
1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.



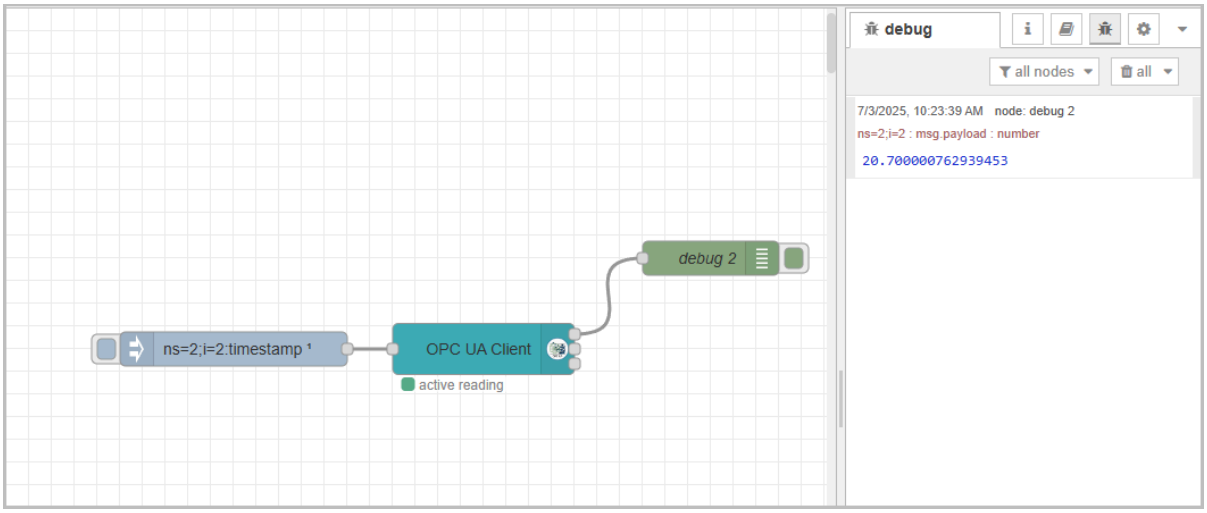
3. Drag an **OpcUa-Client** node, and then double-click it.
4. Click ✚ next to **Endpoint** to enter the information of your OPC UA server.



5. Click **Add**, and set **Action** to **READ**.
6. Leave everything else as default, and then click **Done**.



7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



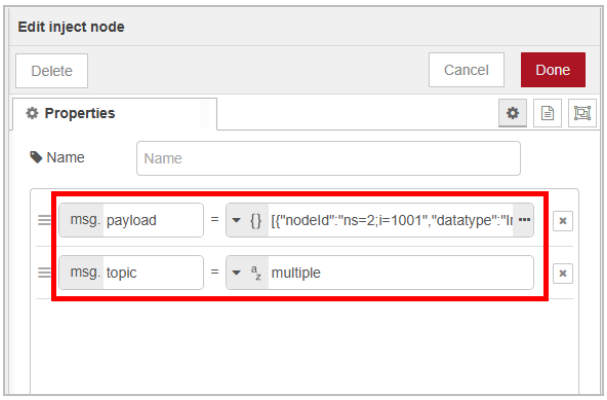
Subscribing Data from OPC UA Server

1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.

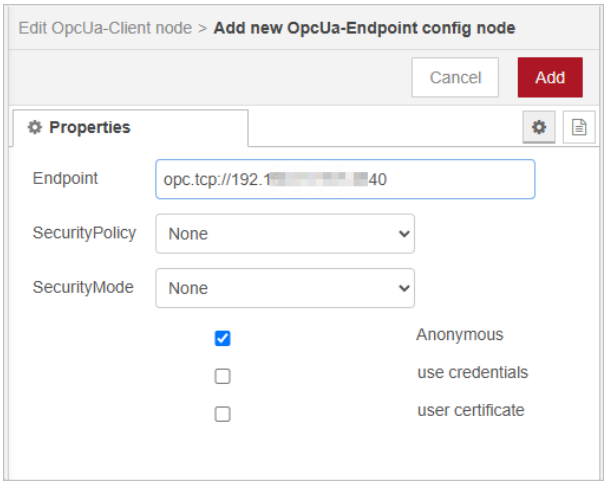
- msg.payload:

```
[
    {
        "nodeId": "ns=2;i=1001",
        "datatype": "Int32"
    },
    {
        "nodeId": "ns=2;i=1002",
        "datatype": "Int32"
    }
]
```

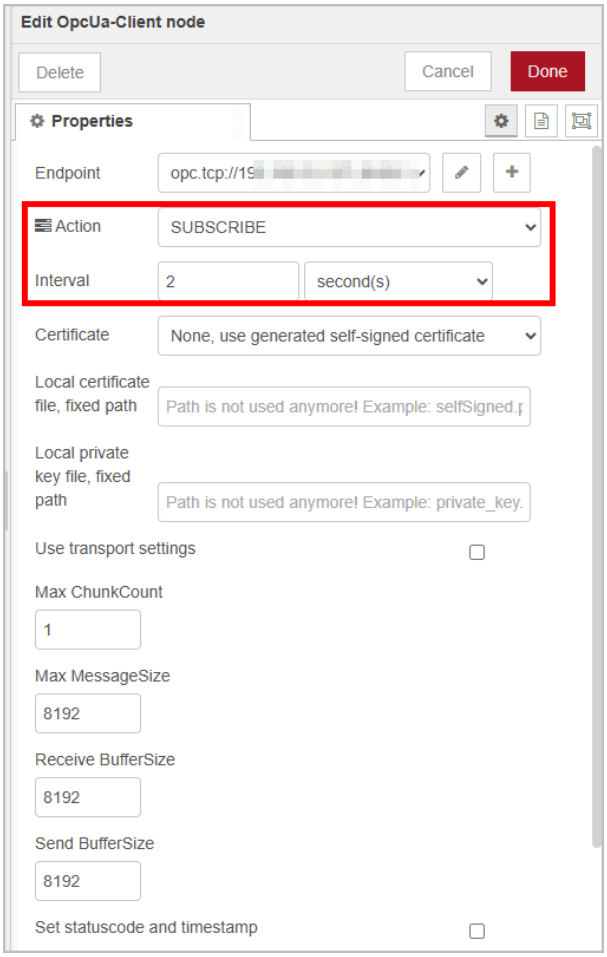
- msg.topic: multiple



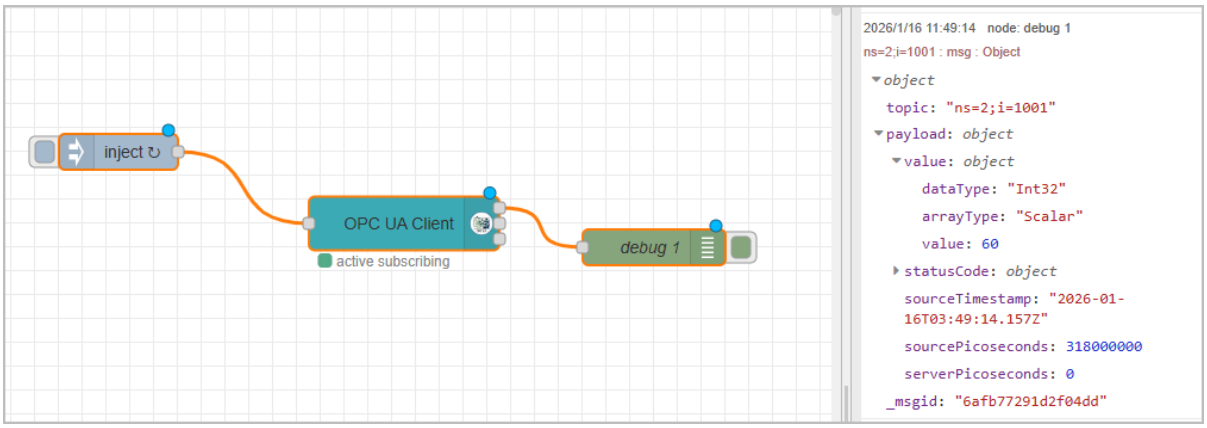
3. Drag an **OpcUa-Client** node, and then double-click it.
4. Click ✚ next to **Endpoint** to enter the information of your OPC UA server.



5. Click **Add**, and set **Action** to **SUBSCRIBE**.
6. Set the subscription interval, and leave everything else as default, and then click **Done**.

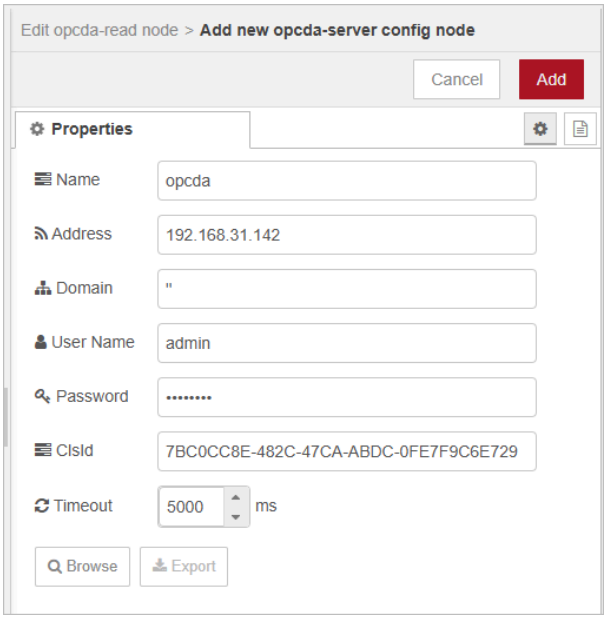


7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



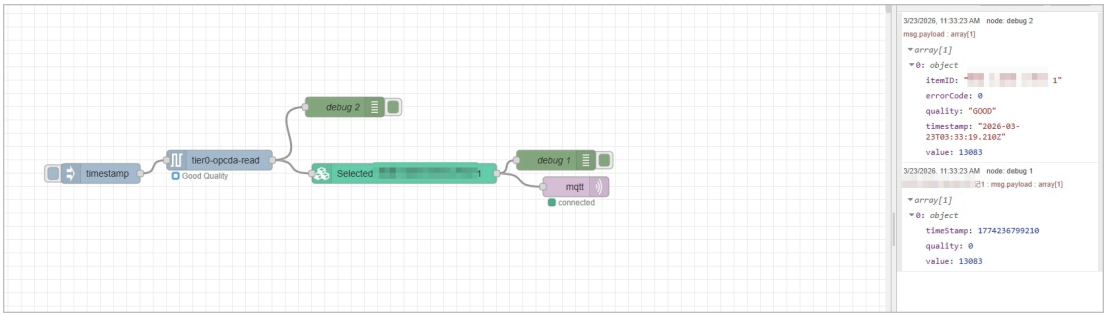
Connecting OPC DA

1. In **Source Flow**, drag an **inject** node.
2. Drag an **opcda-read** node, and then double-click it.
3. Click ✚ next to **Server** to enter the information of your OPC DA server.



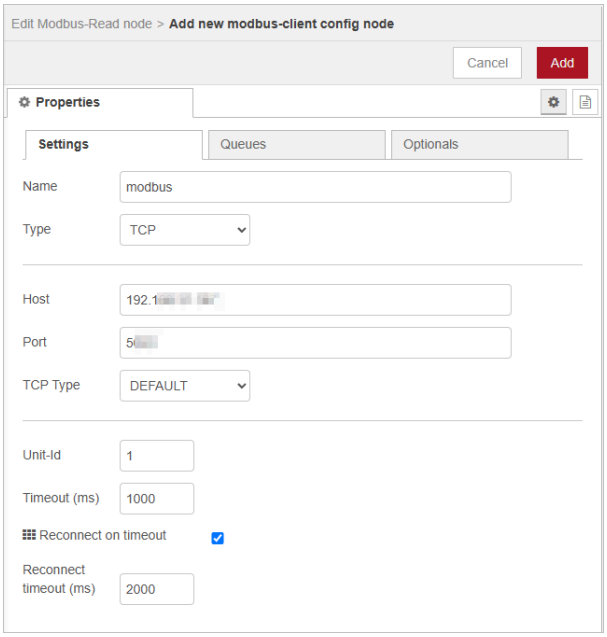
Parameter	Description
Address	OPC DA server IP address.
Domain	The domain name of your OPC DA server.
User Name/Password	The username and password you use to log in to the DA server.
ClsId	The application ID of the DA connetion service you use. For example, KepServer.

5. Click **Browse** to see the connected channels.
6. Click **Add**, and then under **Items**, add the items needed.
7. Connect the **inject** node to the **opcda-read** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.

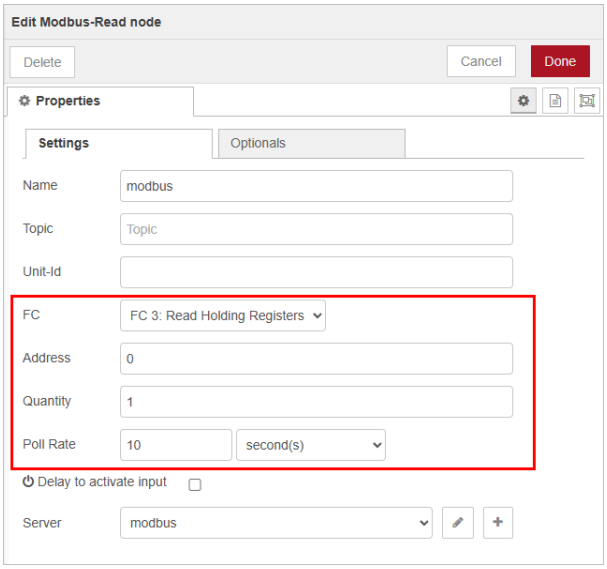


Connecting Modbus

1. In **Source Flow**, drag a **Modbus Read** node.
2. Double-click the node, and then click + next to **Server** to enter the information of your modbus server.



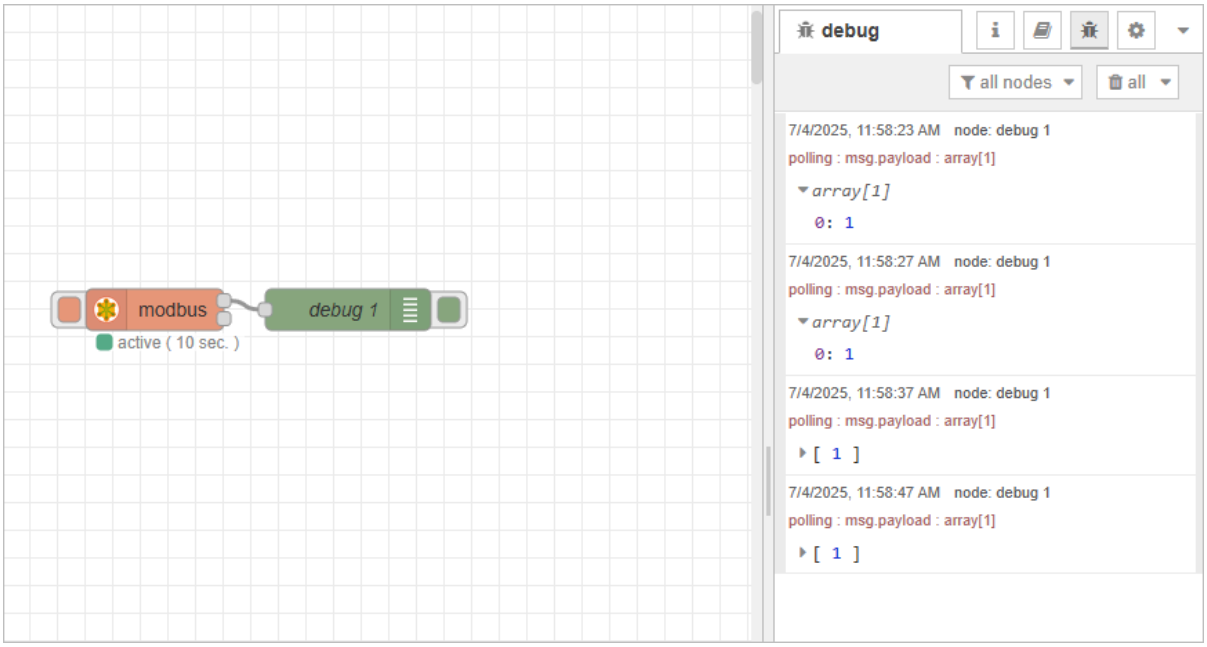
3. CLick **Add**, and then enter the information of the data to be connected.



Parameter	Description
Name	The display name of the node (for visual identification only).
Topic	Optional topic string passed to output messages.
Unit-Id	The Modbus slave ID of the target device (typically 1–247).
FC	Function code that specifies the Modbus action, e.g., FC 3 = Read Holding Registers.
Address	The starting register address to read from (usually zero-based).
Quantity	The number of consecutive registers to read.
Poll Rate	How often the node polls the Modbus device (e.g., every 10 seconds).

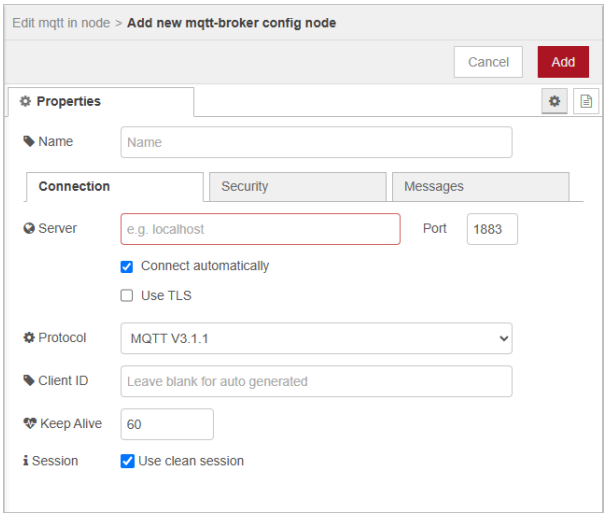
Delay to activate input	Optional delay before activating the input signal for polling.
Server	Reference to a configured Modbus server (IP, port, protocol, etc.).

4. Click **Done**.
5. Connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then click **Modbus Read** node to trigger the flow.



Connecting MQTT

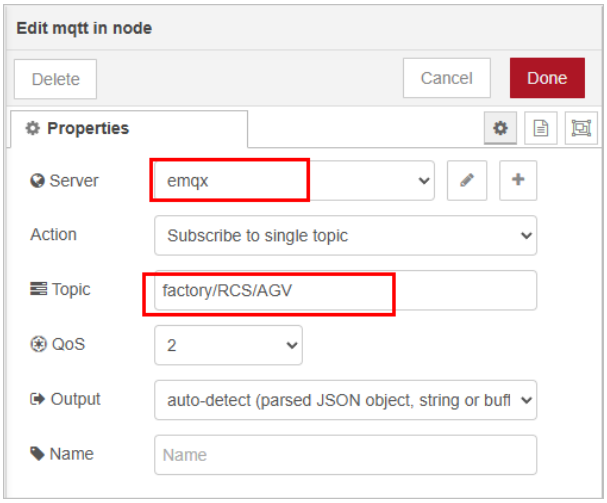
1. In **Source Flow**, drag an **mqtt in** node.
2. Double-click the node, and then click ➕ next to **Server**.



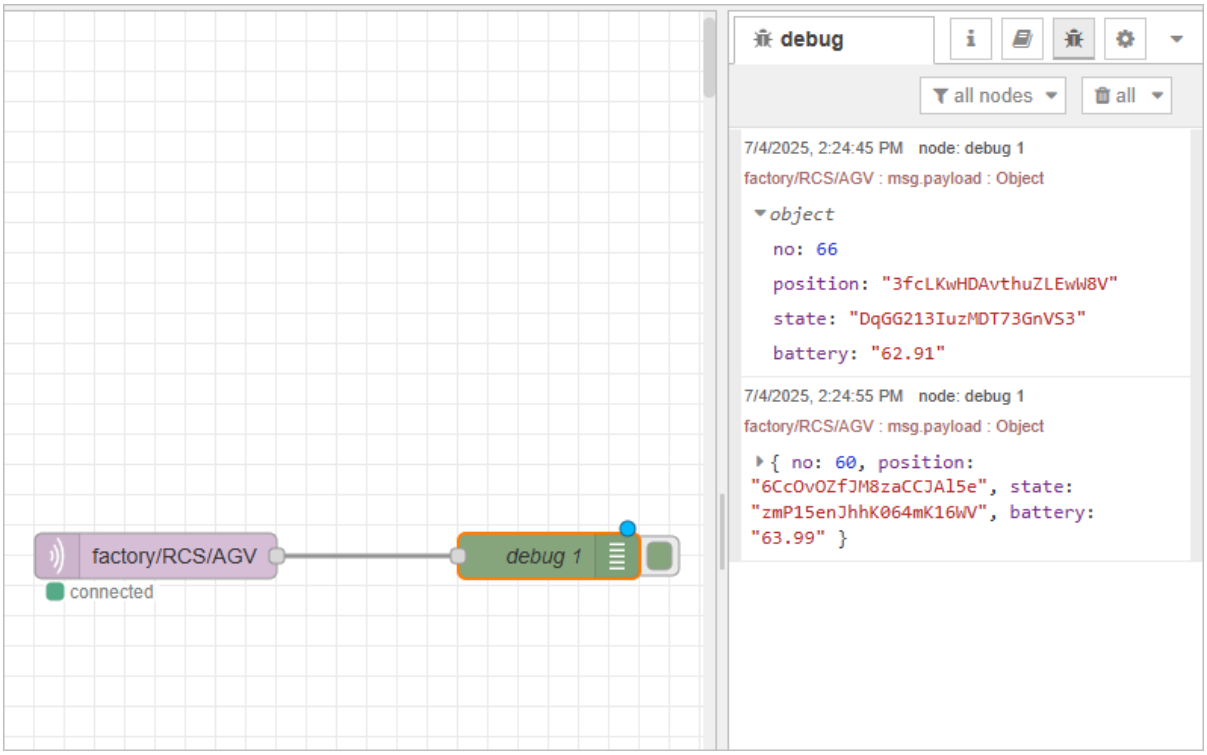
Parameter	Description

Name	Optional name for the MQTT broker configuration (for internal reference).
Server	The hostname or IP address of the MQTT broker (e.g., `localhost` or `broker.hivemq.com`).
Port	The port used to connect to the MQTT broker (default is `1883` for non-TLS, `8883` for TLS).
Connect automatically	If enabled, Node-RED will try to connect to the broker automatically when the flow starts.
Use TLS	Enable this option if the broker requires a secure TLS/SSL connection.
Protocol	The MQTT protocol version to use for connection (e.g., MQTT v3.1.1 or v5.0).
Client ID	Optional client ID. If blank, a unique one will be auto-generated. Must be unique per client.
Keep Alive	The maximum period (in seconds) between communications with the broker. Helps detect dropped connections.
Use clean session	If enabled, no persistent session is maintained with the broker; otherwise, session state (like subscriptions) is retained.
Username (Security tab)	Username for authenticating with the MQTT broker, if required.
Password (Security tab)	Password corresponding to the username for broker authentication. Stored securely.

3. Click **Add**, and then enter the topic to be subscribed.
4. Click **Done**.



5. Connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then once the topic receives data, the flow will be triggered.



Connecting File

> **INFO**
>
> Before processing files in **Source Flow**, you need to put the file in to the corresponding docker.

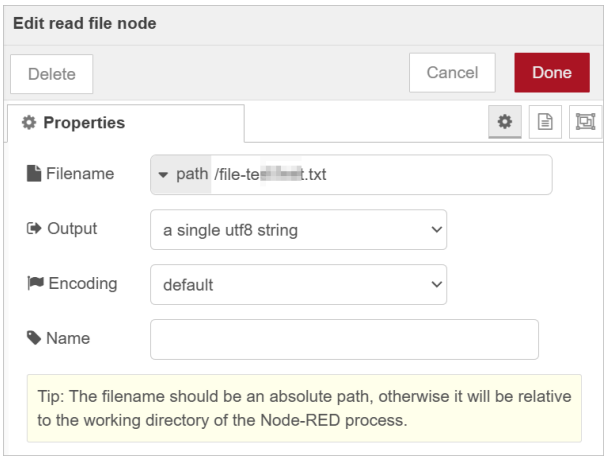
```
scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/
docker cp /root/config.json <docker name or ID>:/app/config.json
```

> **GENERAL DOCKER COMMANDS**

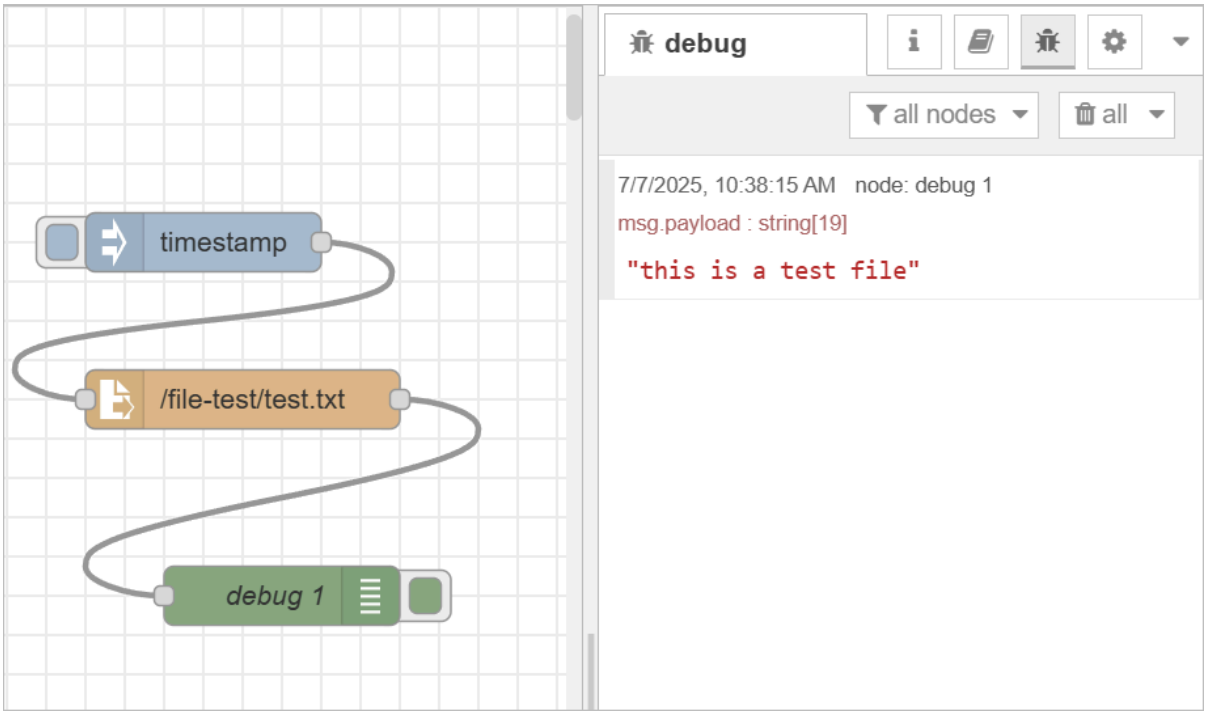
Command	Description
`docker ps`	Lists all running Docker containers.
`docker ps | grep [container]`	Filters running containers based on the specified docker information.
`docker exec -it [container] /bin/sh`	Starts a shell session inside a running container using `/bin/sh`.
`mkdir [directory]`	Creates a new directory with the specified name inside a container.

Connecting a Text File

1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

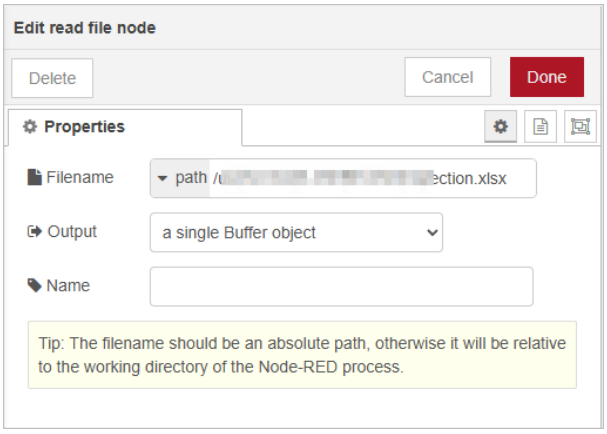


4. Click **Done**, and then connect a debug node at the end.
5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



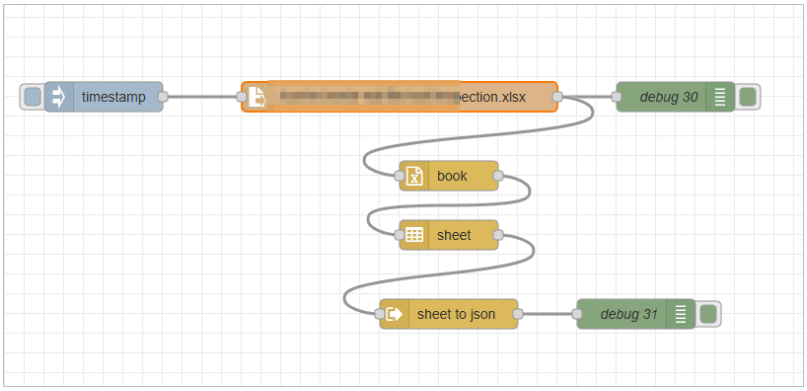
Connecting an Excel File

1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

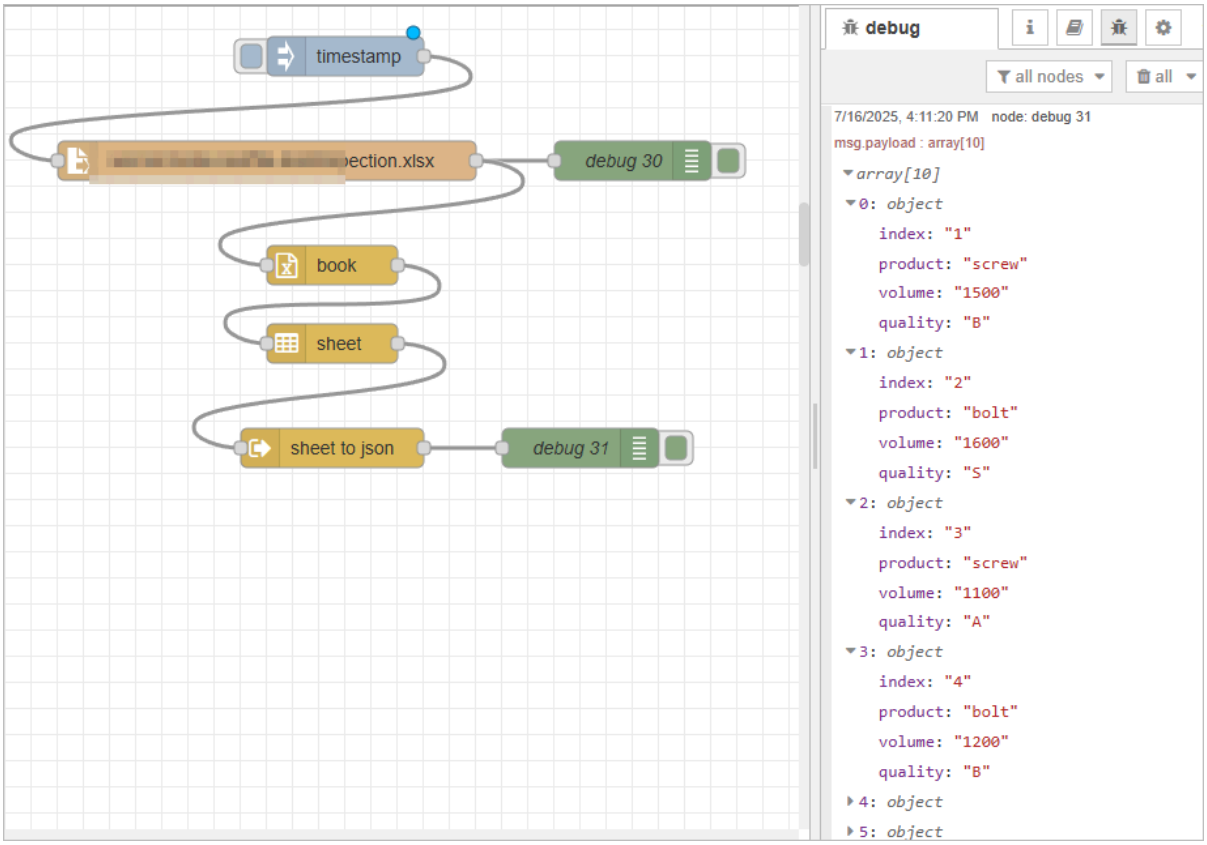


4. Click **Done**, and then connect a **book**, a **sheet** and a **sheet to json** node by order to convert table data to json.

> **INFO**
>
> Install the **node-red-contrib-spreadsheet-in** node to get necessary nodes for conversion.



5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



TIP

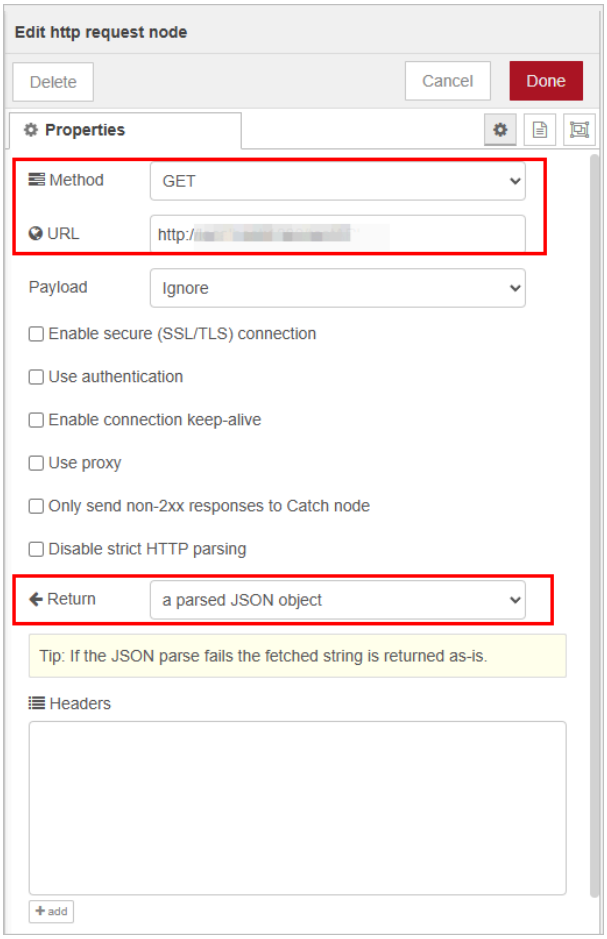
There are other nodes that can be used to process file data. For example, node-red-contrib-xlsx-to-json.

Connecting RestAPI

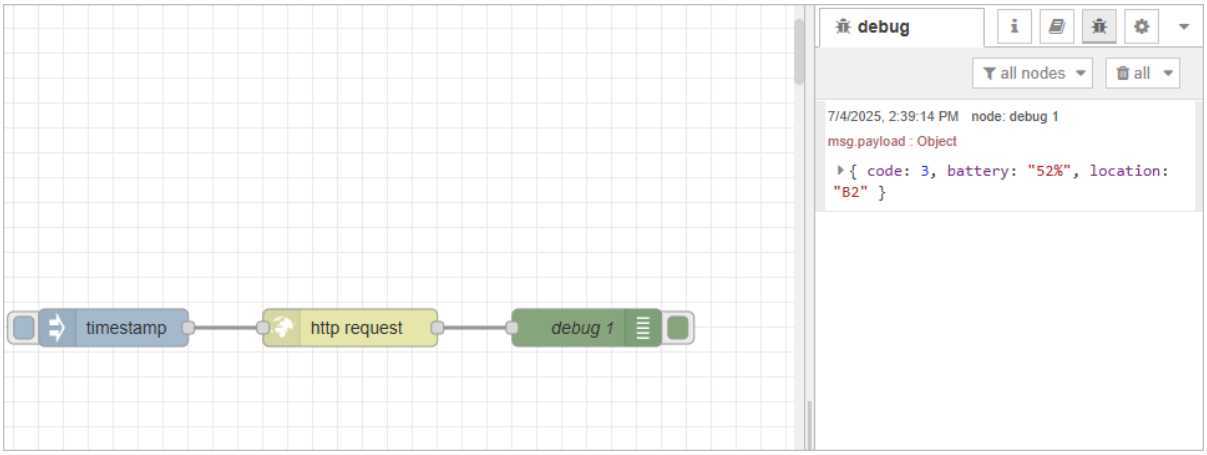
1. In **Source Flow**, drag an **inject** node.
2. Drag an **http request** node, and then double-click it.
3. Enter the API information.

TIP

Set the **Return** to **a parsed JSON obbject** for Tier0 namespace to parse.



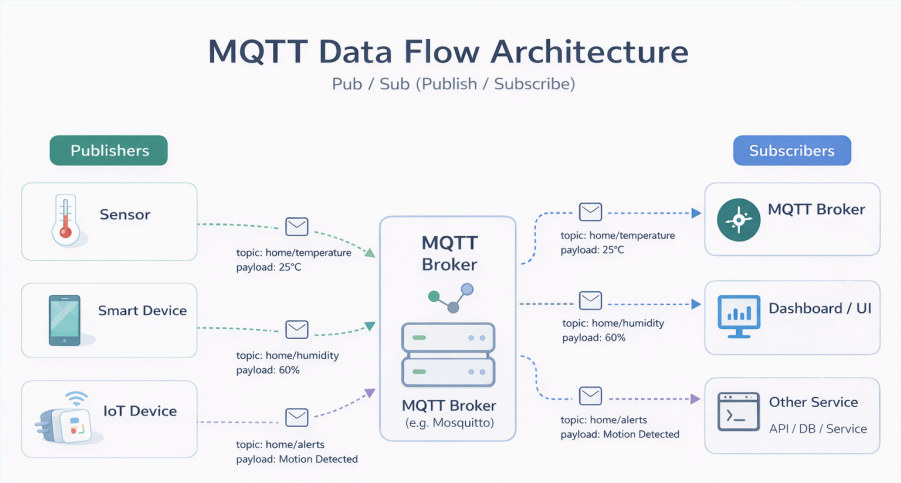
5. Click **Done**, and then connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connect Data via MQTT

WHAT IS MQTT?

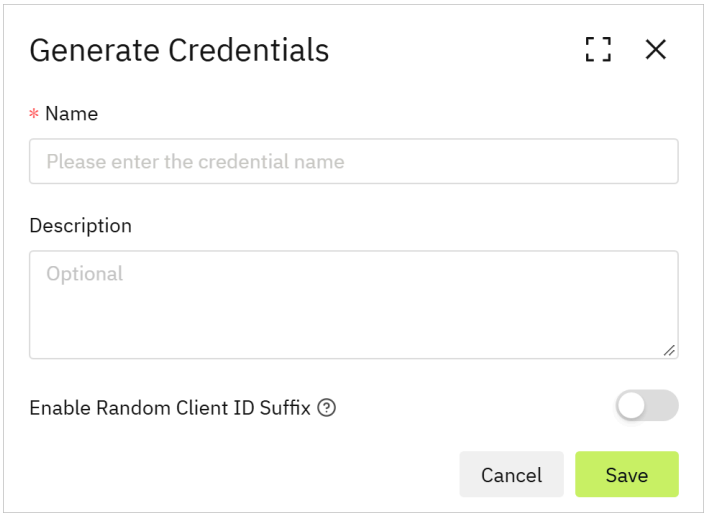
- MQTT is a lightweight real-time messaging protocol that lets connected devices (clients) send data to a central server (broker) and receive updates immediately.
- Access control can be added on the central server (broker) for security reasons, so devices (clients) must be authenticated before they can connect.
- For more details about MQTT, please refer to MQTT Protocol.



Generating MQTT Credentials

Tier0 offers an embedded MQTT broker that allows third-party clients to connect and publish data to/subscribe to data from **UNS**.

1. Log in to Tier0, select **Edge** > **MQTT Access**.
2. Click **Generate Credentials**, enter the name, and then click **Save**.



> **INFO**
>
> - Once you **Enable Random Client ID Suffix**, the generated **Client ID** itself and with different suffix can match the username and password for connections.
> - Make a note of the generated credentials. You will need them to configure the MQTT client.

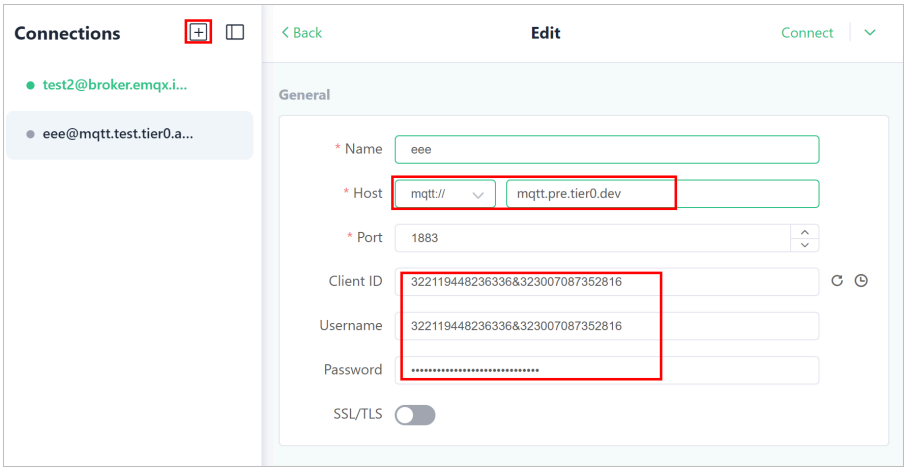
Connecting MQTT Clients with Credentials

> **NOTE**
>
> The following clients are used as examples to show how to use the credentials to connect to the internal MQTT broker.

1. Install an MQTT client, such as MQTTX, MQTT Explorer and others.
2. Use the credentials from Tier0 to connect the client.

- **MQTTX**

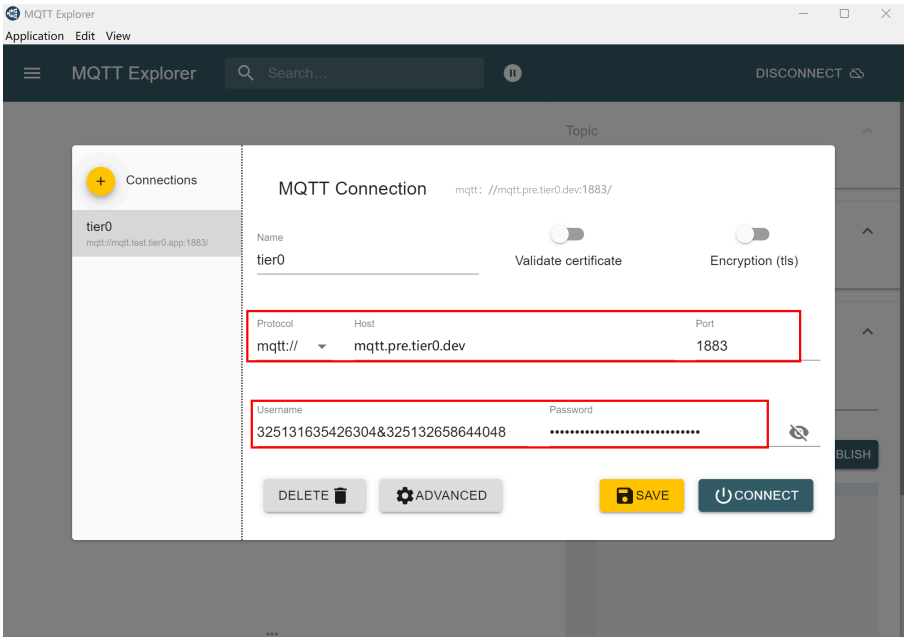
1. Add a new connection in MQTTX with the generated credentials.



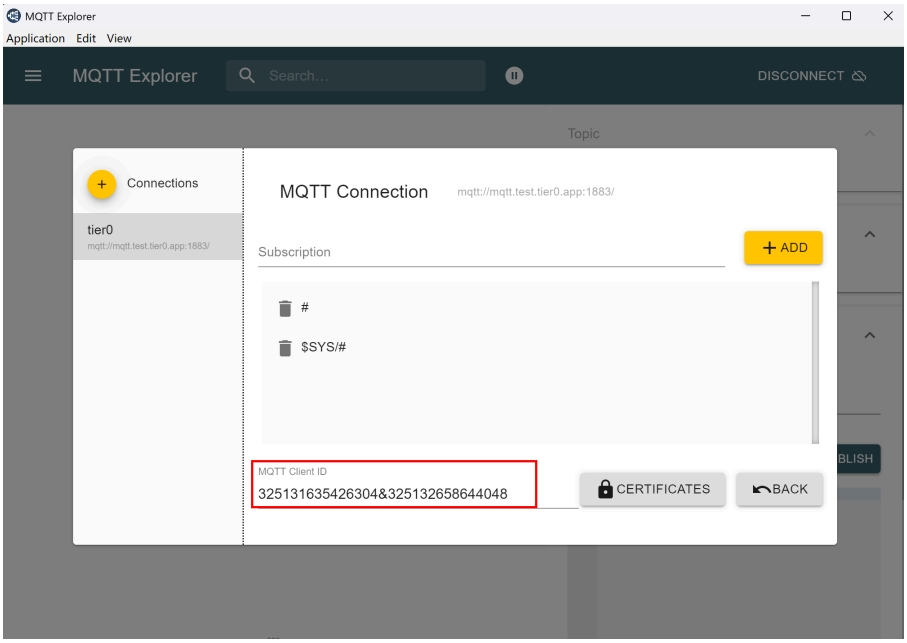
2. Click **Connect**.

- **MQTT Explorer**

1. Add a new connection in MQTT Explorer with the generated credentials.



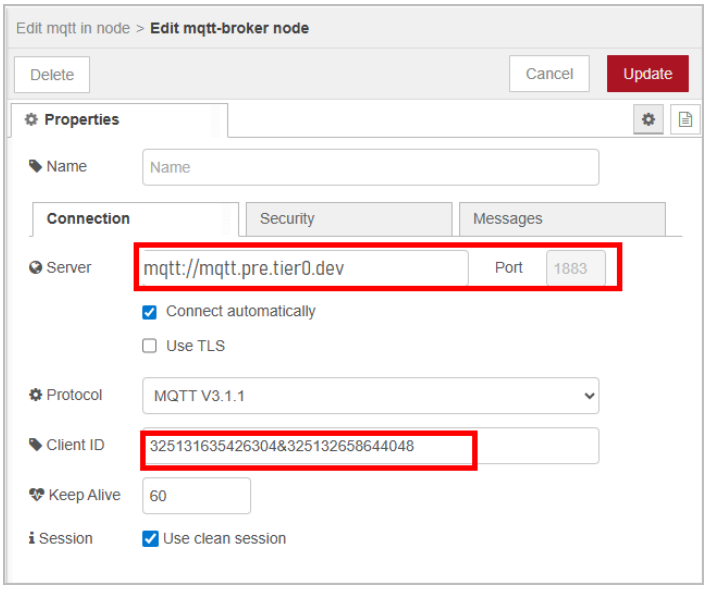
2. Click **ADVANCED**, change the client ID.

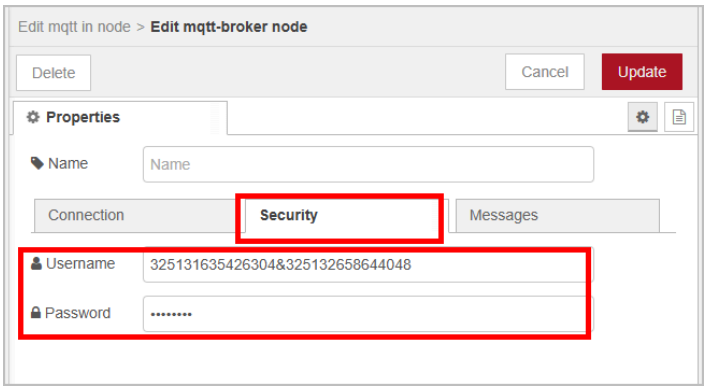


3. Click **BACK** and then **CONNECT**.

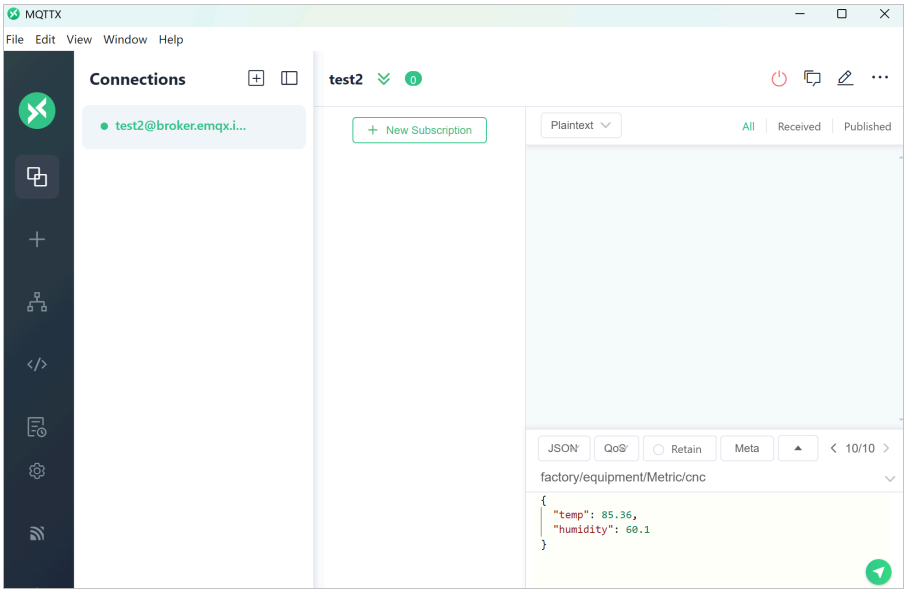
- **Node-RED**

1. Open your Node-RED instance, double-click the `mqtt in` node.
2. Add a new MQTT broker with the generated credentials from Tier0.





3. Go back to the **Edge** page on Tier0, click the **Clients** tab and check whether the client is connected.
4. Copy the topic of the data model under **UNS**, and publish a message to it through the client.

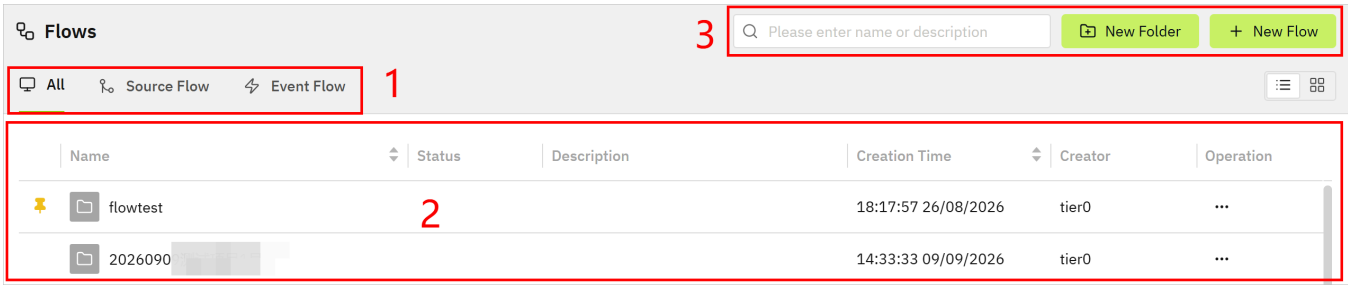


5. Go back to **UNS**, check whether the data has been received under the target topic.

Process Data

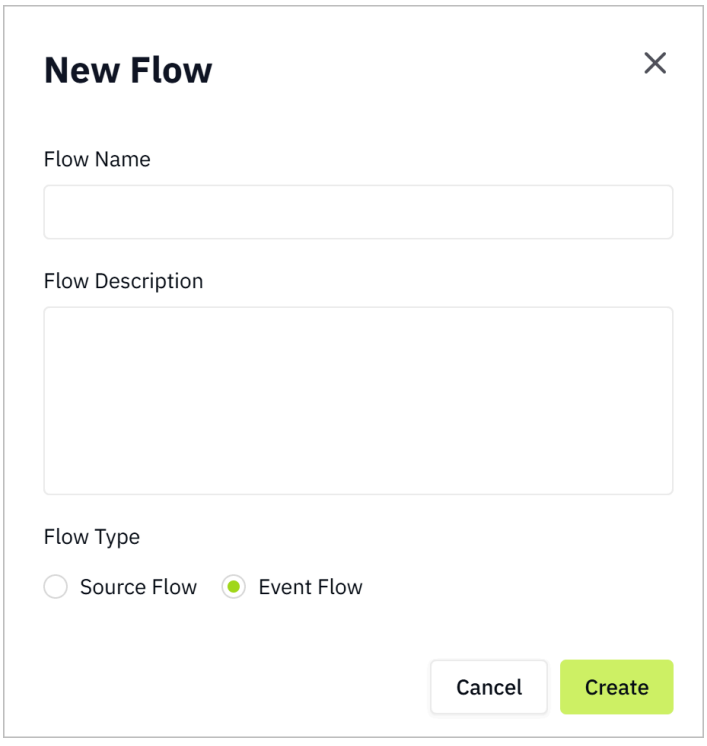
Process Data

1. Log in to Tier0, and then select **Flows**.



Index	Item	Description
1	Flow Display	Select to view all flows or only one category.
2	Flow List	Displays flow information. Click the flow name to start editing the data flow.
3	Search/Filter	Search for/filter existing data flows.
	New Folder	Add folders to categorize data flows.
	New Flow	Add new data flows.

2. Click **New Flow**, enter name and description, select **Event Flow** as **Flow Type**, and click **Save**.



3. Click the flow you just added, and build data events to process your data.

Combining Multiple Sources

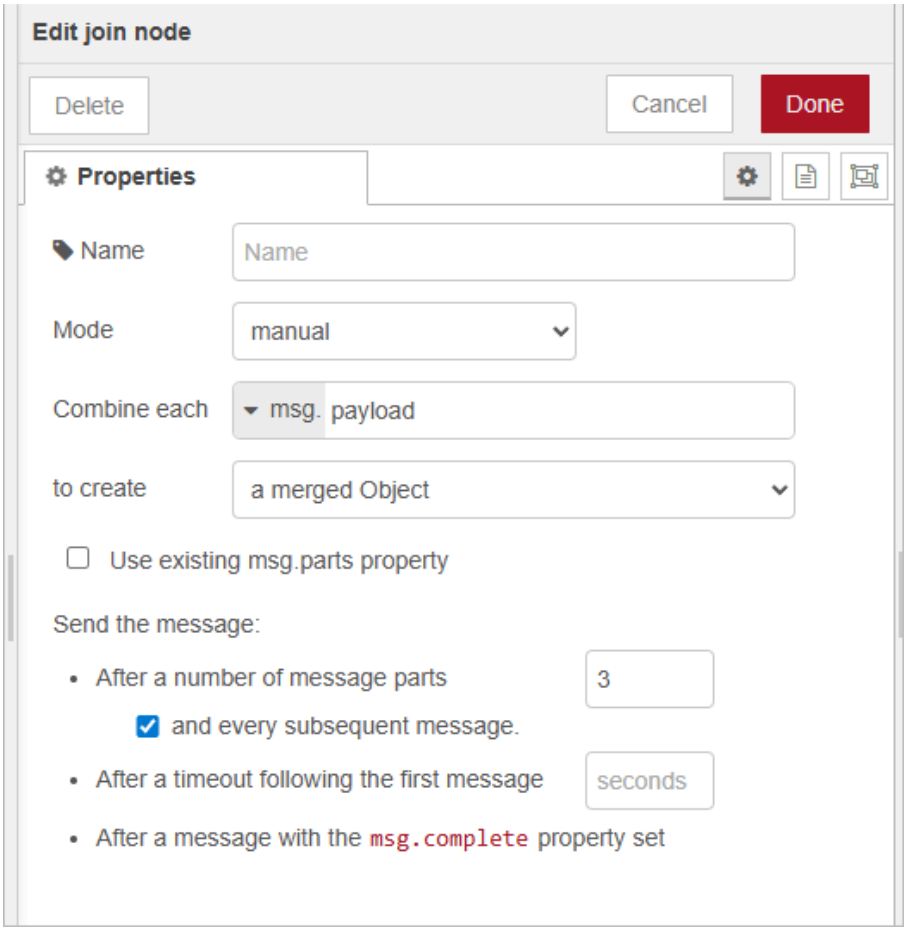
Example

There are 3 data sources connected to Tier0, and we need to combine them together for the next step.

```
{
  "workshop": {
    "furnace1": {
      "temp": 85
    },
    "furnace2": {
      "humidity": 64
    },
    "furnace3": {
      "pressure": 80
    }
  }
}
```

What Node to be Used?

In this example, we use **join** node to combine previous messages.

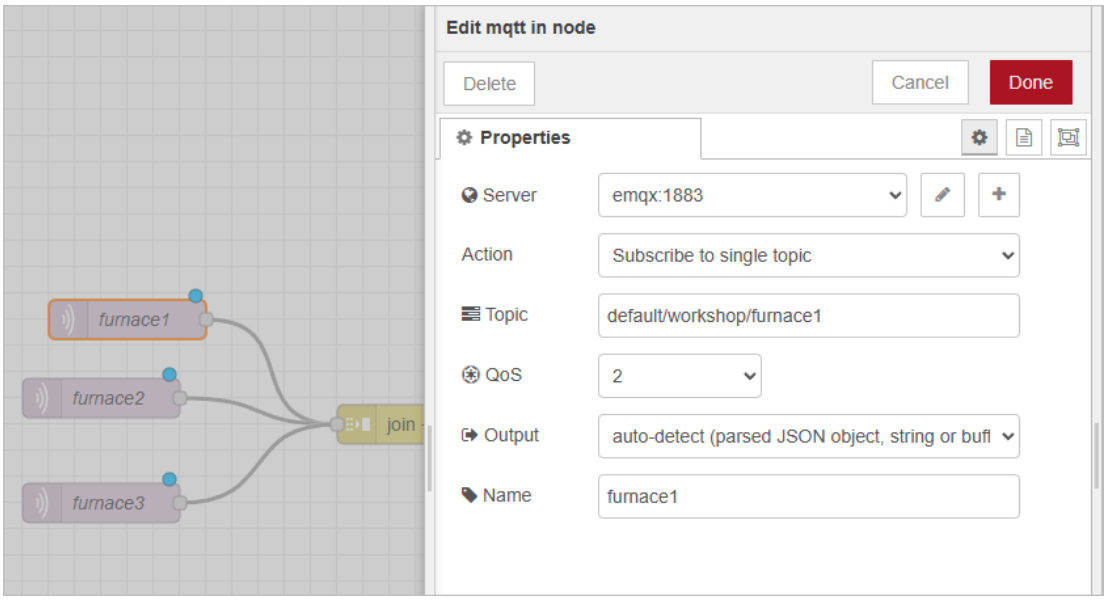


Parameter	Description
Mode	Join mode. • **manual**: You configure how to merge messages yourself. • **auto**: Automatically reassemble sequences split by a **split** node, based on `msg.parts`. • **reduce sequence** – Runs a JavaScript reduce function across the sequence to produce a single result.
Combine each	Message part you want to combine. Commonly `msg.payload`, but can be others (e.g. `msg.data.value`).
to create	Result type: • **a merged Object**: Merge all objects into a single object (later keys overwrite same-name keys). • **a key/value Object**: Use a **key property** (default `msg.topic`) as the key, and the **Combine each** property as the value. • **an Array**: Append each value to an array in order. • **a String / a Buffer**: Concatenate text or binary data (String can use a separator; Buffer is joined as binary).
Use existing `msg.parts` property	When selected, the node uses the `msg.parts` from previous nodes (usually created by a **split** node) to determine grouping and when to send the joined output. Fields like `msg.parts.count` and `msg.parts.index` are used.

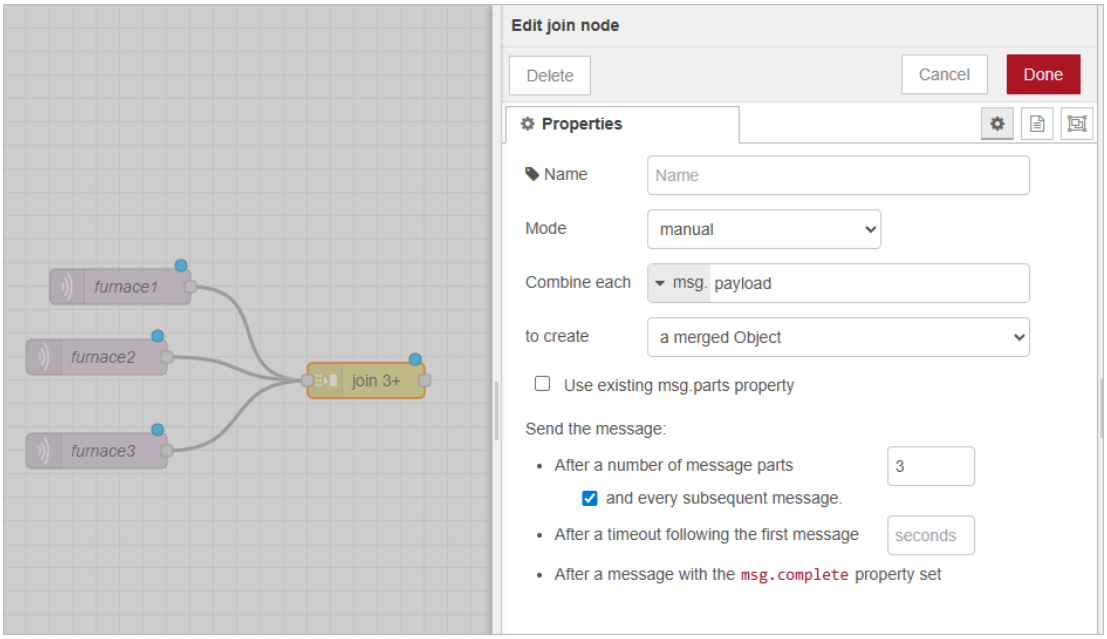
Parameter	Description
Send the message → After a number of message parts	Outputs the joined result after receiving the set number of fields.
Send the message → and every subsequent message	When selected, after the initial output, the node outputs an updated result upon new messages arrival.
Send the message → After a timeout following the first message	Optional. Starts a timer when the first message arrives; once the timeout (seconds) is reached, the node outputs whatever has been collected.
Send the message → After a message with the `msg.complete` property set	Outputs the joined result immediately when a message arrives with `msg.complete = true`.
(Advanced) String/Buffer separator	For **String** output, an optional separator can be set. For **Buffer** output, data is concatenated directly with no separator.

How to Combine Sources?

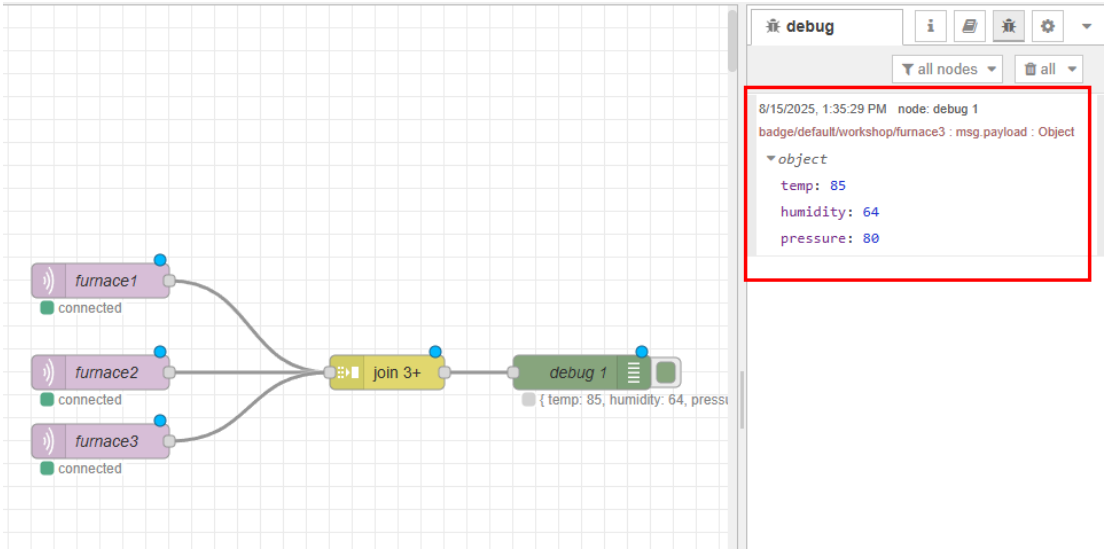
1. In **Event Flow**, drag in 3 **mqtt in** nodes, and subscribe to corresponding topics relatively.



2. Connect a **join** node to all 3 nodes, and merge them into an object.



3. Connect a **debug** node, and trigger the flow to check the results.



Filtering Data

Example

The **furnace** data source contains 3 fields, and we want to route messages based on the temperature threshold:

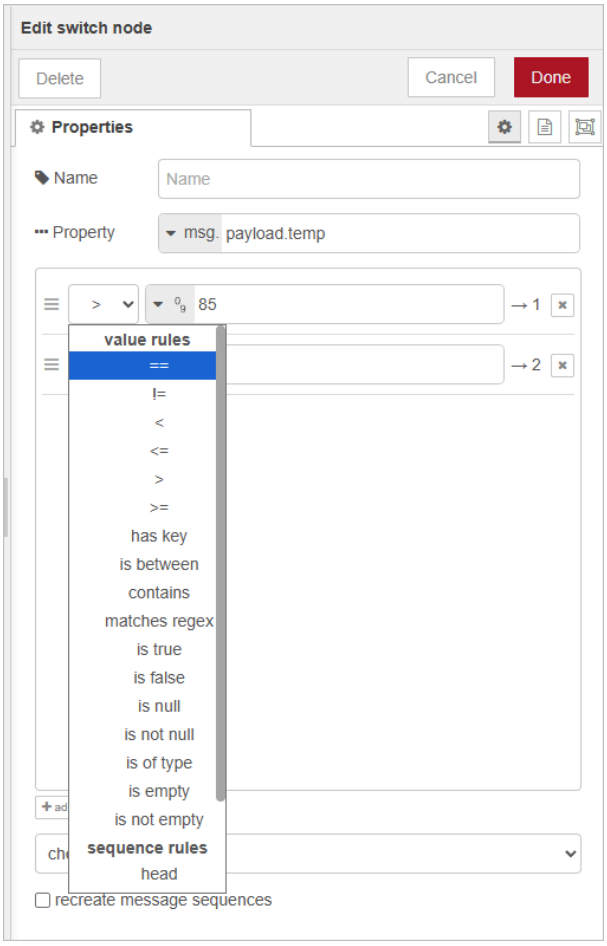
- If **temp > 85** → go to **Output 1**
- If **temp ≤ 85** → go to **Output 2**

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
```

```
    }
}
```

What Node to be Used?

In this example, we use the **switch** node to route messages according to conditions.

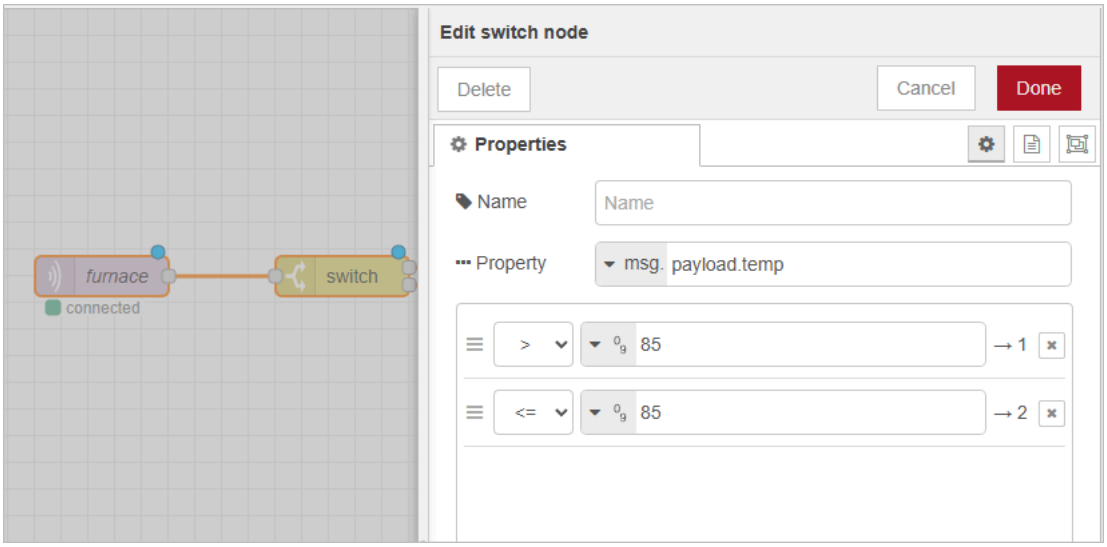


Parameter	Description
Property	The property path to evaluate. Commonly `msg.payload` or a deeper property, such as `msg.payload.temp` in this example.
Rules	A set of evaluation rules, each corresponding to one output port.
Type	The data type for comparison: number, string, boolean, JSONata, etc. **INFO** Ensures numeric comparisons are not treated as string comparisons.

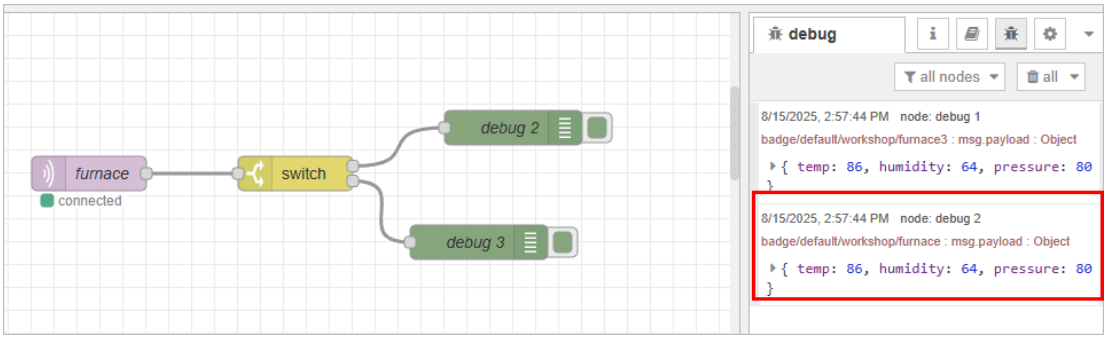
Parameter	Description
Value	The value to compare against can come from: a constant value (e.g. 85), `msg.` fields (e.g. `msg.threshold`), `flow.` / `global.` context, or a `JSONata` expression.
checking all rules	Evaluates all rules in order. If a message matches multiple rules, it will be sent to multiple outputs.
Stop after first match	Stops checking once the first rule matches, so the message is sent to only one output.
recreate message sequences	When enabled, preserves and rebuilds message sequence metadata (msg.parts)—usually created by a split node—so that the sequence is maintained even after branching.

How to Filter Data?

1. In **Event Flow**, add an **mqtt in** node to get the data from Tier0.
2. Drag in a **switch** node and configure its properties.



3. Connect a **debug** node to each output of the **switch** node.
4. Trigger the flow and check the results.



Changing Data

Example

We receive a message and need to rename fields, calculate new values, and remove unused fields for cleaner downstream processing.

Input message

```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```

Target transformation

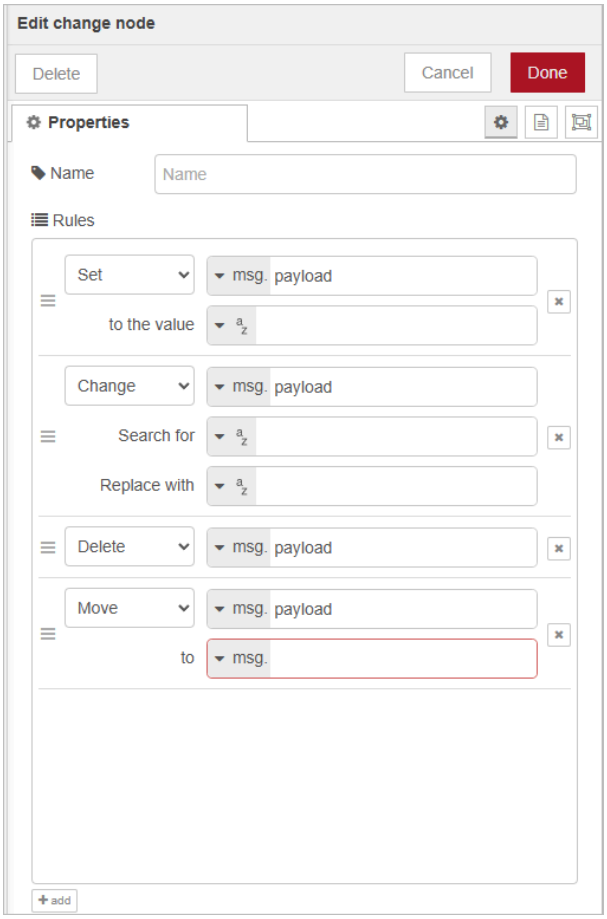
- Move furnace.temp_c to furnace.temperature.celsius
- Add furnace.temperature.fahrenheit (converted from Celsius)
- Remove pressure
- Change the site from A1 to B1

Output message (example)

```
{
    "furnace":{
        "humidity":64,
        "temperature":{
            "celcius":30.5,
            "fahrenheit":86.9
            }
        },
    "site":"B1"
}
```

What Node to be Used?

In this case, we use the **change** node to perform operations like **Set**, **Change**, **Delete** and **Move** on message properties.



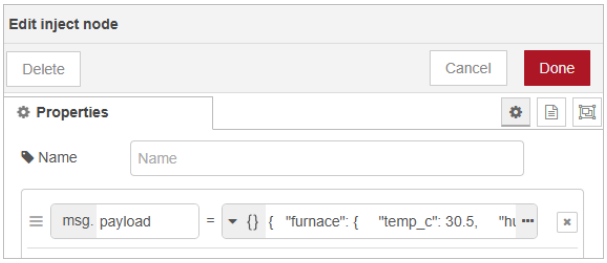
Parameter	Description
Rules	Each rule is executed in order, top to bottom. Multiple rules can be added to perform complex transformations.
Rule Type	• **Set**: Set or create a property value. • **Change**: Find and replace. • **Delete**: Remove a property. • **Move**: Move a property from one path to another.
Property/Source/Target	Defines which property to operate on, such as `msg.payload` or `msg.payload.furnace.temp_c`. Can also reference `flow.` or `global.` context variables.
Target Value	• **Set**: The value to assign. • **Change**: The value to find and replace.

How to Use the Change Node?

1. In **Event Flow**, drag in an **inject** node, and add the following json as the input.

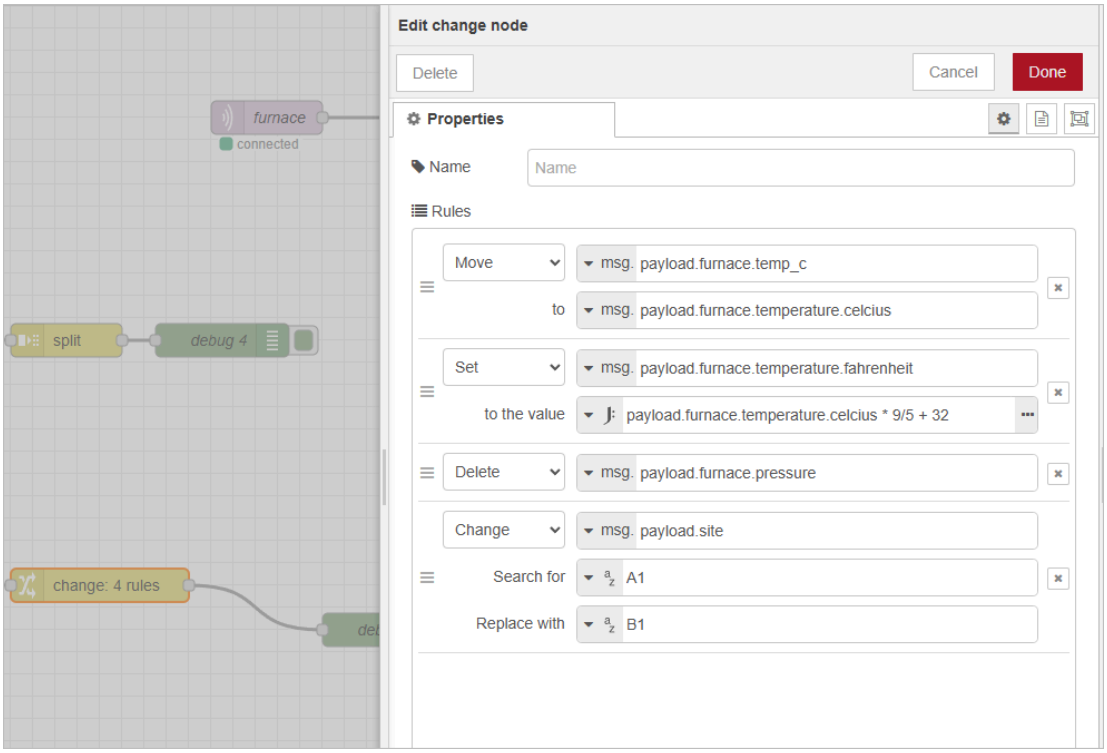
```
{
  "furnace": {
```

```
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```

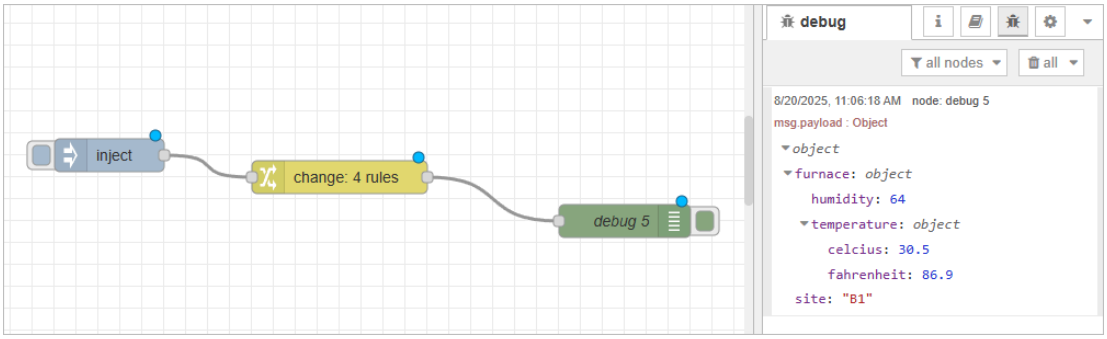


2. Connect it to a **change** node.
3. Open the **change** node configuration panel and add rules.

- Move msg.payload.furnace.temp_c → msg.payload.furnace.temperature.celcius
- Set msg.payload.furnace.temperature.fahrenheit → payload.furnace.temperature.celcius * 9/5 + 32
- Delete msg.payload.pressure
- Change value of msg.payload.site from A1 → B1



4. Connect the **change** node to a **debug** node to check the transformed output.



Spliting Data

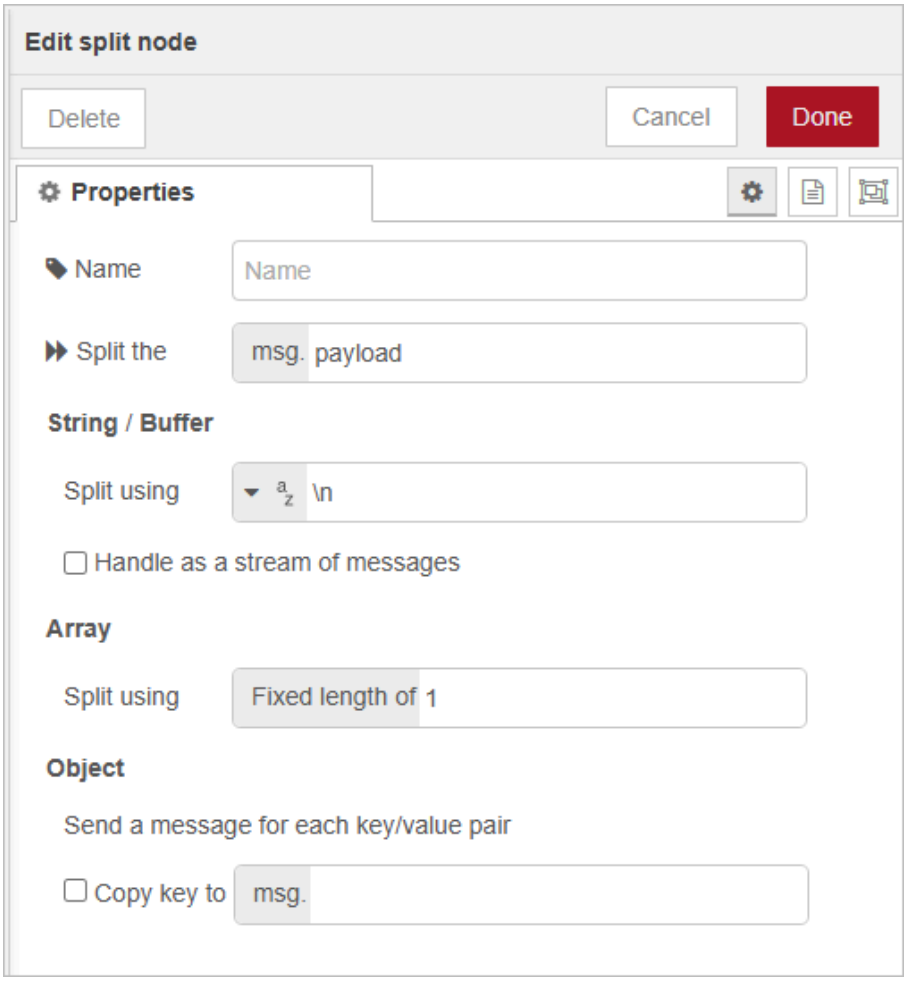
Example

We have a single message containing multiple furnace readings, and we need to split them into individual messages for further processing.

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
  }
}
```

What Node to be Used?

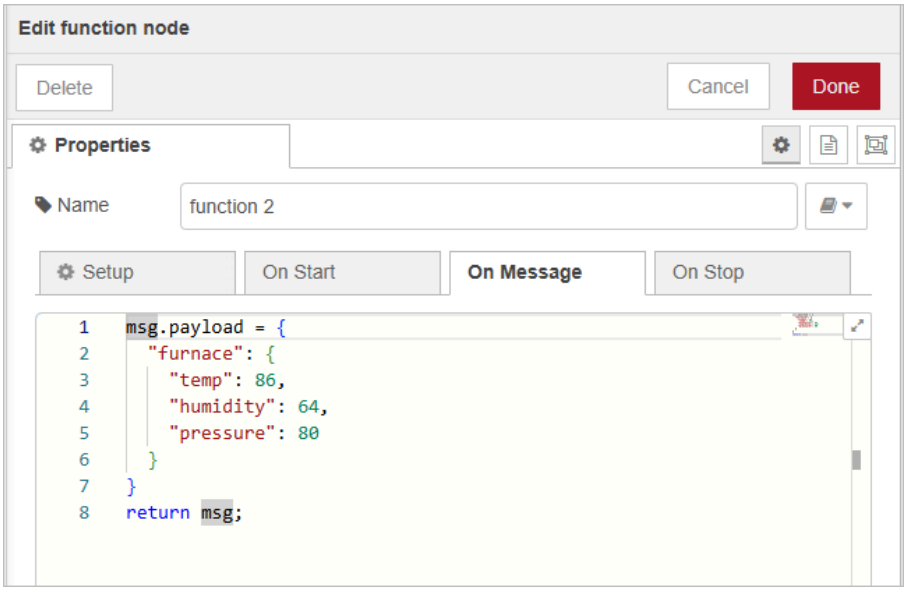
In this example, we use the **split** node to break one message into multiple smaller messages.



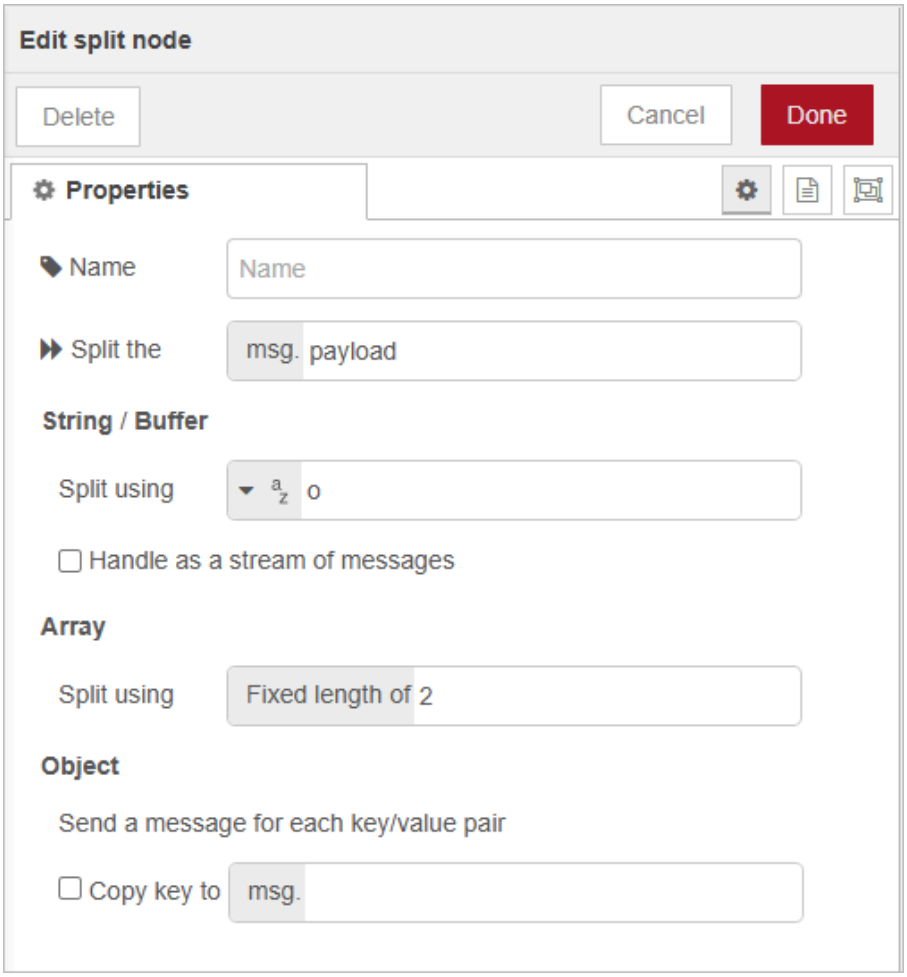
Parameter	Description
Name	Optional label for the node to help identify it in the flow.
Split the	The property of the message to split, usually `msg.payload`, but can be any property such as `msg.payload.data`.
String / Buffer - Split using	For string or buffer payloads, specifies how to split: • **string**: Use a specific string or regular expression pattern to split the message. • **buffer**: Defines chunk size by which the message is split. • **Fixed length of**: Split the string according to the set length. • Leave empty to return the tring unsplit.
Handle as a stream of messages	When checked, the node will process incoming data as a continuous stream, emitting split messages as data arrives, rather than waiting for the full payload.
Array - Split using	**Fixed length of N**: Splits the array into multiple messages with N elements each.
Object - Send a message for each key/value pair	The value of each key/value pair will be returned as payload.
Copy key to `msg.payload`	If checked, the original key will be returned as `msg.payload` of the split messages.

How to Split Messages?

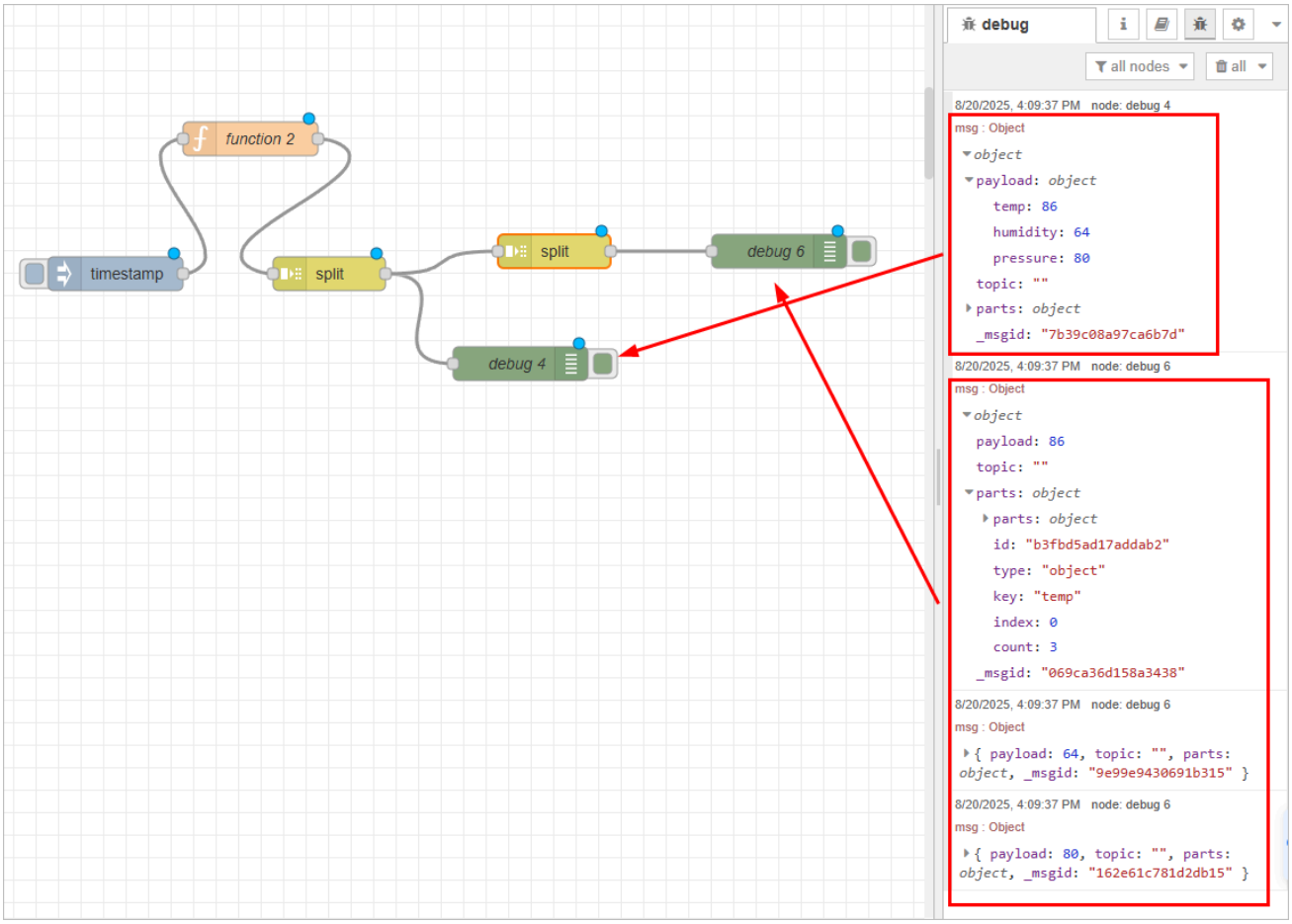
1. In **Event Flow**, drag in an **inject** node, and use a **function** node to transmit the object.



2. Connect it to 2 **split** nodes in order, and configure both properties as the same on the **Object** part.



3. Connect the **split** node to a **debug** node to inspect each individual output message.

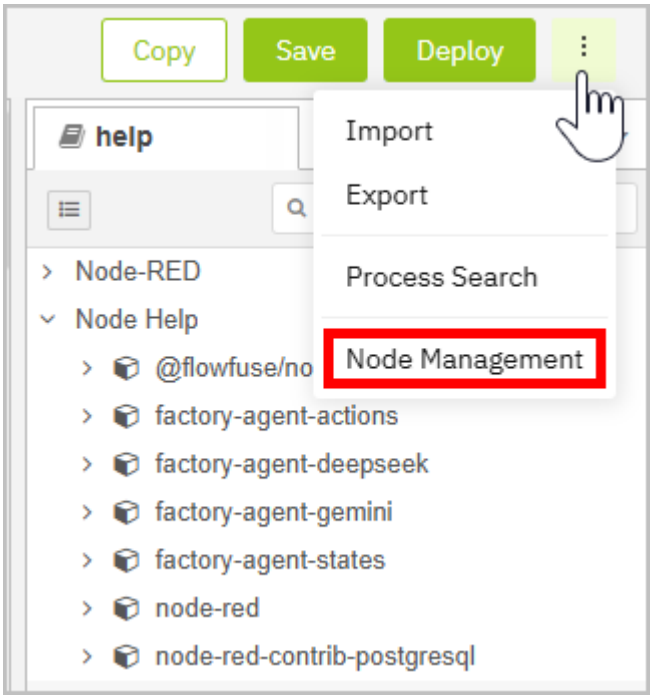


Visualize Data

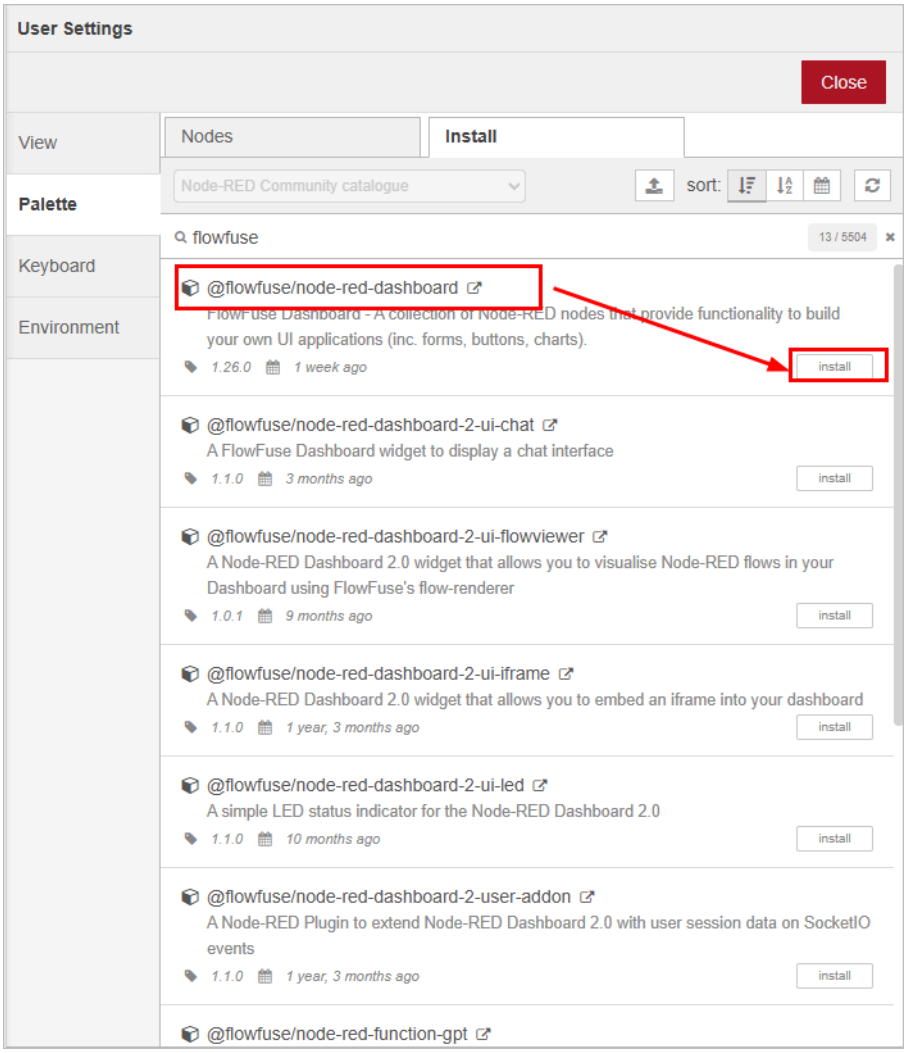
How to Build a Dashboard on Event Flow

Data collected to, processed on Tier0, can be visualized in charts, tables, texts in custom styles and other formats on Tier0 as well.

1. Log in to Tier0, and select **Unified Namespace** > **Flows**.
2. Click **New Flow** to add an event flow.
3. Click the flow name to start editing.
4. Click ⋮ at the upper-right corner, and then click **Node Management**.



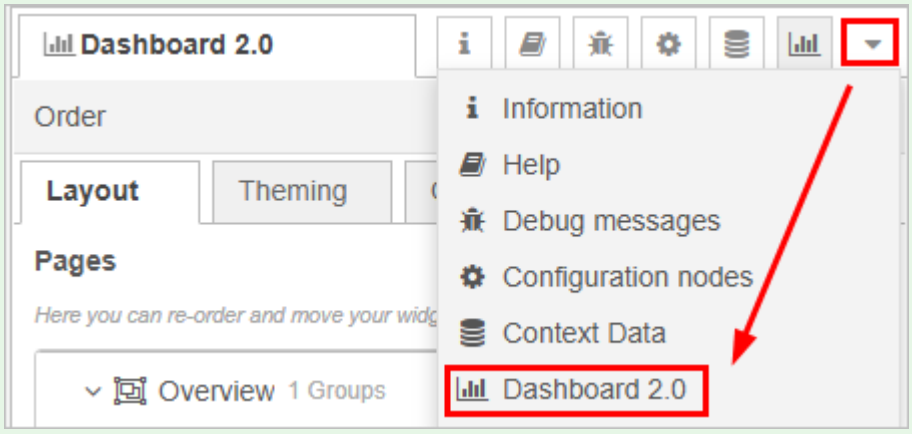
5. Under **Install**, search for and install `@flowfuse/node-red-dashboard`.

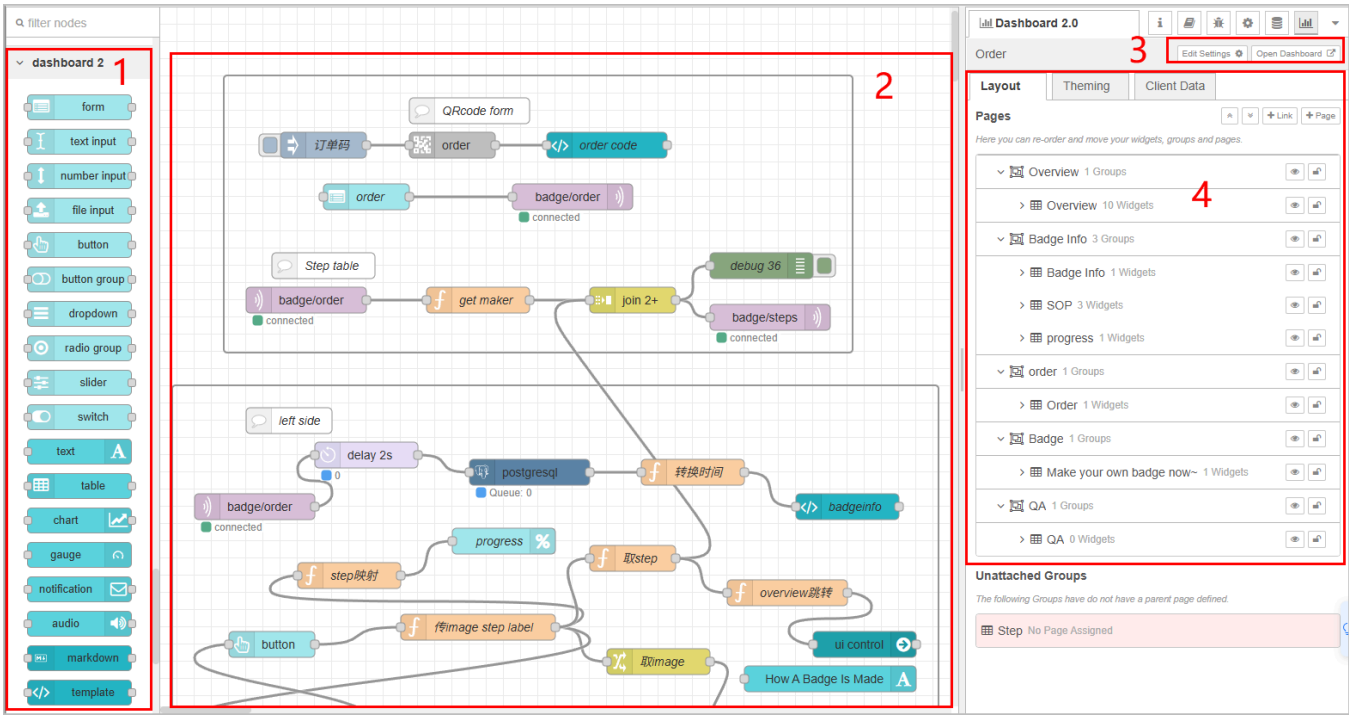


6. Use nodes under flowfuse to build a dashboard.

TIP

Click at the upper-right corner, and then click **Dashboard2.0** to adjust the layout.





Index	Item	Description
1	Node	Node of Dashboard 2.0, including gauge, form, progress bar and more.
2	Canvas	Where you drag in nodes to build a dashboard.
3	Operations	Edit the `ui-base` node and view the dashboard.
4	Layout	Displays all nodes in the corresponding hierarchical layout.

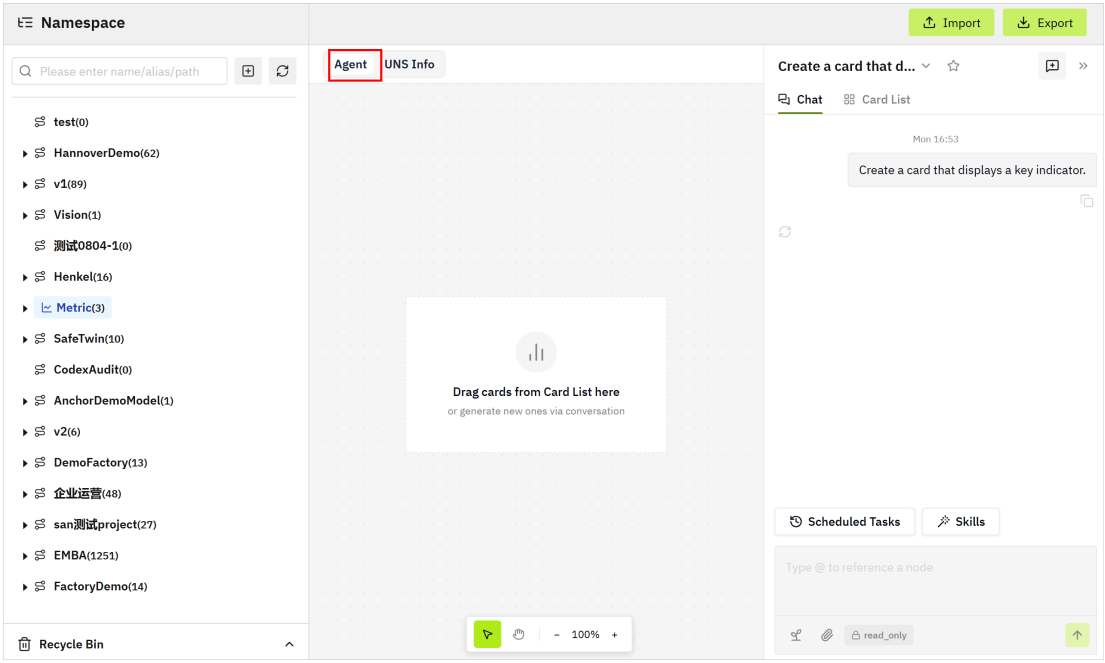
> **INFO**
>
> For details on how to use each node, see Flowfuse Node Docs.

Work with UNS Agent

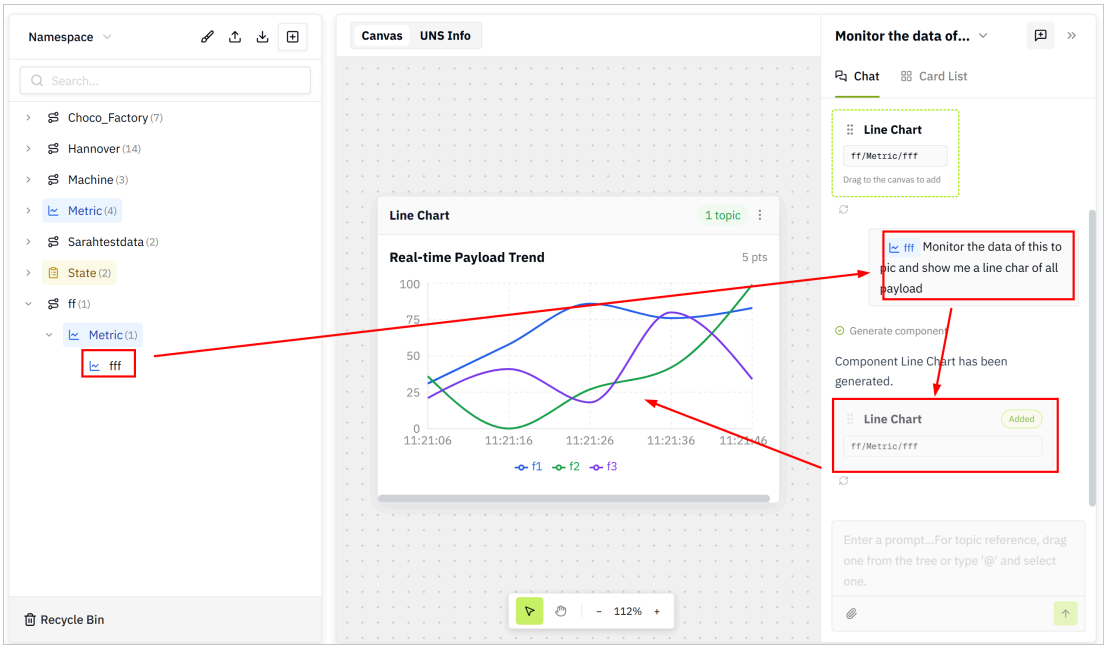
Tier0 adopts AI agent inside the UNS module, enabling you to interact with the UNS models simply by talking to the agent in natural language.

> **CAUTION**
>
> Before starting a conversation with the agent, you need to set the API key for the agent in > **Settings** > **AI Settings**. Otherwise the agent will not be able to respond to your instructions.

1. Log in to Tier0, in **UNS**, select the **Agent** tab.



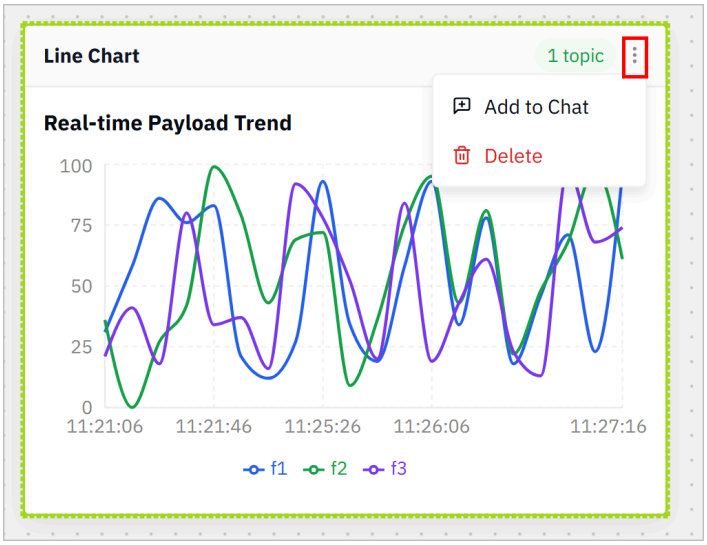
2. In the chat window on the right, drag a topic from the left side and drop it into the input box or directly @ and select a topic for data reference, and then enter your instructions.
3. Drag the cards generated based on your instructions to the canvas, and you can see the visualized data.



TIP

- Click at the lower-left corner of the conversation message to send the last instruction again.
- Click **Card List** at the top to see all the generated cards in the current session.
- Click at the upper-right corner to start a new session.

4. (optional) Click at the upper-right corner of the card to add it to the chat or delete it.



WHAT DOES THE AGENT SUPPORT?

The UNS agent can take over the following tasks:

- Generate data models based on your instructions.
- Generate data flows based on your instructions, including **Source Flow** for data connection and **Event Flow** for data processing.

Advanced Guide

Sync Data

Sync Data between Center and Edge

Tier0 Enterprise is classified into 2 versions based on the license type.

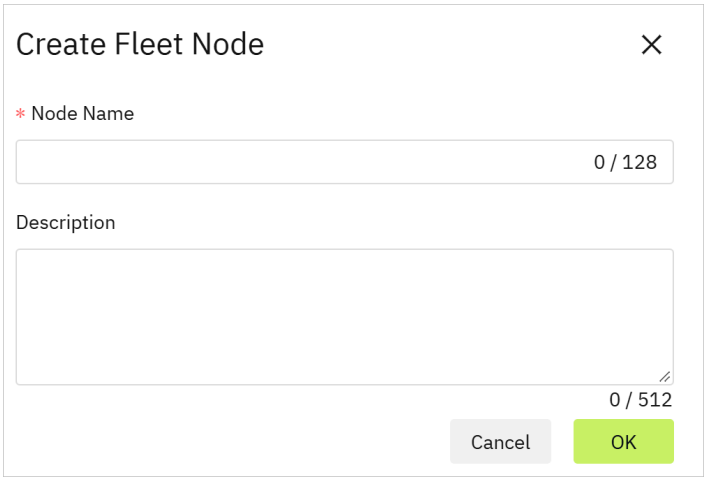
- **Center Node**: When the license type is **Cluster**, the corresponding Tier0 is the center node, which can sync data from edge nodes and assign users to manage edge nodes.
- **Edge Node**: When the license type is **Base** and **Plus**, the corresponding Tier0 is the edge node, which can sync data to the center node.

Connecting Edge to Center

INFO

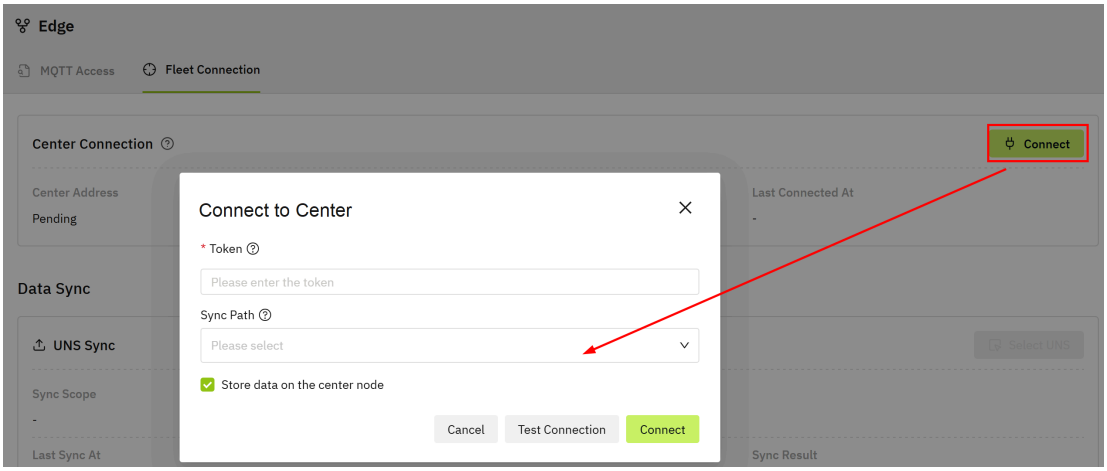
An edge node can be connected with both Tier0 Enterprise (center node) and Tier0 Cloud.

1. In a center node, select **Edge** > **Fleet Management**.
2. Click **Create Fleet Node**, enter the name and click **OK**.



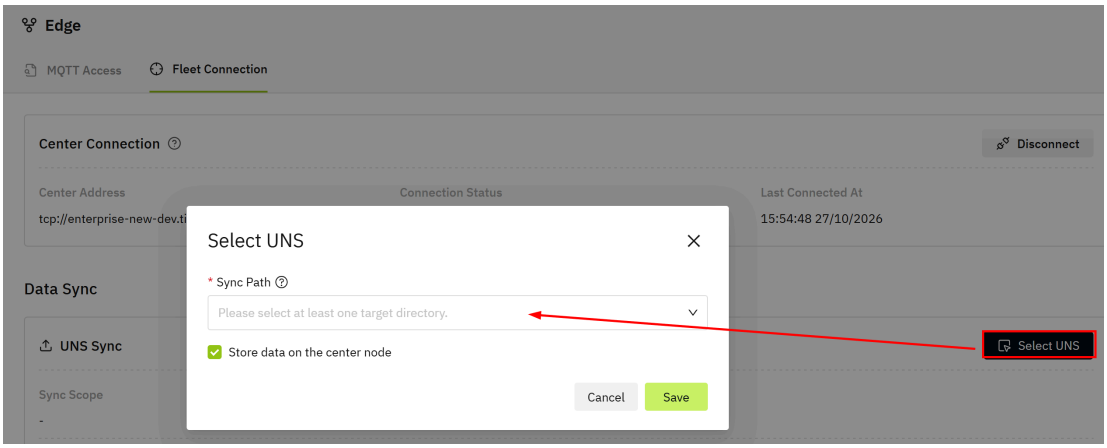
3. Copy the access token, and then go to the **Edge** > **Fleet Connection** page on an edge node.
4. Click **Connect**, enter the access token, and click **Connect**.

> **NOTE**
>
> Once connected, information like resources, containers, UNS, operations of the edge node will be displayed on the center node.



Syncing Data from Edge to Center

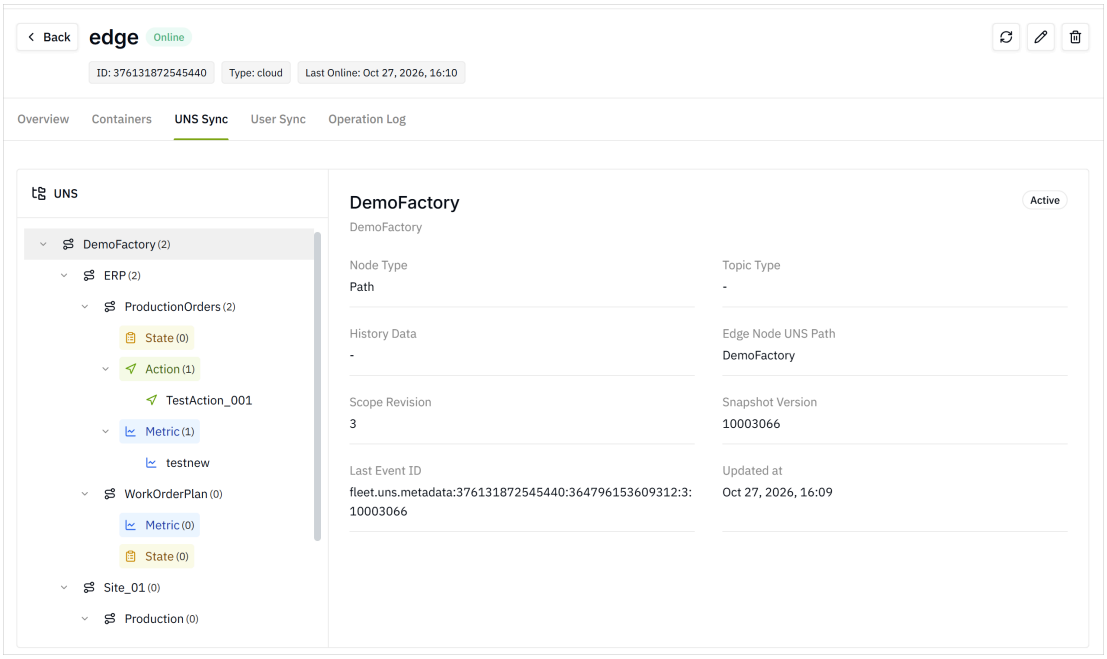
1. On the **Fleet Connection** page on the edge node, click **Select UNS**.



2. Select UNS paths to sync to the center node, and click **Save**.

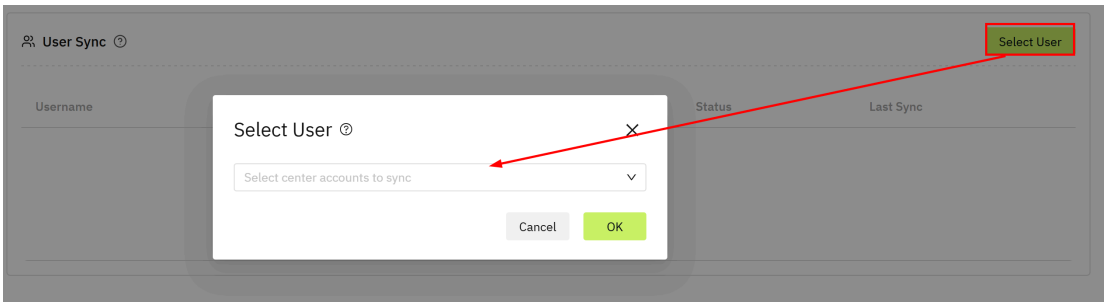
> **INFO**
>
> Select **Store data on the center node**, and all the data under the selected UNS paths will be synced to the center node. Otherwise, only the UNS structure is synced.

3. Go back to the center node and check the synced UNS.



Assigning Users to Manage Edge

1. On the center node, select the connected edge node in **Fleet Management**.
2. Under **User Sync**, click **User Sync** to select users to manage the edge node.



3. Click **OK**, and go to the edge node.
4. Under **Fleet Connection**, click **Sync User** to sync the assigned users.
5. Once the synchronization is complete, the assigned users can log in to the edge node and manage it.

Obtain Data from Tier0

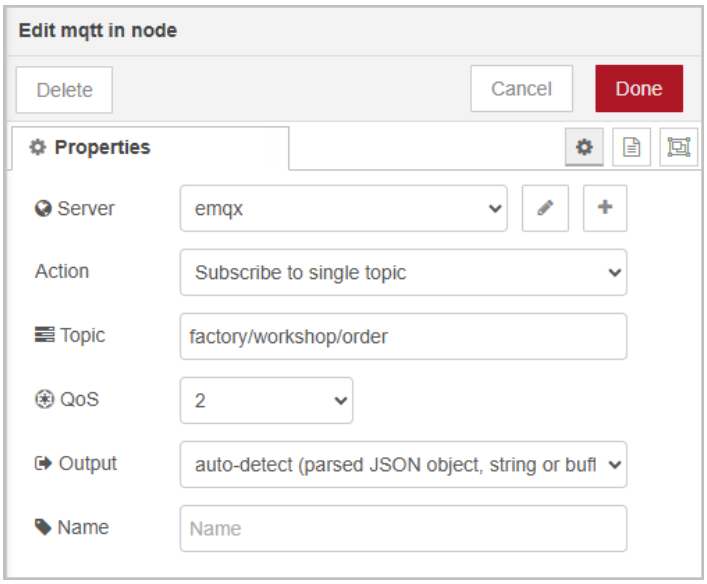
Data can be collected to Tier0 through various protocols, it can also be obtained through MQTT subscription, or even directly from Tier0 database.

Obtaining Data through MQTT Subscription

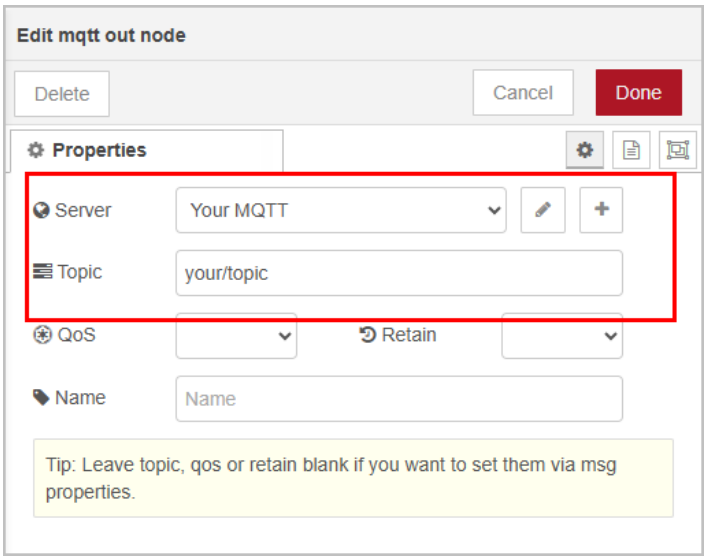
> **INFO**
>
> This method is only suitable for third parties who have experience in MQTT protocol.

1. Log in to Tier0, and then select **Flows** > **Event Flow**.

2. Click **New Flow**, select **Event Flow** as the flow type, enter a name, and click **Save**.
3. Click the flow you just added, drag an `mqtt in` node, and enter the configurations.
    - Tier0 MQTT broker address: `emqx:1883`
    - Topic: The topic to obtain data from, for example, `factory/workshop/order`.



4. Process the data obtained, or directly use an `mqtt out` node to transmit the data to the third party MQTT.



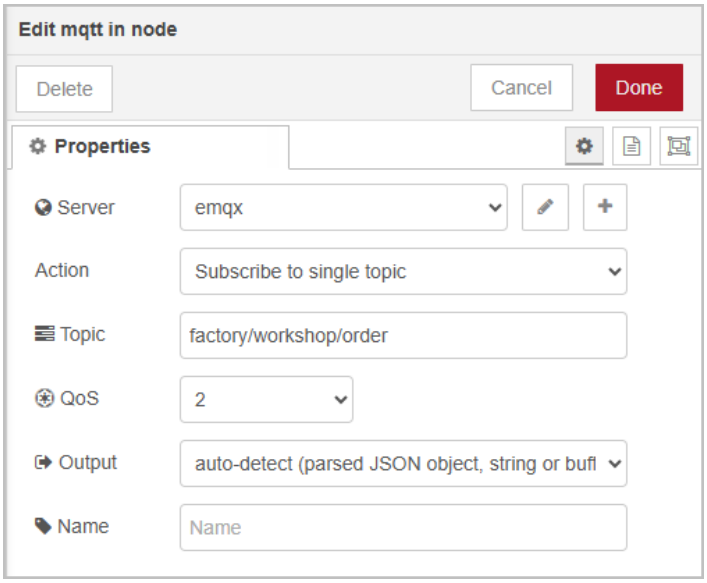
5. Deploy the flow, and data will be pushed to the third party when the topic data changes.

Obtaining Data through API

> **INFO**
>
> This method is suitable for third parties who do not support MQTT and need event-driven data.

1. Log in to Tier0, and then select **Flows** > **Event Flow**.
2. Click **New Flow**, select **Event Flow** as the flow type, enter a name, and click **Save**.
3. Click the flow you just added, drag an `mqtt in` node, and enter the configurations.
    - Tier0 MQTT broker address: `emqx:1883`

- Topic: The topic to obtain data from, for example, `factory/workshop/order`.



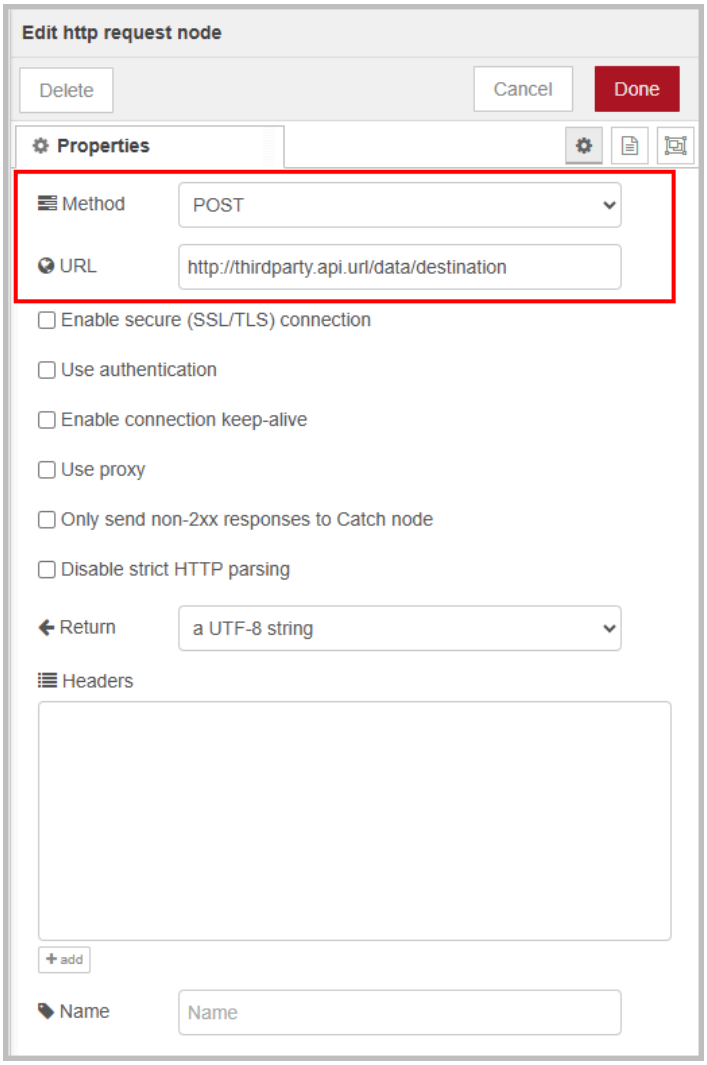
4. Drag a `function` node to the canvas, and write a script to format the data obtained according to third-party receiver API.

```
if (typeof msg.payload === "string") {
    msg.payload = JSON.parse(msg.payload);
}

msg.headers = {
    "Content-Type": "application/json"
};

return msg;
```

5. Use an `http request` node to send the data to the third party through API.

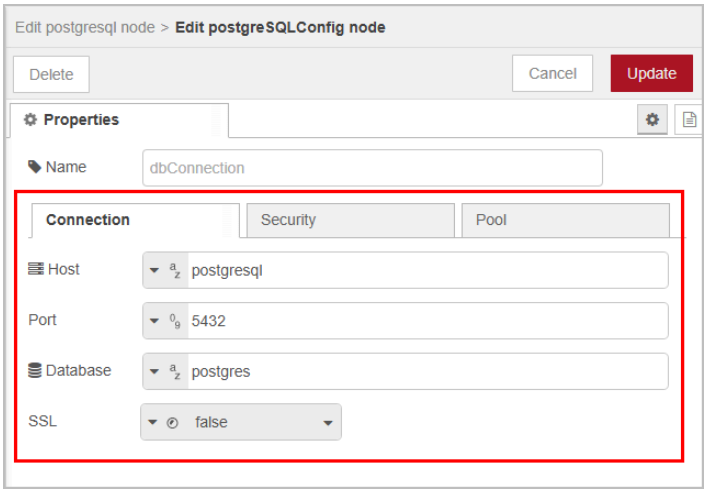


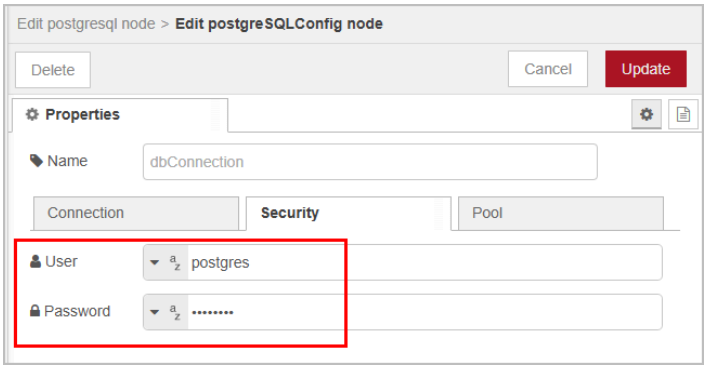
6. Deploy the flow, and data will be pushed to the third party when the topic data changes.

Obtaining Data Directly from Tier0 Database

> **INFO**
>
> This method is suitable for requests on history data.

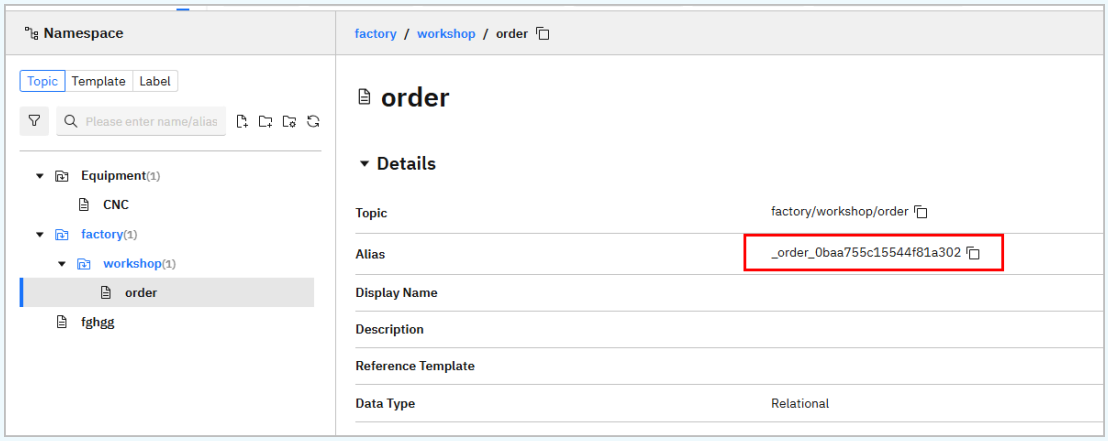
1. Log in to Tier0, and then select **Flows** > **Event Flow**.
2. Click **New Flow**, select **Event Flow** as the flow type, enter a name, and click **Save**.
3. Click the flow you just added, drag an `inject` node.
4. Drag a `postgres` node to the canvas, and connect to Tier0 PostgreSQL database.
    - **Connection**
        - Host: `postgresql`
        - Port: `5432`
        - Database: `postgres`
        - SSL: `false`
    - **Security**
        - User: `postgres`
        - Password: `postgres`



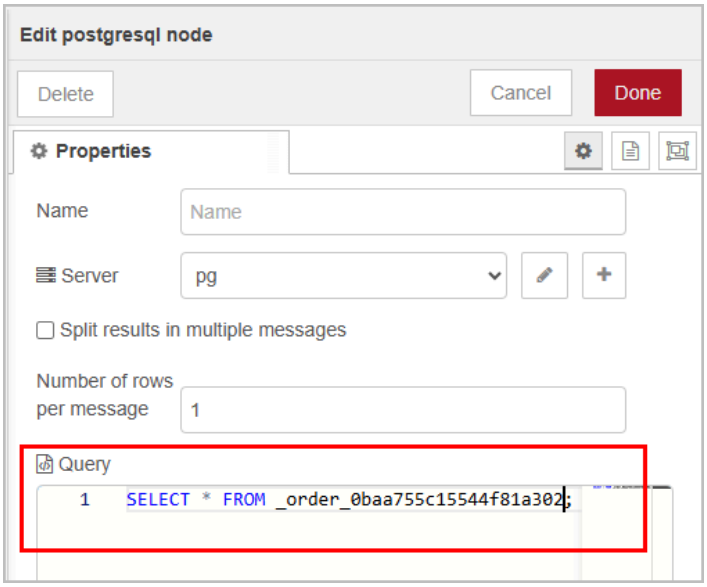


5. Write SQL statements to query data from Tier0 database.

> **INFO**
>
> Get the table name from topic **Details** under **Namespace**.




```
SELECT * FROM _order_0baa755c15544f81a302;
```



6. Add an `http request` or `mqtt out` node to transmit the queried data to the third party.

Use Data

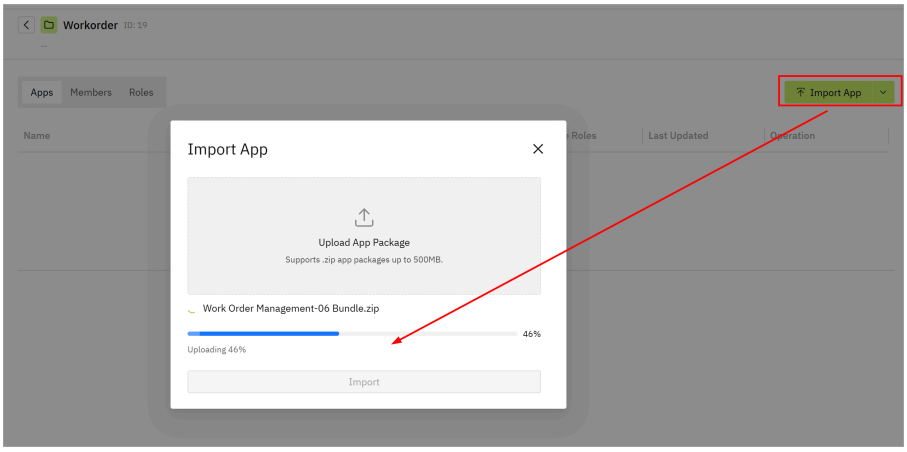
Industrial Apps

Tier0 Cloud provides App Builder for you to build industrial applications through no code writing yourself, and export them to local Tier0 Enterprise.

Importing Apps

> **GET THE APP PACKAGE FIRST**
>
> In Tier0 Cloud, export the bundle of the application you built. For details, see Exporting Apps.

1. In Tier0 Enterprise, go to **App** > **Projects** and click **Create Project**.
2. Enter project name and click **Save**.
3. Inside the project, click **Import App** and select the app package you exported from Tier0 Cloud.

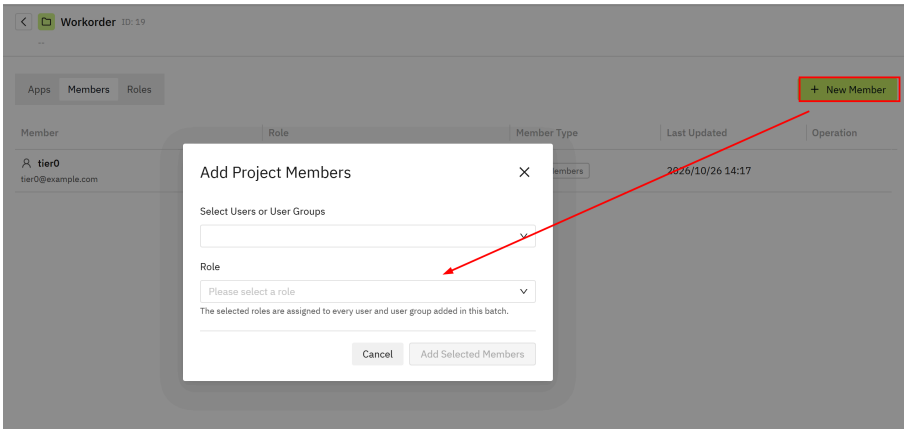


4. Check the application information and then click **Import**.

Configuring App

1. Inside the project, go to the**Members** tab.
2. Add members to the project and assign them roles so they can access the application within the granted permission scope.

> **ROLES**
>
> - Roles are created during application development in Tier0 Cloud. For details, see Roles.
> - Roles are listed under the **Roles** tab in the project.



Accessing App

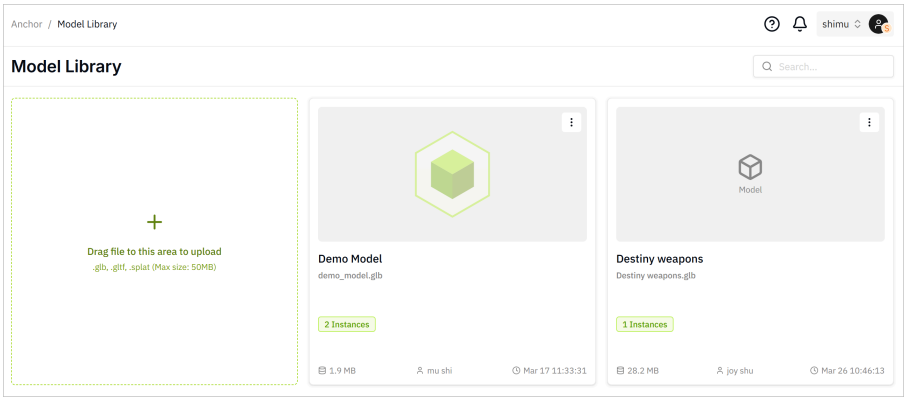
1. Go to **App** > **Launchpad** and select the project.
2. Click the application you want to access.

Configure 3D Models

The digital twin feature is available on the platform to help you visually build data models.

Uploading Digital Twin Models

1. Log in to Tier0, select **Anchor** > **Model Library**.

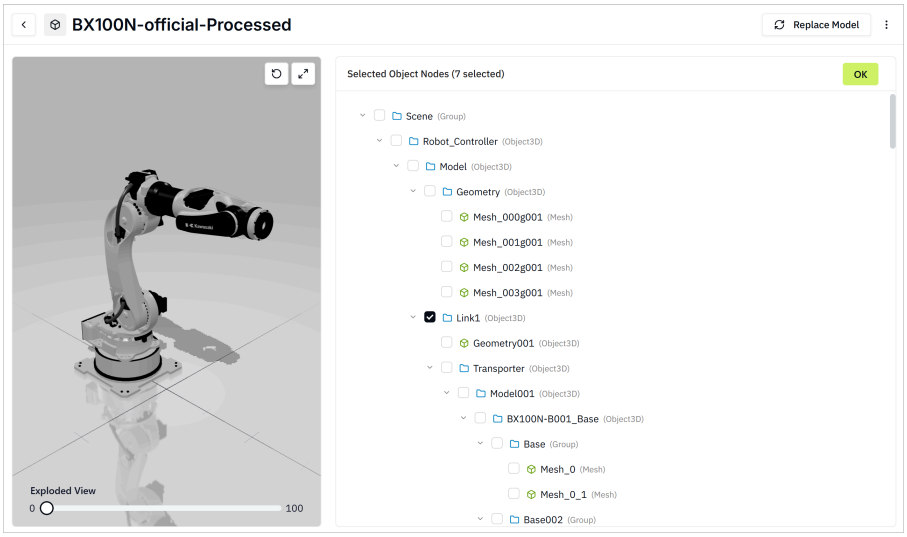


2. Click +, select a model file, and upload it to the platform.

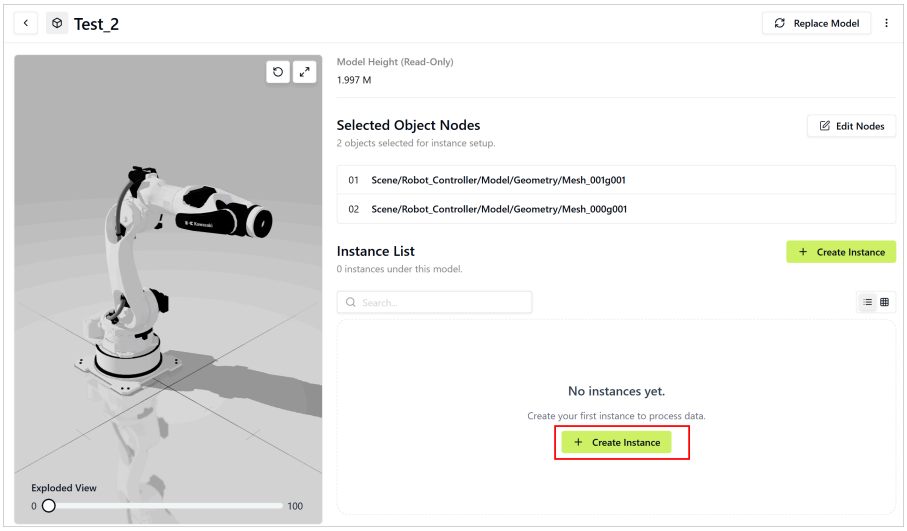
Mapping UNS Data to Digital Twin Models

> **INFO**
>
> There are 2 types of data mapping with 3D models.
>
> - **Static Mapping**: Displays UNS data as a static tag on the scene. Often used in Gaussian Models (no movable parts but in need of data display) or static attributes of structured models (e.g., temperature of AC).
> - **Motion Mapping**: Data mapped to the movable parts of the model, and the parts move according to the data change.

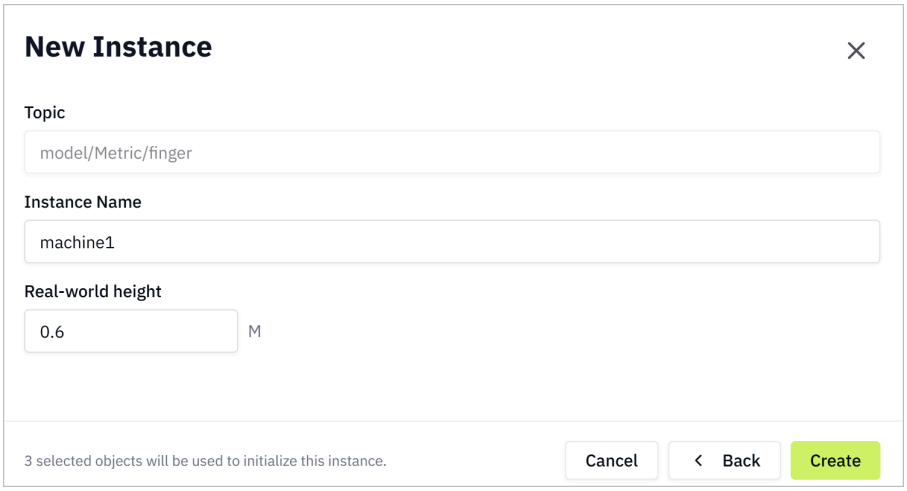
1. After uploading the model, click the model to start mapping it to UNS data.
2. Select model nodes to be mapped, and click **OK**.



3. Click **Create Instance**, select the UNS data model that controls the model animation, and then click **Next**.



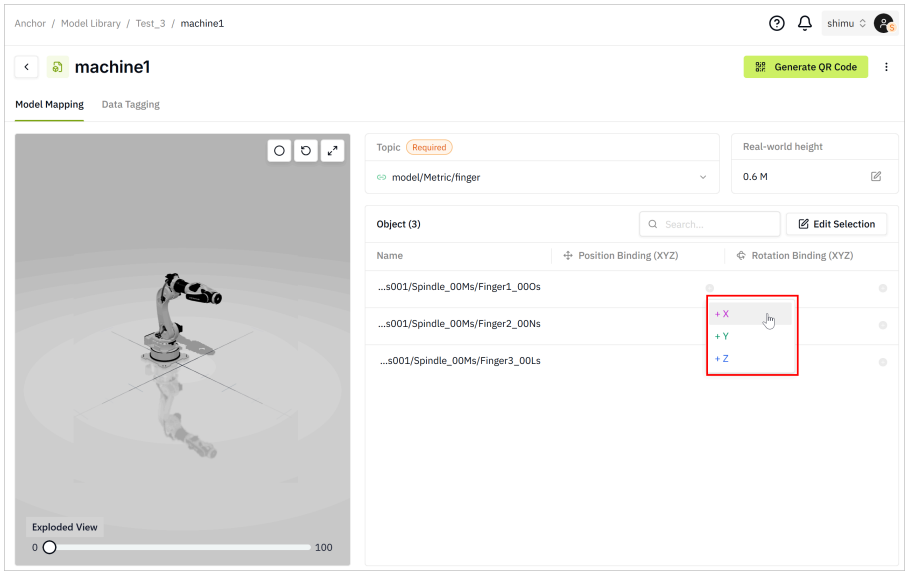
4. Enter the **Instance Name**, set the **Real-world Height** of the model, and click **Create**.



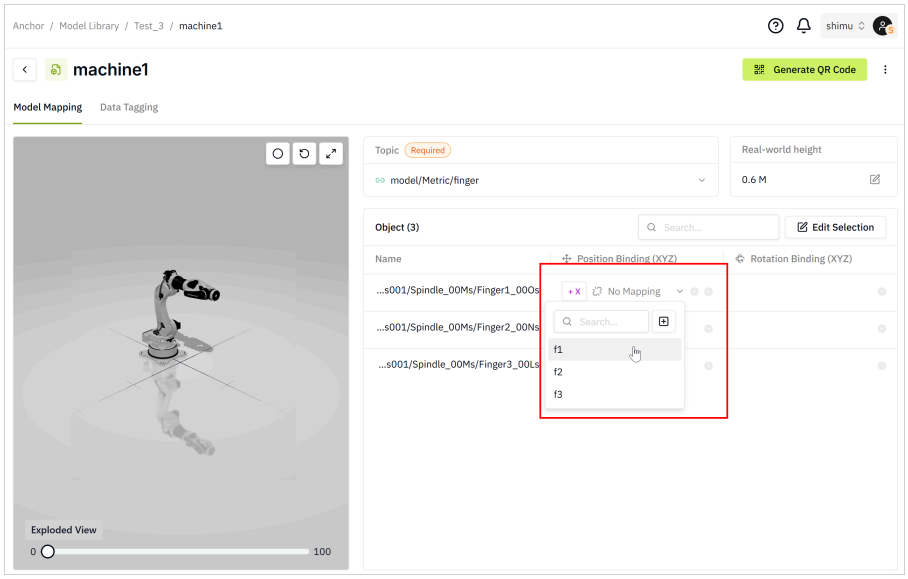
Motion Mapping

Bind the UNS data to movable parts of the model.

1. On the **Model Mapping** tab, click , and select the axis of the model position coordinatesto be mapped.



2. Map the data field with the selected axis.

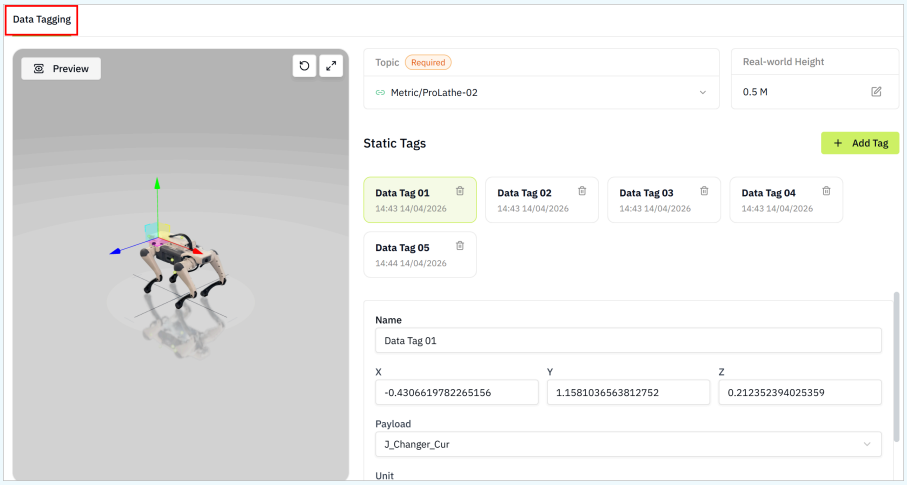


3. Do the same for **Rotation Binding** to determine how the model parts rotate according to the data change.

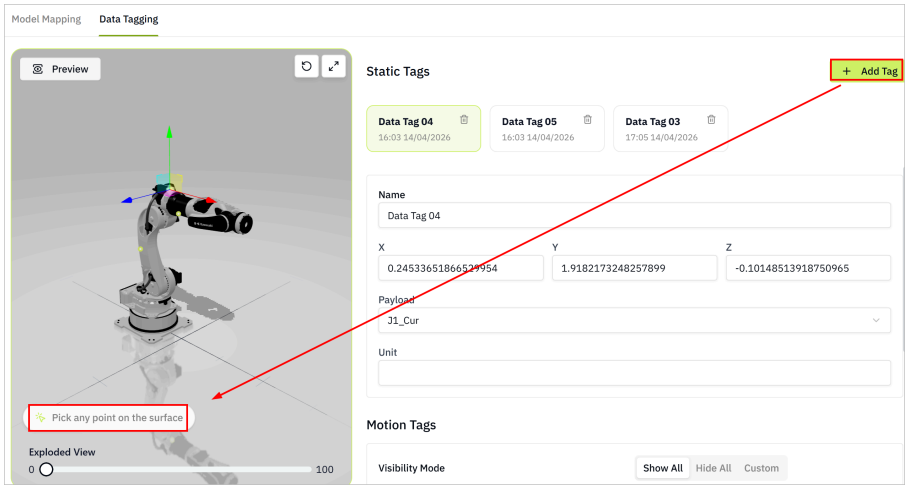
Static Mapping

Bind the UNS data with the model as a static tag.

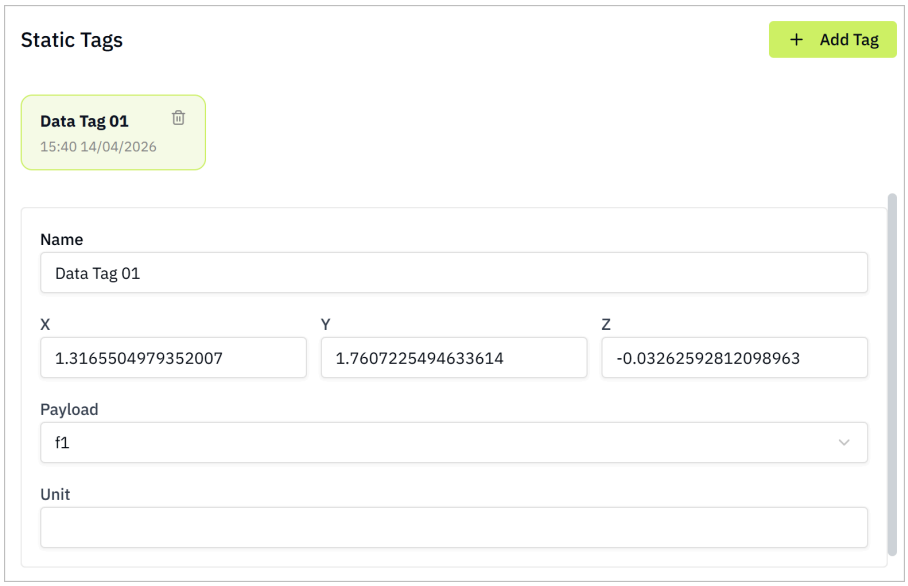
> **INFO**
>
> Gaussian models are only available for static mapping, as they have no movable parts.



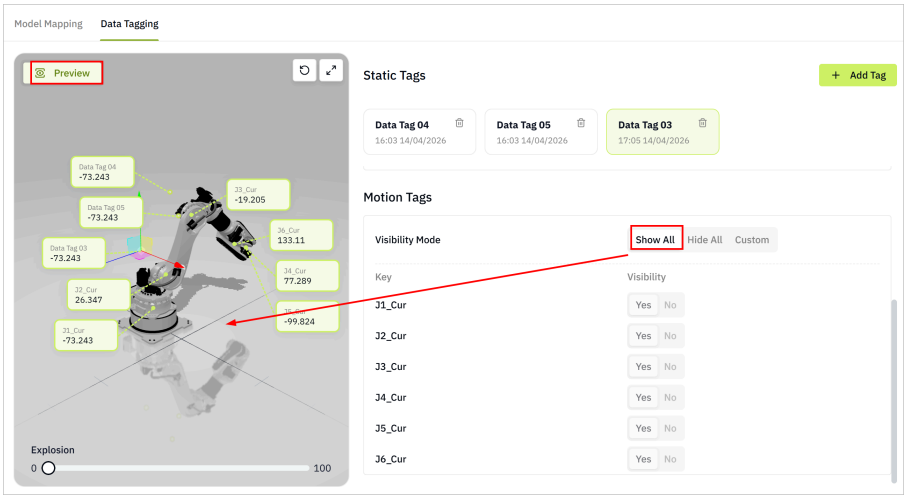
1. Click **Data Tagging**, and click **Add Tag**.



2. Click on the model to select the point the tag represents, and configure the tag attributes.

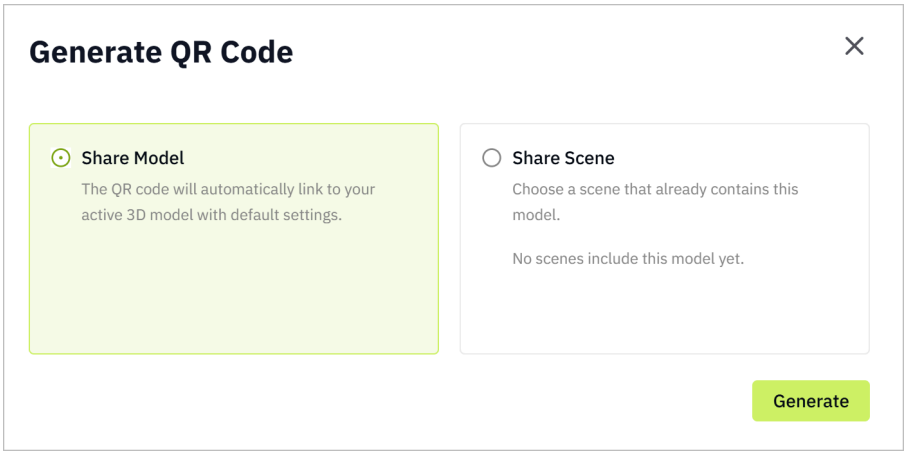


3. Set the motion tags visibility mode, and click **Preview**.



Viewing Digital Twin Models

1. After mapping UNS data to the model, click **Generate QR Code** at the upper-right corner.
2. Select **Share Model**, and click **Generate**.



3. Scan the QR code to view the digital twin model on your mobile device.



> **INFO**
>
> - When the model is added to a scene, share the scene to view the model with position, lighting and other settings.
> - You can also download the code, or copy the code link for later use.

Quick Operations during Mapping

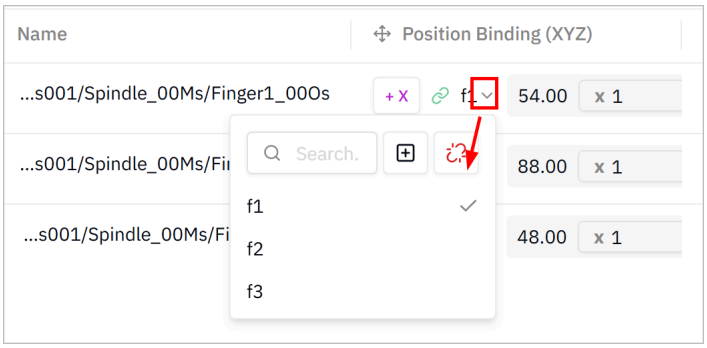
> **INFO**
>
> Node mappings are saved automatically.

On the Model Window

- Click **Replace Model** at the upper-right corner to replace the model while keeping the mapping settings after analysis.
- Drag the slider of **Exploded View** at the bottom of the model window to view the model parts.
- Click to display the model in its original state.
- Click to view the model in full screen.

On the Mapping Tab

- Click **Edit Nodes** at the upper-right corner to modify selected nodes.
- Click to disconnect a model node from the instance.
- Click to add payload to UNS topics for quick mapping with new data.

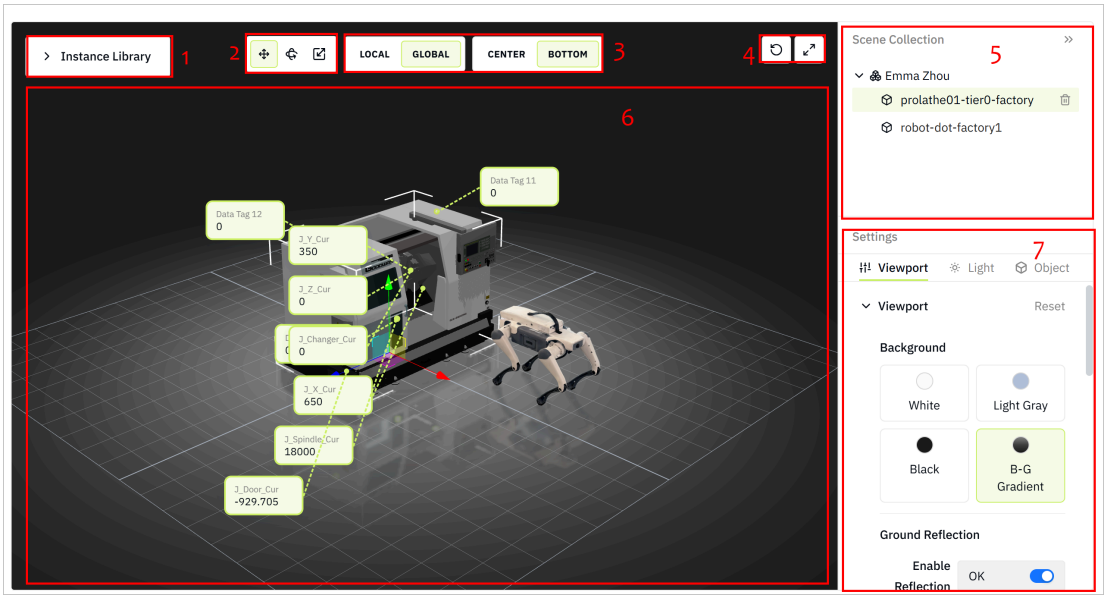


Build 3D Scenes

Tier0 enables you to build a scene using 3D models that have been mapped with real-time UNS data for live display.

Additional Parameter Description

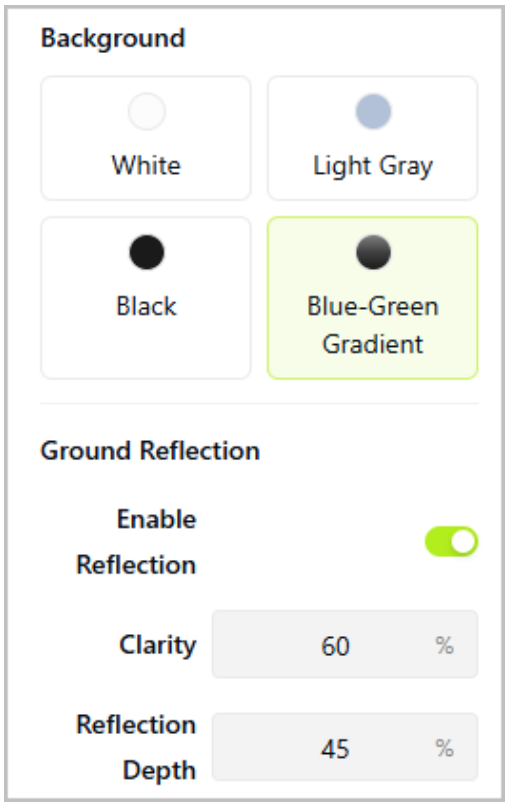
- Scene Description.



No.	Module	Item	Description
1	Instance Library		Lists all configured models from **Model Library**.

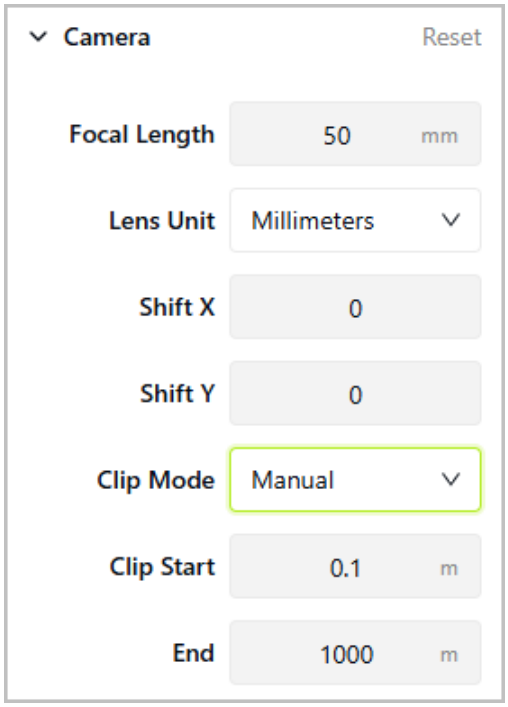
2	Model Position	/ /	Moves/rotates/scales the model along the X, Y and Z axes.
3	Axis	Local/Global	Decides whether the model coordinates is based on the model itself, or the entire screen.
		Center/Bottom	Sets the coordinates position to be at the center or bottom of the model.
4	General Operations	/	Restores the scene to the original viewport/Views the scene in full screen.
5	Scene Collection		Lists all models in the scene.
6	Canvas		Displays the scene canvas.
7	Settings	Viewport	Sets the scene viewport, including background, camera parameters and grid layout.
		Light	Sets the lighting conditions in the scene under different types of presets.
		Object	Displays the attributes of the selected model in the scene. You can edit its position by manually changing the values.

- **Viewport**



Parameter	Description
Background	Background style of the scene canvas.
Ground Reflection	Controls whether a reflection effect appears beneath the models, simulating a reflective ground surface.
Enable Reflection	Turns the ground reflection effect on or off.
Clarity	Adjusts how clear or sharp the reflection looks. Higher values produce a clearer reflection.
Reflection Depth	Adjusts the vertical range or depth of the reflection. A higher value usually makes the reflection extend farther downward.

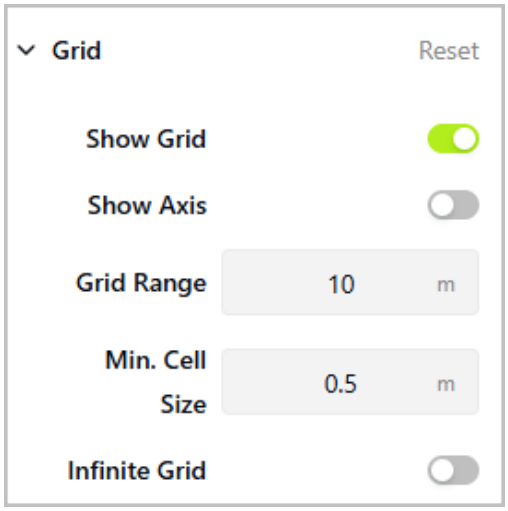
- **Camera**



Parameter	Description
Focal Length	Sets the camera focal length. A smaller value gives a wider view.
Lens Unit	Specifies the unit used for the lens setting. In this example, the focal length is measured in millimeters.
Shift X/Y	Moves the camera view horizontally/vertically without rotating the camera. Positive or negative values shift the framing left or right/up or down.

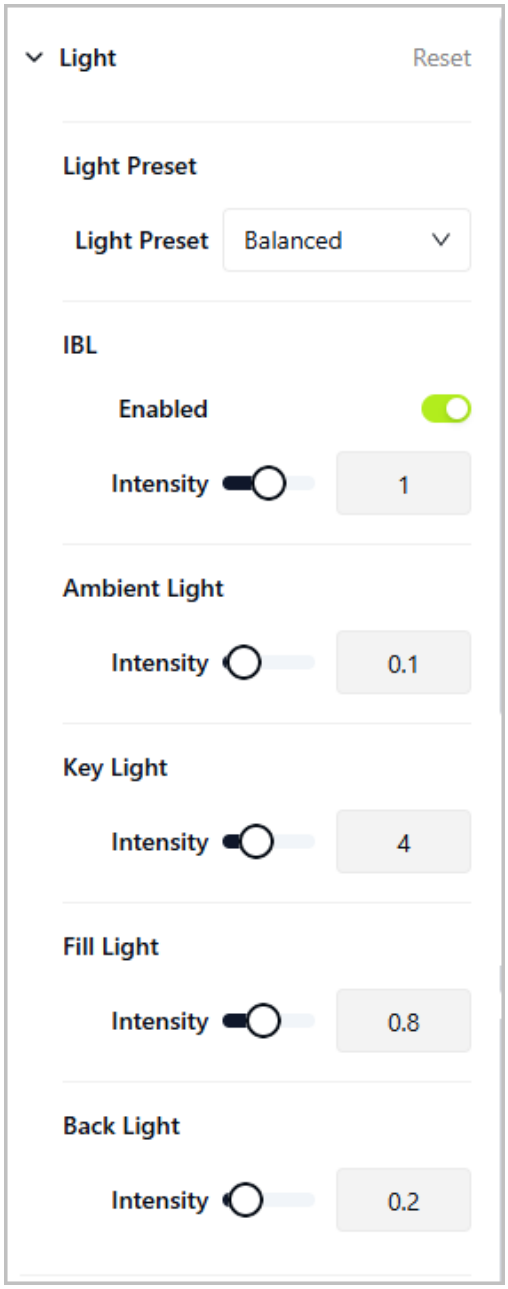
Parameter	Description
Clip Mode	Specifies how the camera clipping range is controlled. In **Auto** mode, the system automatically calculates the visible range based on the scene. In **Manual** mode, you can set the near and far clipping distances using **Clip Start** and **End**.
Clip Start	Defines the nearest distance from the camera at which objects start to be visible. Objects closer than this distance will be clipped.
End	Defines the farthest distance from the camera at which objects remain visible. Objects beyond this distance will be clipped.

- **Grid**



Parameter	Description
Show Grid	Whether the grid is visible in the scene.
Show Axis	Whether the coordinate axes are displayed in the scene.
Grid Range	Defines the overall visible size of the grid area.
Min. Cell Size	Defines the minimum size of each grid cell.
Infinite Grid	Whether the grid extends infinitely instead of being limited to a fixed area.

- **Light**

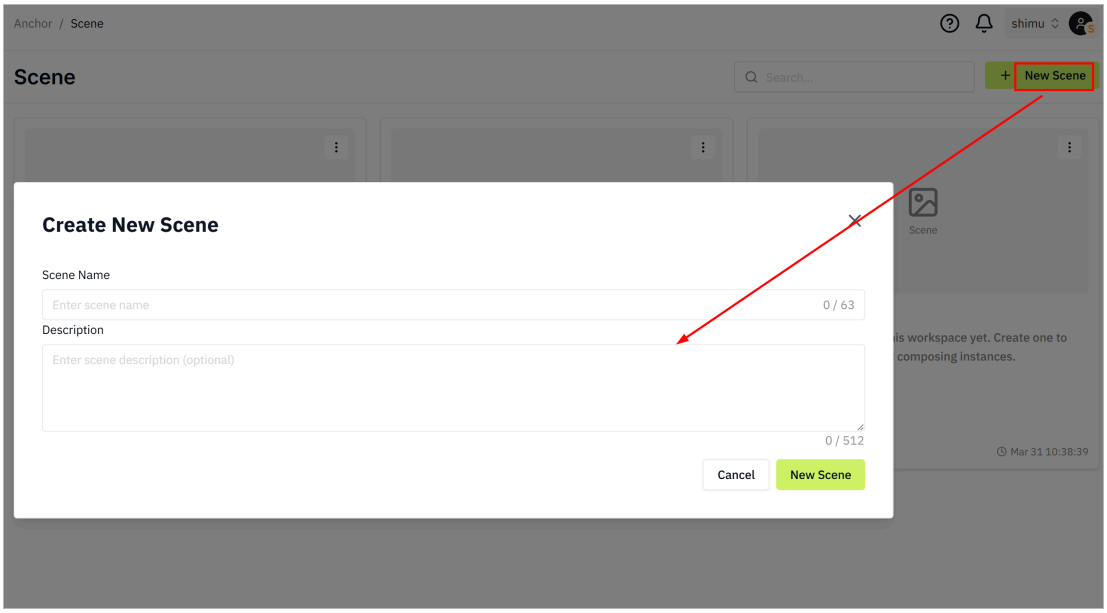


Parameter	Description
Light Preset	Selects a predefined lighting setup. Available options include **Balanced**, **Soft**, and **Contrast**.
IBL	Enables image-based lighting, which uses an environment image to provide realistic ambient illumination and reflections for the scene. The intensity value controls the strength of the IBL effect.
Ambient Light	Controls the intensity of ambient light, which provides overall non-directional illumination to the entire scene.
Key Light	Controls the intensity of the main light source, which is the primary light shaping the object and creating the main highlights and shadows.
Fill Light	Controls the intensity of the fill light, which softens shadows created by the key light and reduces contrast.

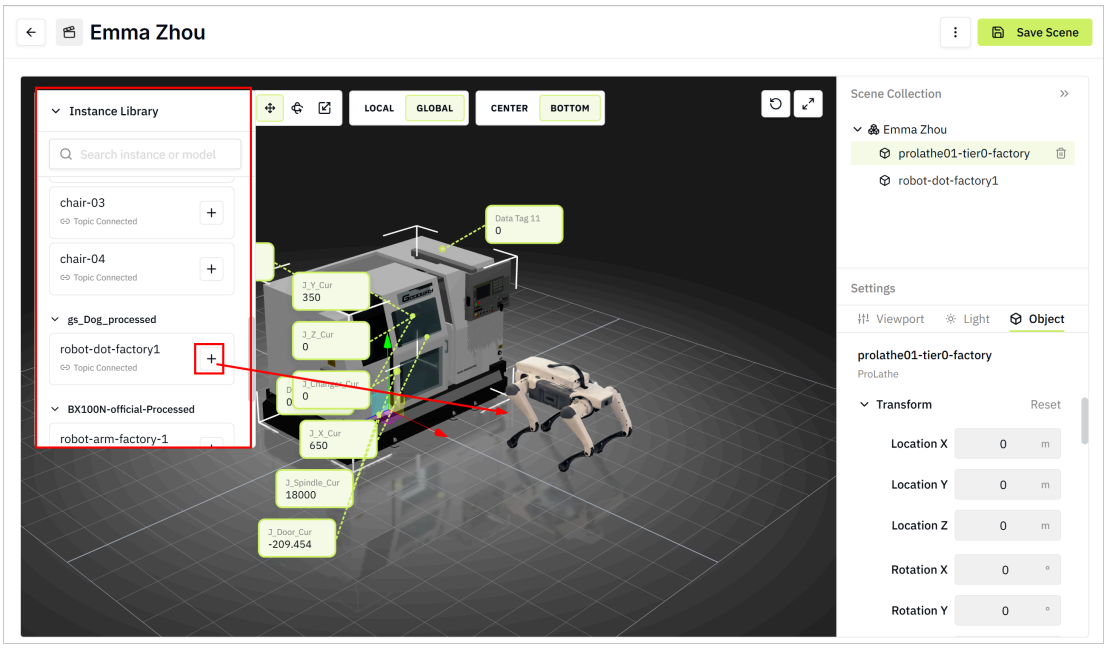
Parameter	Description
Back Light	Controls the intensity of the back light, which illuminates the object from behind to help separate it from the background.

How to Build a Digital Twin Scene

1. Log in to Tier0, select **Anchor** > **Scene**.
2. Click **New Scene** at the upper-right corner, enter the scene information, and click **Create**.



3. Open the created scene, select instances from **Instance Library** on the left.



4. Configure the models added to the scene.

    i. Set the model coordinates.

ii. Adjust the model position, rotation and scale based on the selected coordinates.
iii. On the right side, edit the viewport, camera and grid as needed.
iv. Edit lighting settings.
v. (optional) Edit the model position by setting the numbers under **Object** > **Transform**.

5. Click **Save Scene** at the upper-right corner.

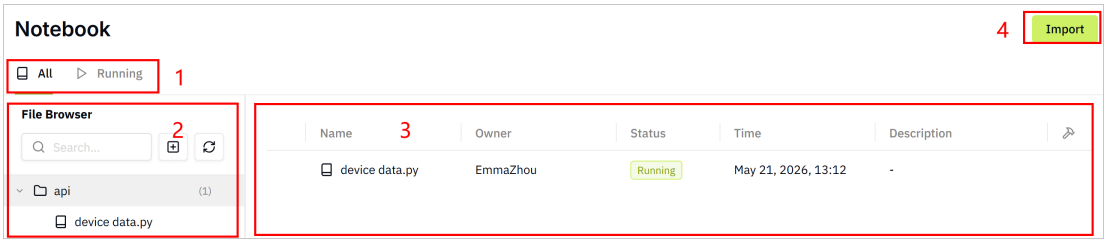
Analyze Data

Analyze Data

Tier0 adopts Marimo Notebook for advanced data analysis with Python.

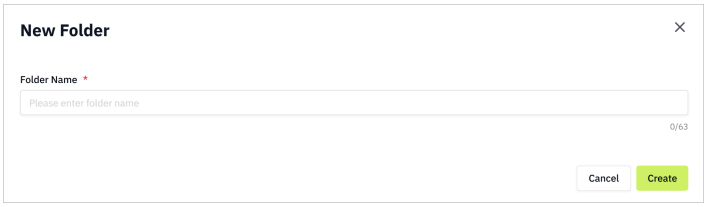
Creating Notebooks

1. Log in to Tier0, and then select **Notebook**.

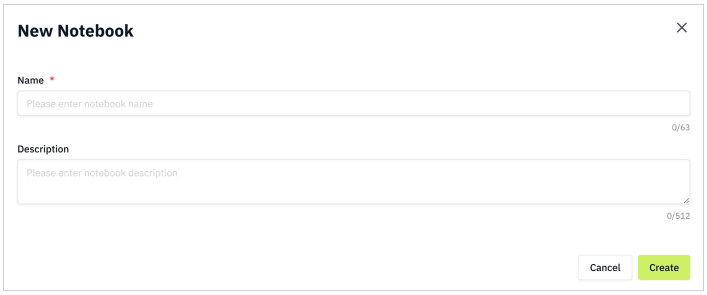


No.	Item	Description
1	Category	Displays notebooks by their status.
2	Notebook Treemap	Lists notebooks in a folder-file hierarchical manner, you can add notebooks, refresh lists and search for notebooks.
3	Notebook List	Displays all notebooks of the selected folder in details.
4	Import	Import notebooks.

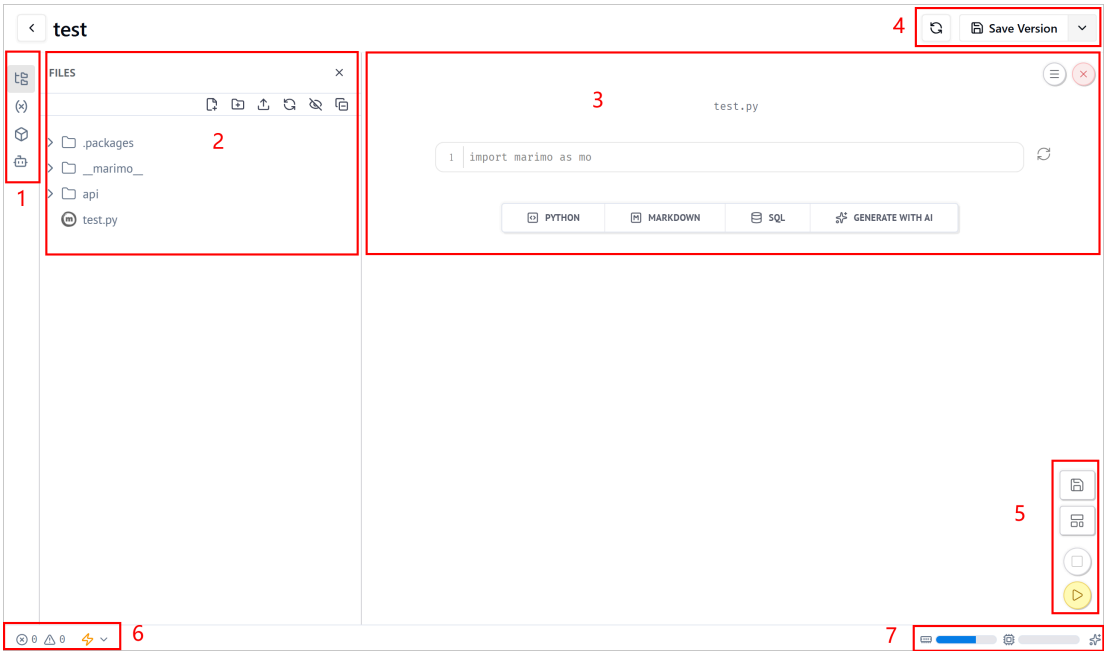
2. Select > **New Folder**.



3. Enter a folder name and click **Create**.
4. Click the created folder, and then select > **New Notebook**.



5. Enter the notebook information and click **Create**.
6. Click the created notebook on the right side, and start editing.

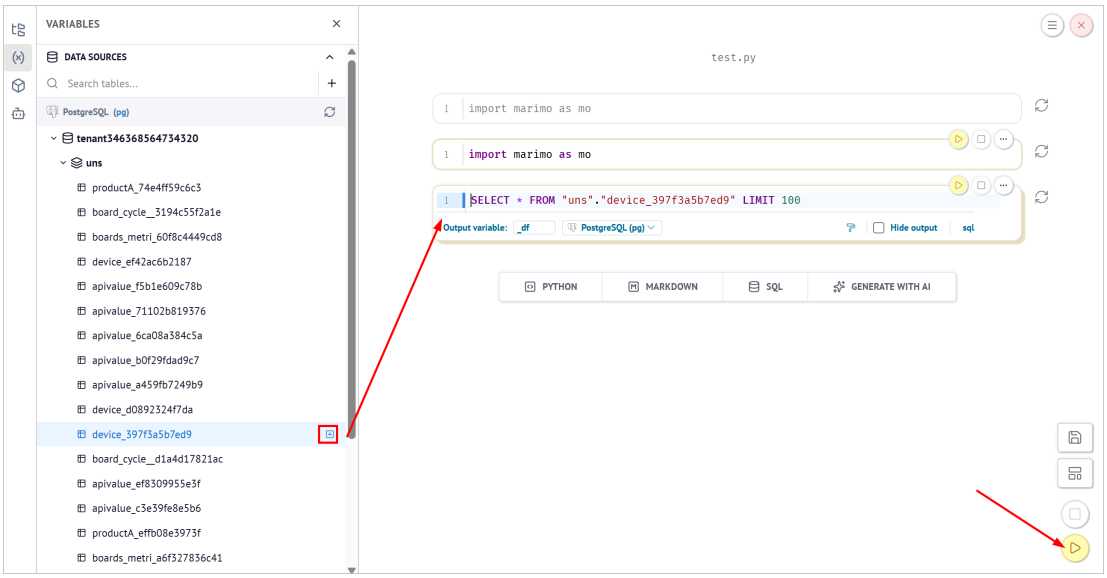


No.	Item	Description
1	Modules	Displays notebook modules, including data sources and variables, dependency packages and AI bot.
2	Files	Displays notebook hierarchy.
3	Canvas	Displays cells and the corresponding results. You can select to create different types of cells, shutdown the notebook and change its settings.
4	Tool Bar	Refresh cells and save the notebook to different versions.
5	Notebook Operations	Save the notebook. Switch the current viewer style between cells & result, and pure result. Stop/run all cells.
6	Developer Mode	Enter developer panel, and set runtime activity.
7	Status	Displays the memory, CPU and AI status.

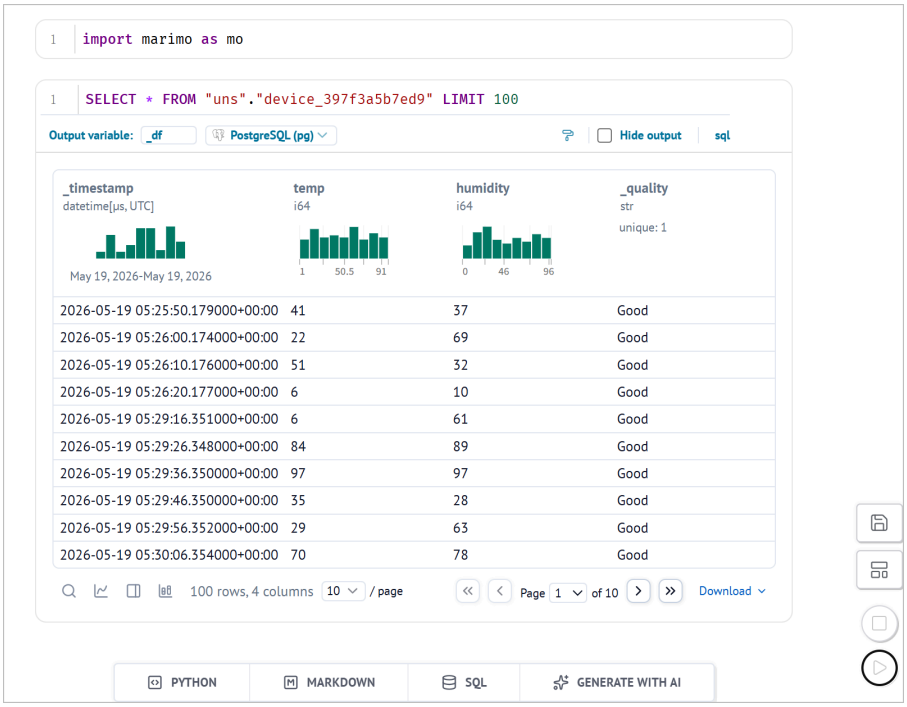
Using Notebook to Run SQL Queries

> **KNOW BEFORE YOU START**
>
> Data stored in database from UNS acts as an existing data source in notebook.

1. Log in to Tier0, and then select **Notebook**.
2. Create a notebook and access it, Click **VARIABLES** on the left side.
3. Find the table you need on the left, click  next to the table name to generate SQL statements automatically.

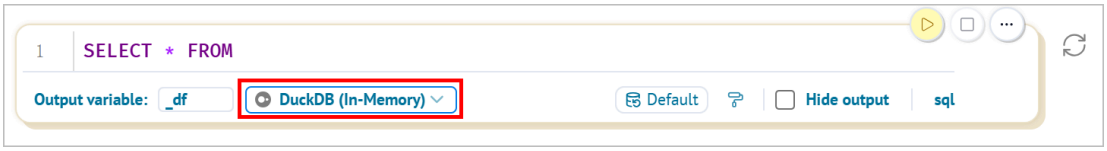


4. Click at the lower-right corner to run all cells and view the query result.



> **TIP**
>
> - You can click + under **DATA SOURCES** on the left to add sources as needed.
> - Certain packages might be missing while working with notebook. In this case, click **Manage packages** on the left to install.

5. (optional) Click **SQL** to add a new cell, and then select a connected database to run SQL statements.



Using Notebook to Analyze Data

Example Background

- Data source is temperature and humidity data of a device collected from a factory and sent to Tier0 UNS.
- Analysis on the data includes set risk thresholds, recognize risks according to combined temperature and humidity data, and generate tables, charts for easy view.

Steps

1. Log in to Tier0, and then select **Notebook**.
2. Create a notebook, inside it add the following cells.

- Cell 1: Import libraries.

```
import marimo as mo
import pandas as pd
import altair as alt
```

- Cell 2: Query data from the database

```
SELECT * FROM "uns"."device_15d9d69fd5eb" LIMIT 100
```

- Cell 3: Clean data

```
# Convert SQL result to pandas DataFrame
if hasattr(device_df, "to_pandas"):
    df = device_df.to_pandas()
elif hasattr(device_df, "to_dicts"):
    df = pd.DataFrame(device_df.to_dicts())
else:
    df = pd.DataFrame(device_df)
```

```
# Clean and normalize data
df["_timestamp"] = pd.to_datetime(df["_timestamp"])
df = df.sort_values("_timestamp").reset_index(drop=True)

df["temp"] = pd.to_numeric(df["temp"], errors="coerce")
df["humidity"] = pd.to_numeric(df["humidity"], errors="coerce")

df = df.dropna(subset=["temp", "humidity"])

df.head()
```

- Cell 4: Set risk thresholds and generate risk status fields.

```
temp_warning = mo.ui.slider(
    start=0,
    stop=100,
    value=70,
    label="Temperature warning threshold",
)

temp_critical = mo.ui.slider(
    start=0,
    stop=100,
    value=85,
    label="Temperature critical threshold",
)

humidity_warning = mo.ui.slider(
    start=0,
    stop=100,
    value=70,
    label="Humidity warning threshold",
)

humidity_critical = mo.ui.slider(
    start=0,
    stop=100,
    value=85,
    label="Humidity critical threshold",
)

mo.vstack([
    mo.md("## Threshold Settings"),
    mo.hstack([temp_warning, temp_critical]),
    mo.hstack([humidity_warning, humidity_critical]),
])
```

- Cell 5: Generates a table with risk status and anomaly status based on the thresholds set in Cell 4.

```
analyzed_df = df.copy()

def classify_risk(row):
    temp = row["temp"]
    humidity = row["humidity"]

    temp_high = temp >= temp_warning.value
    temp_critical_high = temp >= temp_critical.value

    humidity_high = humidity >= humidity_warning.value
    humidity_critical_high = humidity >= humidity_critical.value
```

```
    if temp_critical_high and humidity_critical_high:
        return "Critical: High temp + high humidity"
    elif temp_critical_high:
        return "Critical: High temperature"
    elif humidity_critical_high:
        return "Critical: High humidity"
    elif temp_high and humidity_high:
        return "Warning: Combined risk"
    elif temp_high:
        return "Warning: Temperature risk"
    elif humidity_high:
        return "Warning: Humidity risk"
    else:
        return "Normal"

analyzed_df["risk_status"] = analyzed_df.apply(classify_risk, axis=1)

analyzed_df["is_anomaly"] = analyzed_df["risk_status"] != "Normal"

analyzed_df.head()
```

- Cell 6: Generates anomaly summary statistics

```
total_count = len(analyzed_df)
anomaly_count = analyzed_df["is_anomaly"].sum()
normal_count = total_count - anomaly_count
anomaly_rate = anomaly_count / total_count * 100 if total_count > 0 else 0

critical_count = analyzed_df["risk_status"].str.startswith("Critical").sum()
warning_count = analyzed_df["risk_status"].str.startswith("Warning").sum()

mo.vstack([
    mo.md("## Joint Anomaly Detection Summary"),
    mo.hstack([
        mo.stat(label="Total Records", value=f"{total_count}"),
        mo.stat(label="Normal Records", value=f"{normal_count}"),
        mo.stat(label="Warning Records", value=f"{warning_count}"),
        mo.stat(label="Critical Records", value=f"{critical_count}"),
        mo.stat(label="Anomaly Rate", value=f"{anomaly_rate:.1f}%"),
    ])
])
```

- Cell 7: Generates a bar chart to show the distribution of different risk statuses.

```
risk_summary = (
    analyzed_df
    .groupby("risk_status")
    .size()
    .reset_index(name="count")
    .sort_values("count", ascending=False)
)

risk_bar_chart = alt.Chart(risk_summary).mark_bar().encode(
    x=alt.X("count:Q", title="Record Count"),
    y=alt.Y("risk_status:N", sort="-x", title="Risk Status"),
    tooltip=["risk_status", "count"]
).properties(
    title="Risk Status Distribution",
    width=700,
```

```
    height=300
)

risk_bar_chart
```

- Cell 8: Generates a scatter chart to show the relationship between temperature and humidity, with points colored by risk status.

```
scatter_chart = alt.Chart(analyzed_df).mark_circle(size=80, opacity=0.75).encode(
    x=alt.X("temp:Q", title="Temperature"),
    y=alt.Y("humidity:Q", title="Humidity"),
    color=alt.Color("risk_status:N", title="Risk Status"),
    tooltip=[
        alt.Tooltip("_timestamp:T", title="Timestamp"),
        alt.Tooltip("temp:Q", title="Temperature"),
        alt.Tooltip("humidity:Q", title="Humidity"),
        alt.Tooltip("risk_status:N", title="Risk Status"),
    ],
).properties(
    title="Temperature and Humidity Joint Anomaly Detection",
    width=700,
    height=420,
)

temp_warning_line = alt.Chart(pd.DataFrame({"x": [temp_warning.value]})).mark_rule(
    strokeDash=[6, 4]
).encode(
    x="x:Q"
)

humidity_warning_line = alt.Chart(pd.DataFrame({"y": [humidity_warning.value]})).mark_rule(
    strokeDash=[6, 4]
).encode(
    y="y:Q"
)

scatter_chart + temp_warning_line + humidity_warning_line
```

- Cell 9: Generates a table to show the details of anomaly records.

```
anomaly_df = analyzed_df[analyzed_df["is_anomaly"]].copy()

anomaly_df = anomaly_df[
    ["_timestamp", "temp", "humidity", "risk_status"]
].sort_values("_timestamp")

mo.vstack([
    mo.md("## Anomaly Details"),
    mo.md(f"Detected **{len(anomaly_df)}** abnormal records based on the current thresholds."),
    mo.ui.table(anomaly_df, page_size=10),
])
```

- Cell 10: Generates a summary of the analysis conclusion.

```
most_common_risk = (
    risk_summary.iloc[0]["risk_status"]
    if len(risk_summary) > 0
```

```
    else "N/A"
)

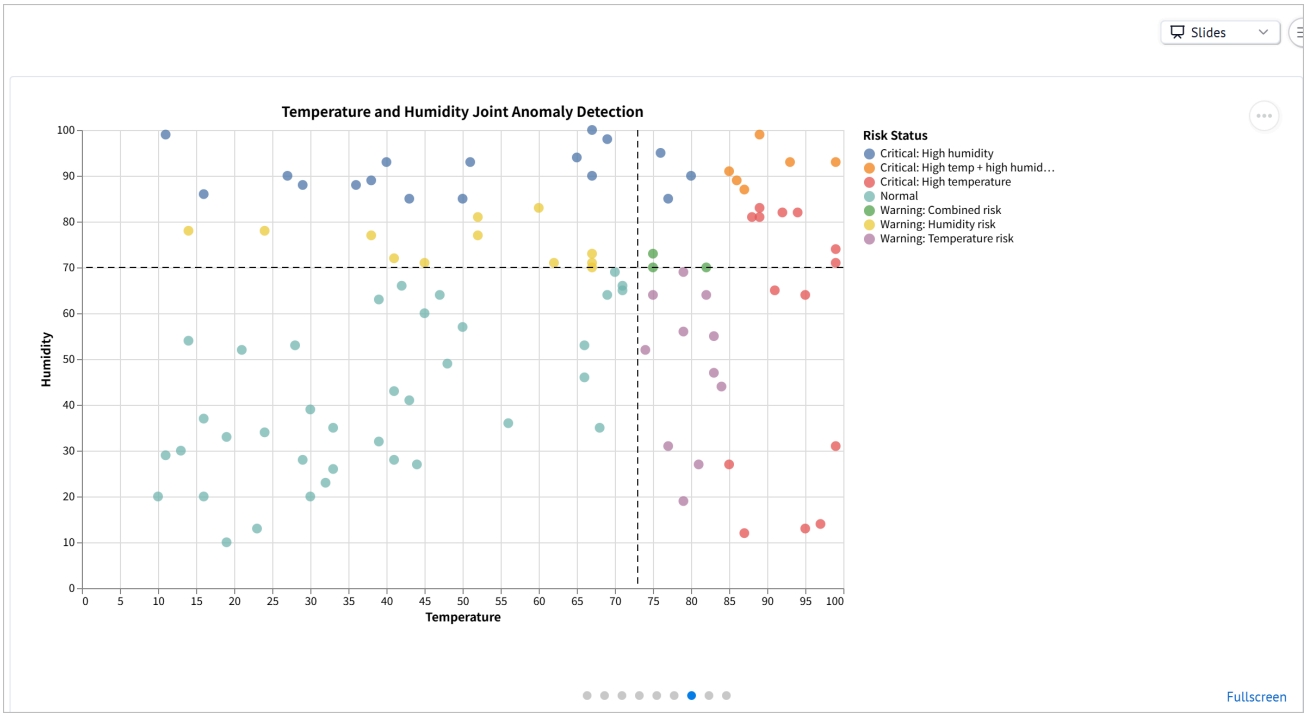
mo.md(f"""
## Analysis Conclusion

Based on the current threshold settings:

- Total records analyzed: **{total_count}**
- Abnormal records detected: **{anomaly_count}**
- Overall anomaly rate: **{anomaly_rate:.1f}%**
- Warning records: **{warning_count}**
- Critical records: **{critical_count}**
- Most common status: **{most_common_risk}**

This analysis uses both temperature and humidity to identify equipment environment risks.
Compared with single-threshold monitoring, joint anomaly detection can better identify combined environmental risks,
such as high temperature and high humidity occurring at the same time.
""")
```

3. Click at the lower-right corner to run all cells and view the results.



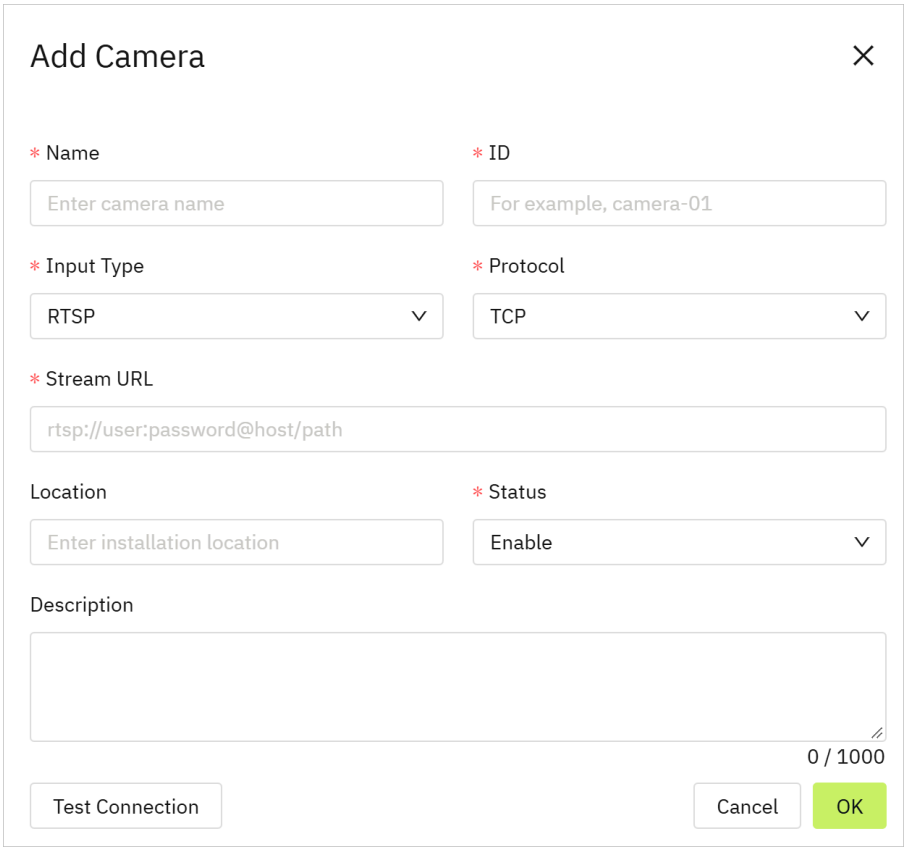
> **TIP**
>
> Change the view style at the upper-right corner among **Vertical**, **Grid** and **Slides**.

Visual Analytics

You can connect camera feeds to Tier0 and apply visual analytics to your live camera images.

Connecting Cameras

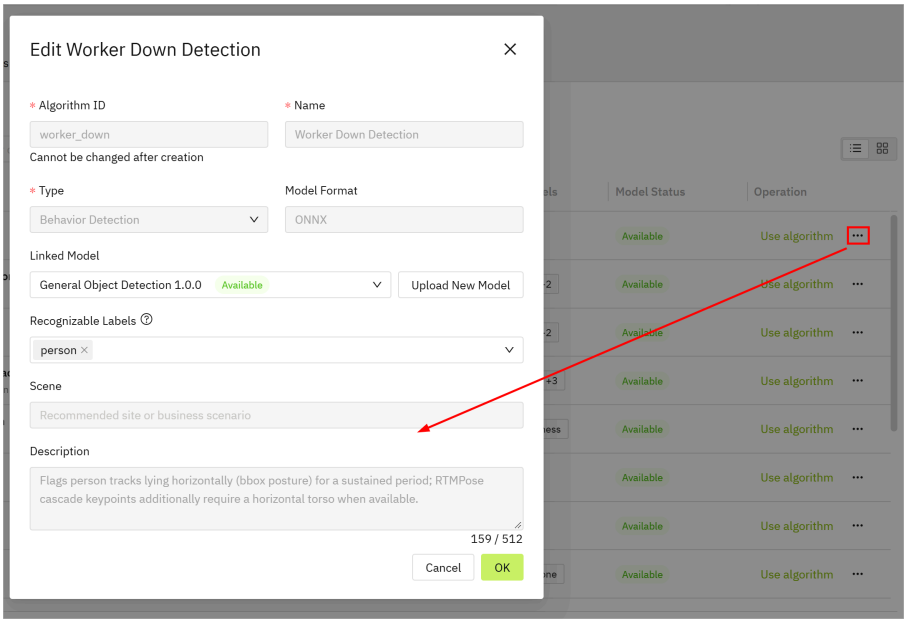
1. In Tier0 Enterprise, go to **Analysis** > **Vision**.
2. Under the **Camera** tab, click **Add Camera**.



3. Enter the camera information and stream URL, test the connection, and then click **OK**.

Editing Algorithms

1. Under the **Algorithm** tab, check the algorithm list and find one that fits your needs.
2. Click **Details** to view the algorithm details, click **Edit** when you need to update the model or add recognizable items.



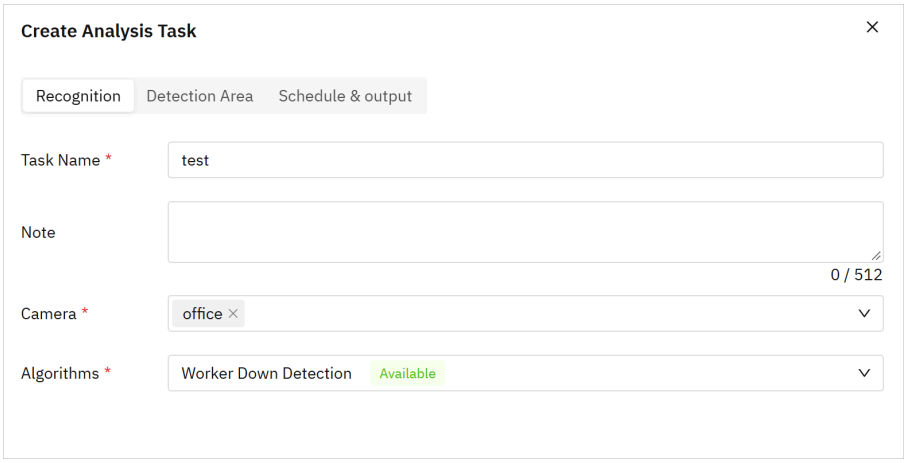
3. Click **OK**.

Analyzing Camera Feeds

1. Under the **Analysis Tasks** tab, click **Create Task**.

> **TIP**
>
> You can also click **Use Algorithm** under the **Algorithm** tab to create a task based on the selected algorithm.

2. Under **Recognition** tab, enter the task name, select the camera and algorithm.

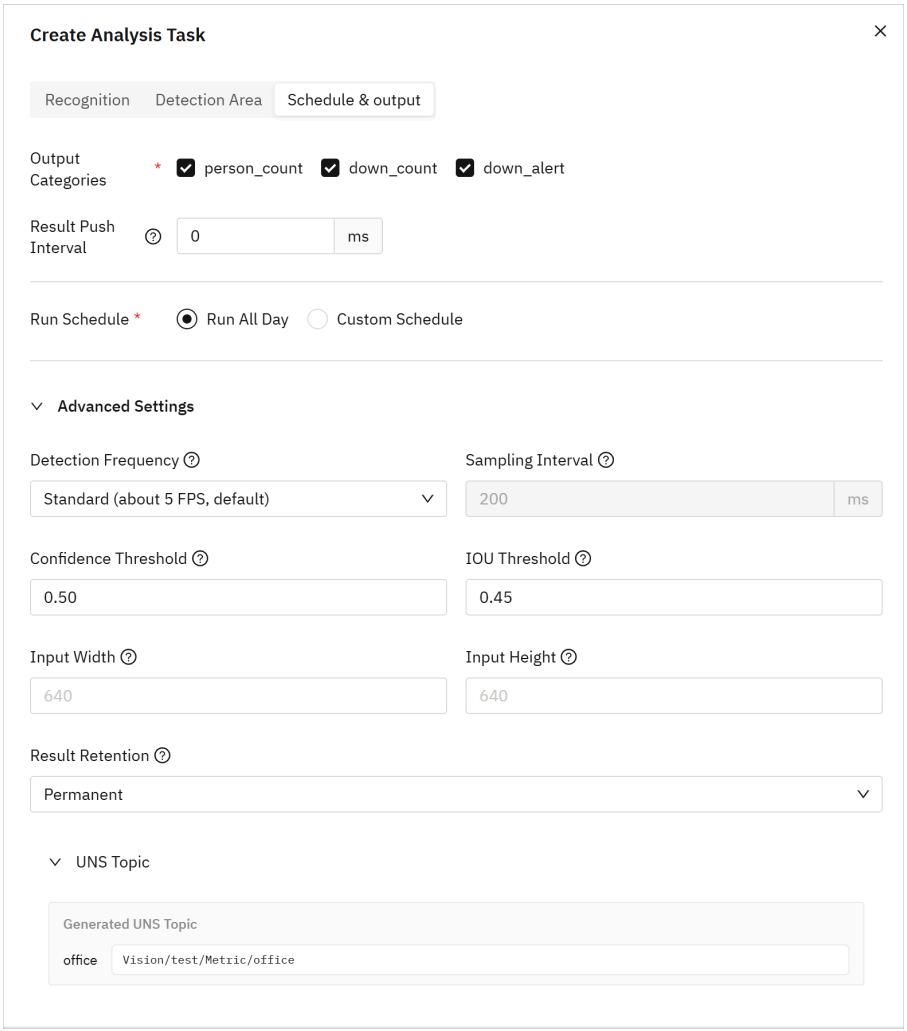


3. Switch to the **Detection Area** tab, click the plus icon at the lower-right corner on the image, click on the image to draw a detection area.
4. Click **Finish** to finish the area drawing.



5. Under the **Schedule & output** tab, configure the analysis running schedule and output settings.

> **TIP**
>
> Description of each parameter will be displayed when you hover over it.



6. Click **Create** and the result can be viewed under the **Analysis Results** tab.

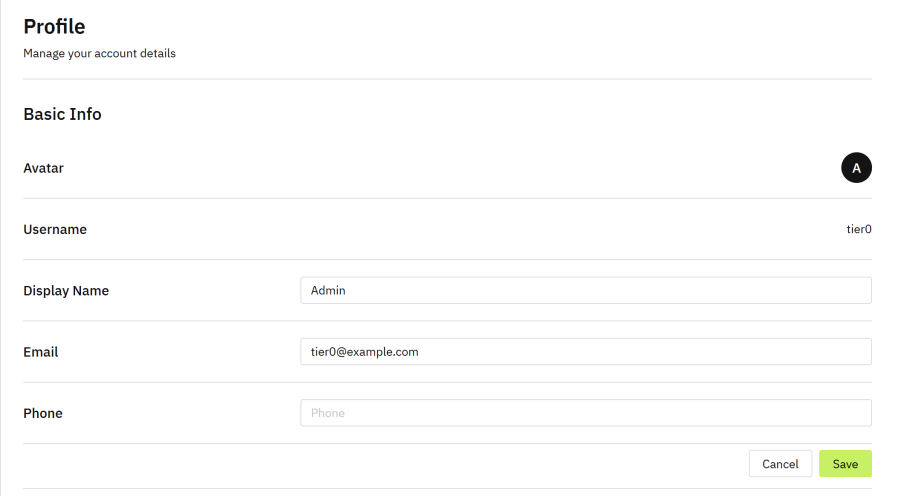
System Settings

User

In this section, you can change the user profile, system language and reset the password.

Changing User Profile

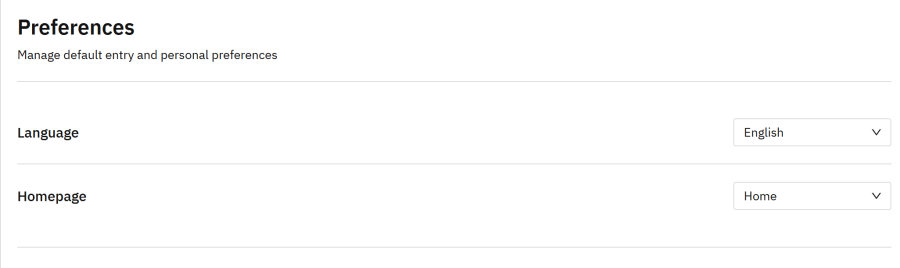
1. In Tier0, click the user icon at the upper-right corner, and then click **Profile**.
2. Click **Edit** to change the user profile, including the display name, email address and phone number.



3. Click**Save**.

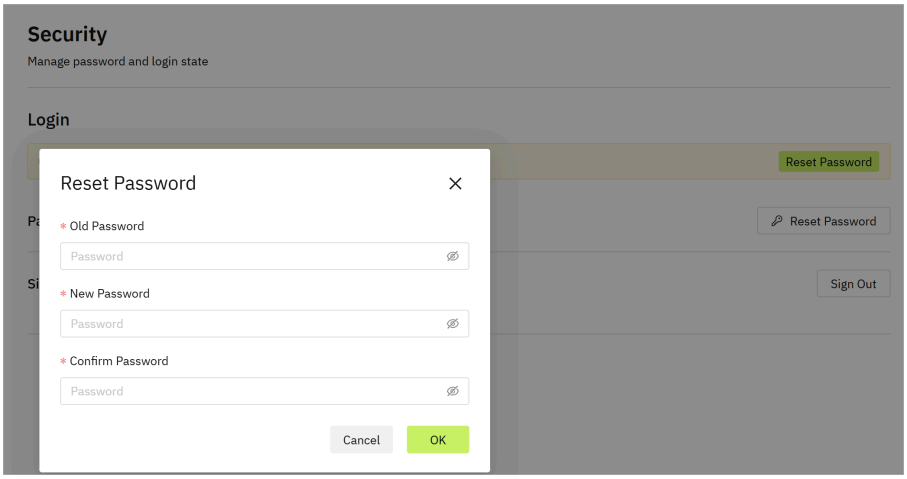
Changing Preferences

1. In Tier0, click the user icon at the upper-right corner, and then click **Preferences**.
2. Change the system language between English and Chinese.
3. Change the homepage of Tier0 as needed.



Changing Password

1. In Tier0, click the user icon at the upper-right corner, and then click **Security**.
2. Click **Reset Password**, enter corresponding passwords and click **OK**.



3. (optional) Click **Sign Out** to directly sign out.

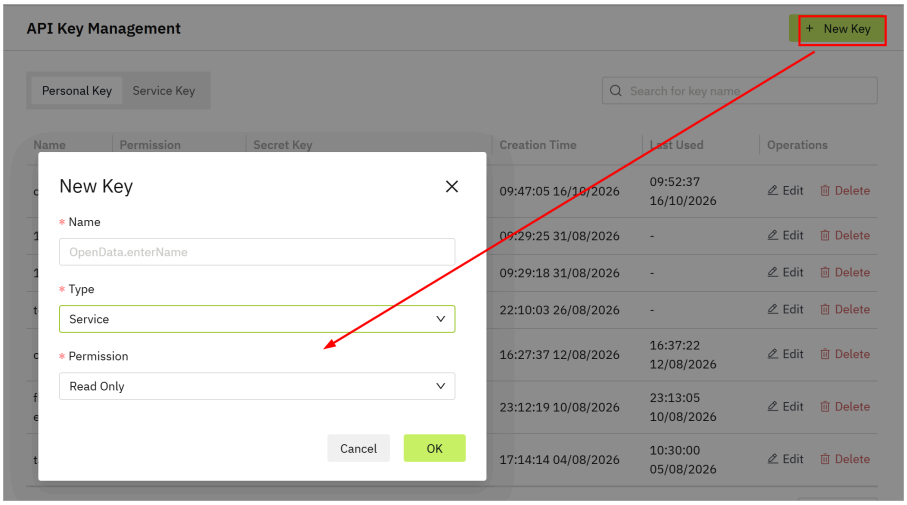
Service

Setting API Key

> **INFO**
>
> API key is necessary for API and CLI access.

1. In Tier0, click the user icon at the upper-right corner, and then click **API Key**.
2. Click **New Key**, and set the key type and permisson.

- **Service Key**: Created by admin, all permissions are available.
- **Personal Key**: Created by operators, and only **Read-only** permission.



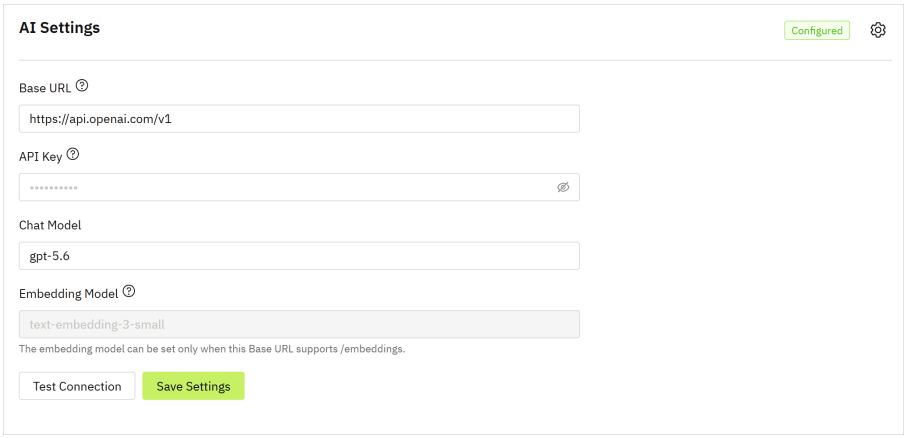
3. Click **OK**.

> **TIP**
>
> Check the guide on the bottom to understand the way of using API and CLI to access the platform with the help of the API key.

Setting AI API Key

> **CAUTION**
>
> This is necessary for using the UNS agent.

1. In Tier0, click the user icon at the upper-right corner, and then click **AI Settings**.
2. Enter the necessary information of your AI model.



> **IF YOU NEED TO SWITCH PROVIDER**
>
> Click ⚙ at the upper-right corner to change the model provider.

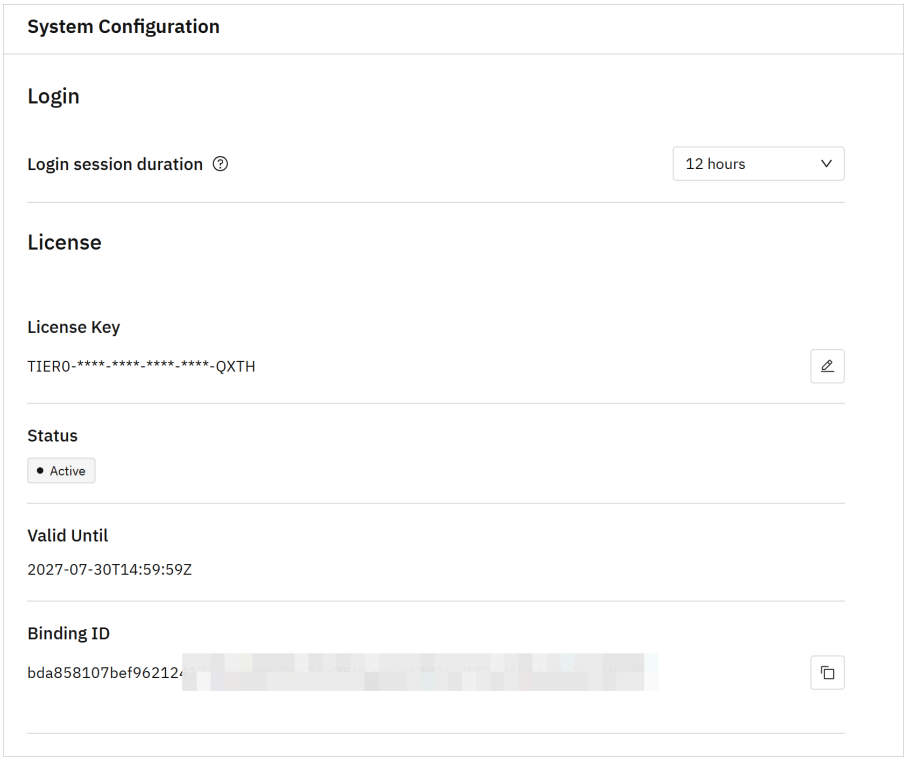
3. Click **Save Settings**.

Platform

System Configuration

You can complete the following tasks on this page:

- **Login**: Change the time period where the account stays signed in.
- **License**
  - Change the Tier0 license type by editing the license key you get from the vendor.
  - View license status, expiration date and binding ID.

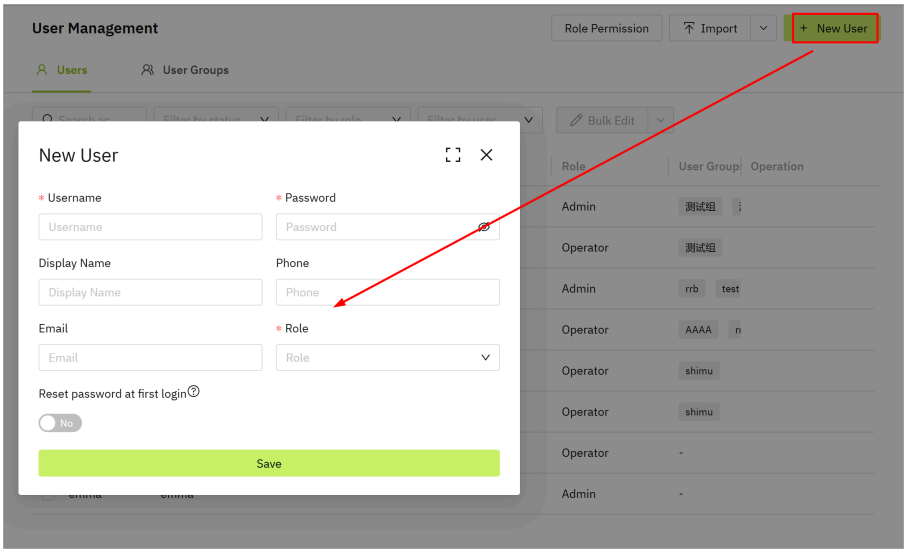


Managing Users

In Tier0, click the user icon at the upper-right corner, and then click **User Management**.

Adding User

1. Click **New User** at the upper-right corner on the **Users** tab.



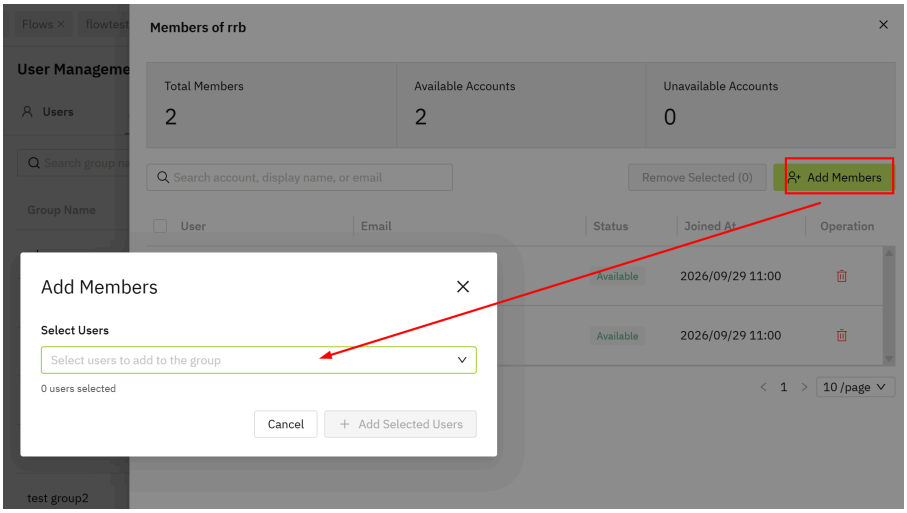
2. Enter the information and select a role for the user, and then click **Save**.

- You can disable, edit, delete user and reset password of users.
- The default user cannot be disabled or deleted.

Adding User Group

1. Click **New User Group** at the upper-right corner on the **User Group** tab.

2. Enter the information, and then click **Save**.
3. Click the group, and then click **Add Members** to add users to the group.



4. Click **Add Selected Users**.

Adding OAuth Client

This function is unavailable presently.

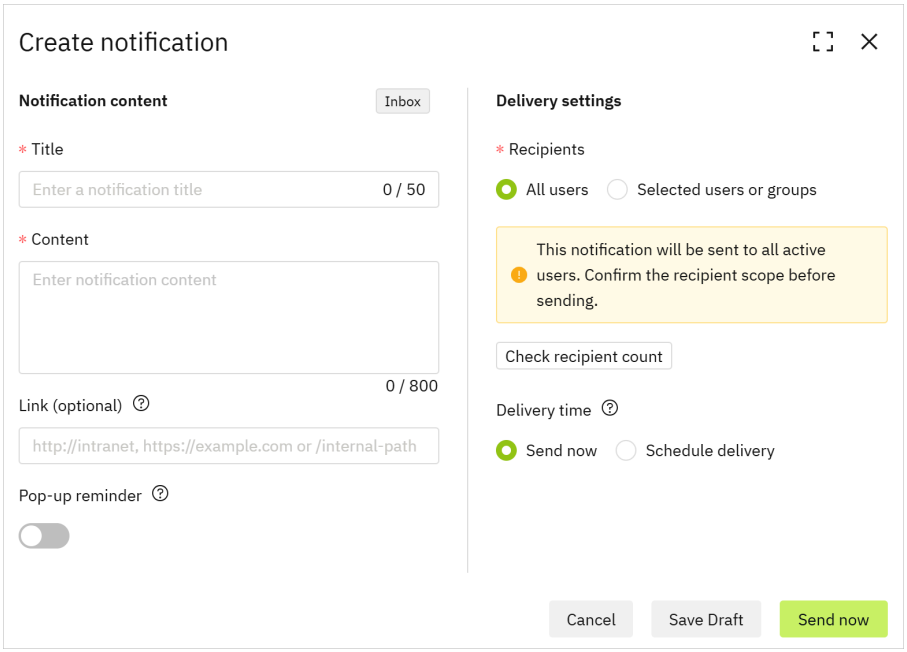
Viewing Logs

- Check detailed operation logs in **Audit Log**.
- View login logs in **Login Log**.

Setting Notifications

You can create notifications and send them to different recipients.

1. In Tier0, click the user icon at the upper-right corner, and then click **System Message Management**.
2. Click **Create notification**, enter the message title and content, and set the delivery information.



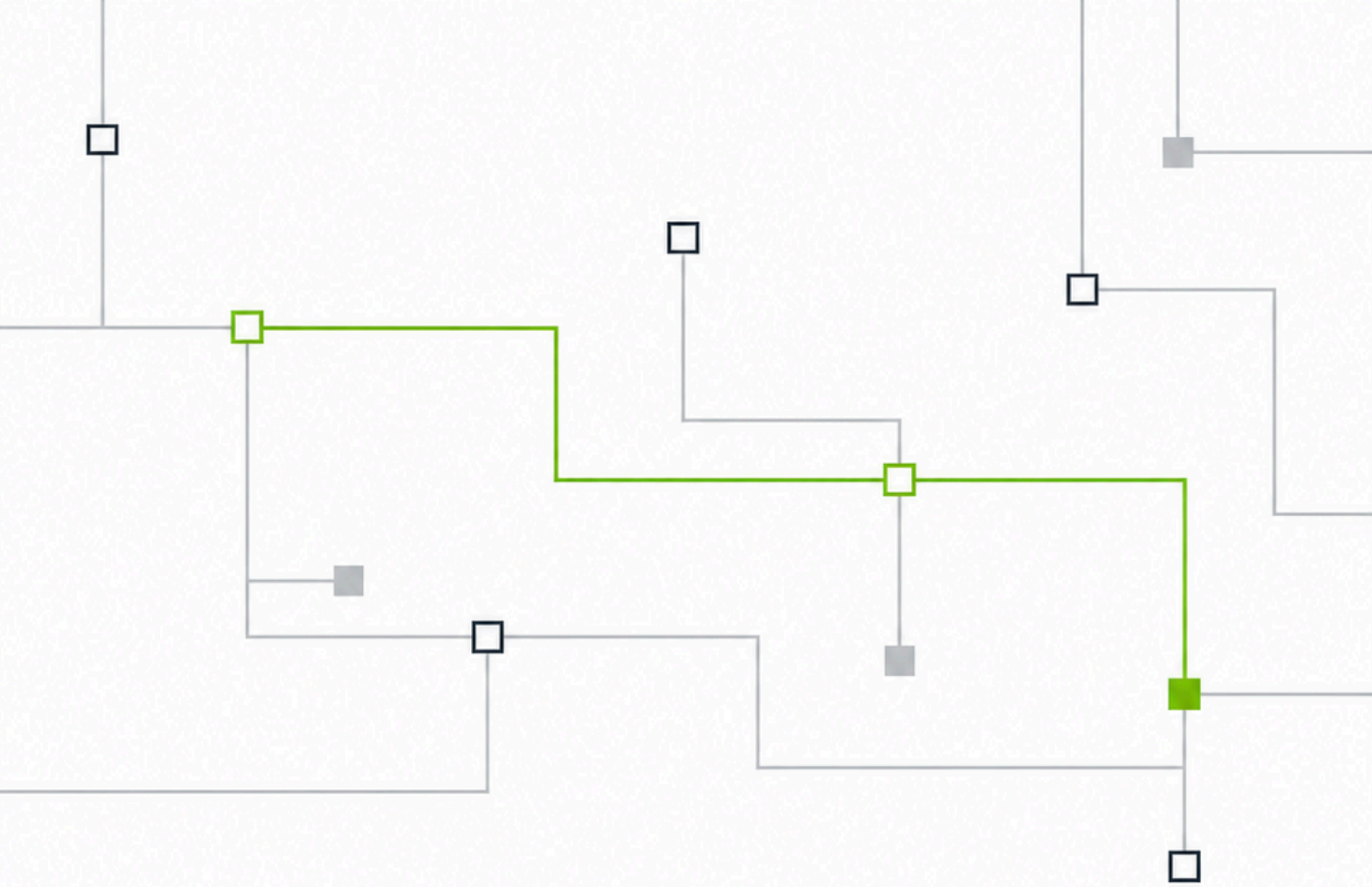
3. Send the notification.

Glossary

Key Terms and Definitions

Term	Definition
Tier0	An open-source industrial data integration platform built on the Unified Namespace (UNS) methodology, designed for managing, processing, and optimizing industrial data.
UNS (Unified Namespace)	A data management method that centralizes data from various devices and systems, providing a structured and unified framework to prevent data silos.
Node-RED	An open-source automation tool that enables users to wire together devices, APIs, and online services using a flow-based programming approach.
MQTT	A lightweight messaging protocol for small sensors and mobile devices optimized for high-latency or unreliable networks, commonly used for IoT communication.
JSON	JavaScript Object Notation, a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.
TimescaleDB	An open-source time-series database built on PostgreSQL, designed for handling large-scale time-series data, especially useful for monitoring systems and IoT applications.
PostgreSQL	An advanced open-source relational database management system that uses SQL for querying and is designed for high performance and reliability.
API (Application Programming Interface)	A set of protocols and tools that allows different software applications to communicate and interact with each other.

Term	Definition
Topic	A unique identifier for data in the Tier0 system that helps classify and organize data from various sources for easy access and management.
REST API	A web service interface that allows applications to communicate over HTTP, using standard methods like GET, POST, PUT, and DELETE to access resources.
Time-Series Data	Data that is collected and recorded at regular intervals over time, such as equipment run time, temperature, and other performance metrics.
JSON Payload	Data transferred in the form of a JSON object that contains relevant information, such as sensor readings or device states, to be processed or stored.
Authentication	The process of verifying the identity of a user or system to ensure that only authorized entities can access resources.
Event-Driven	A programming paradigm in which the flow of the program is determined by events, such as user actions, sensor data, or messages from other systems.
Digital Twin	A digital representation of a physical object, process, or system that uses data to reflect its state and behavior for monitoring, analysis, or simulation.
AI Agent	A software system that uses artificial intelligence to interpret goals, make decisions, and take actions, often by using tools or interacting with other systems.
Marimo Notebook	An interactive Python notebook built with marimo, where cells react to changes in their dependencies to keep results up to date.
API Key	A credential used to identify and authenticate an application or caller when accessing an API.
OAuth	An authorization framework that lets an application obtain limited access to a user's resources without sharing the user's password with that application.
Visual Algorithm	An image or video processing algorithm that analyzes visual data to detect, recognize, measure, or classify objects and patterns.
JSONB	A PostgreSQL data type that stores JSON data in a binary format, offering efficient indexing, faster querying, and improved performance compared to plain JSON fields.



TIER 0

tier0.app